

SOURCE CODE DEPOSIT COPY

Intellectual Property Office of the Philippines
Bureau of Copyright and Related Rights

Reference Number:	2026-06606
Title of Work:	MicroMeasure: A Web-Based Scientific Image Analysis and Measurement Tool
Applicant:	James Salveo L. Olarve
Affiliation:	i-Nano Research Facility, Department of Physics, De La Salle University Manila
Classification:	Class N (Computer Programs)
Programming Language(s):	HTML5 / CSS3 / JavaScript (client-side, single-file web application)
Software Version:	v1.3.0
Source File:	index.html

This deposit copy contains the complete, manually-authored source code of the above-titled computer program, submitted in fulfillment of the deposit requirement for works classified under Class N (Computer Programs). No redactions have been applied.

```

1 <!doctype html>
2 <html lang="en" class="h-full">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>MicroMeasure</title>
7 <script src="https://cdn.tailwindcss.com"></script>
8 <script>
9     function loadOptionalScript(src) {
10         return new Promise((resolve) => {
11             const script = document.createElement('script');
12             script.src = src;
13             script.async = true;
14             script.onload = () => resolve(true);
15             script.onerror = () => resolve(false);
16             document.head.appendChild(script);
17         });
18     }
19
20     const tiffDecoderReady = loadOptionalScript('https://cdn.jsdelivr.net/npm/utif@3.1.0/UTIF.min.js');
21
22     Promise.all([
23         tiffDecoderReady,
24         loadOptionalScript('./_sdk/element_sdk.js'),
25         loadOptionalScript('./_sdk/data_sdk.js')
26     ]);
27 </script>
28 <style>
29     body {
30         box-sizing: border-box;
31     }
32     @import
33     url('https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@400;500&family=Inter:wght@400;500;600;700&display=swap');
34
35     * { box-sizing: border-box; }
36
37     :root {
38         --bg-primary: #0f172a;
39         --bg-secondary: #1e293b;
40         --text-primary: #f1f5f9;
41         --accent-primary: #3b82f6;
42         --accent-secondary: #64748b;
43         --muted: #94a3b8;
44         --line: rgba(255,255,255,0.10);
45     }
46
47     body {
48         font-family: 'Inter', sans-serif;
49         background: var(--bg-primary);
50         color: var(--text-primary);
51     }
52
53     .mono { font-family: 'JetBrains Mono', monospace; }
54
55     .scrollbar-thin::-webkit-scrollbar { width: 6px; height: 6px; }
56     .scrollbar-thin::-webkit-scrollbar-track { background: var(--bg-secondary); }
57     .scrollbar-thin::-webkit-scrollbar-thumb { background: var(--accent-secondary); border-radius: 3px; }
58
59     .tab-btn { transition: all 0.2s ease; }
60     .tab-btn.active {
61         background: var(--accent-primary);
62         color: white;
63         box-shadow: 0 4px 12px rgba(59, 130, 246, 0.3);
64     }
65
66     .tool-card {
67         background: var(--bg-secondary);
68         border-radius: 12px;
69         border: 1px solid rgba(255,255,255,0.1);
70     }
71
72     .btn-primary {
73         background: var(--accent-primary);
74         transition: all 0.2s ease;
75     }
76     .btn-primary:hover {
77         background: #2563eb;
78         transform: translateY(-1px);
79         box-shadow: 0 4px 12px rgba(59, 130, 246, 0.4);
80     }

```

```

80 .btn-primary:disabled {
81     background: #475569;
82     cursor: not-allowed;
83     transform: none;
84     box-shadow: none;
85 }
86
87 .btn-save-image {
88     background: #16a34a;
89     transition: all 0.2s ease;
90 }
91 .btn-save-image:hover {
92     background: #15803d;
93     transform: translateY(-1px);
94     box-shadow: 0 4px 12px rgba(22, 163, 74, 0.4);
95 }
96 .btn-save-image:disabled {
97     background: #475569;
98     cursor: not-allowed;
99     transform: none;
100     box-shadow: none;
101 }
102
103 .btn-secondary {
104     background: var(--accent-secondary);
105     transition: all 0.2s ease;
106 }
107 .btn-secondary:hover { background: #475569; }
108
109 .btn-danger {
110     background: #dc2626;
111     transition: all 0.2s ease;
112 }
113 .btn-danger:hover { background: #b91c1c; }
114
115 input[type="range"] {
116     -webkit-appearance: none;
117     appearance: none;
118     background: transparent;
119 }
120 input[type="range"]::-webkit-slider-track {
121     height: 6px;
122     background: linear-gradient(to bottom, #1e293b, #334155, #1e293b);
123     border-radius: 3px;
124     border: 1px solid #64748b;
125     box-shadow: inset 0 1px 3px rgba(0,0,0,0.5), 0 1px 0 rgba(255,255,255,0.1);
126 }
127 input[type="range"]::-webkit-slider-thumb {
128     -webkit-appearance: none;
129     appearance: none;
130     width: 18px;
131     height: 18px;
132     background: linear-gradient(to bottom, #60a5fa, #3b82f6);
133     border-radius: 50%;
134     cursor: pointer;
135     margin-top: -6px;
136     border: 2px solid #1e293b;
137     box-shadow: 0 2px 6px rgba(59, 130, 246, 0.6), inset 0 1px 0 rgba(255,255,255,0.3);
138 }
139 input[type="range"]::-webkit-slider-thumb:hover {
140     background: linear-gradient(to bottom, #93c5fd, #60a5fa);
141     box-shadow: 0 3px 8px rgba(59, 130, 246, 0.8), inset 0 1px 0 rgba(255,255,255,0.4);
142 }
143
144 .canvas-container {
145     position: relative;
146     overflow: hidden;
147     background: #000;
148     border-radius: 8px;
149 }
150
151 /* Mobile performance & interaction hints for canvases */
152 #mainCanvas, #overlayCanvas {
153     will-change: transform;
154     touch-action: none;
155 }
156
157 .canvas-container.drag-over {
158     outline: 3px dashed #3b82f6;
159     outline-offset: -8px;
160     background: rgba(59, 130, 246, 0.1);

```

```

161     }
162
163     #mainCanvas, #overlayCanvas {
164         position: absolute;
165         top: 0;
166         left: 0;
167     }
168
169     .measurement-item {
170         background: rgba(0,0,0,0.3);
171         border-radius: 8px;
172         padding: 8px 12px;
173         margin-bottom: 6px;
174         display: flex;
175         justify-content: space-between;
176         align-items: center;
177         font-size: 13px;
178     }
179
180     .histogram-canvas {
181         background: var(--bg-primary);
182         border-radius: 8px;
183     }
184
185     .status-badge {
186         display: inline-flex;
187         align-items: center;
188         gap: 6px;
189         padding: 4px 10px;
190         border-radius: 20px;
191         font-size: 12px;
192         font-weight: 500;
193     }
194     .status-badge.warning { background: rgba(245, 158, 11, 0.2); color: #fbbf24; }
195     .status-badge.success { background: rgba(34, 197, 94, 0.2); color: #4ade80; }
196
197     .particle-table {
198         font-size: 11px;
199         width: 100%;
200     }
201     .particle-table th, .particle-table td {
202         padding: 6px 8px;
203         text-align: left;
204         border-bottom: 1px solid rgba(255,255,255,0.1);
205     }
206     .particle-table th {
207         background: rgba(0,0,0,0.3);
208         font-weight: 600;
209         position: sticky;
210         top: 0;
211     }
212
213     .processing-modal {
214         position: fixed;
215         top: 0;
216         left: 0;
217         right: 0;
218         bottom: 0;
219         background: rgba(0, 0, 0, 0.8);
220         backdrop-filter: blur(4px);
221         display: flex;
222         align-items: center;
223         justify-content: center;
224         z-index: 1000;
225         animation: fadeIn 0.2s ease;
226     }
227
228     /* Documentation overlay + modal (match Roughness Visualizer) */
229     .overlay{
230         position:fixed;
231         inset:0;
232         background:rgba(0,0,0,0.55);
233         display:flex;
234         align-items:flex-start;
235         justify-content:center;
236         padding:18px;
237         z-index:9999;
238         overflow:auto;
239     }
240     .overlay.hidden{ display:none !important; }
241     .modal{

```

```

242     width:min(720px, 100%);
243     max-height: calc(100dvh - 36px);
244     background:var(--bg-secondary);
245     border:1px solid rgba(255,255,255,0.12);
246     border-radius:16px;
247     box-shadow:0 24px 80px rgba(0,0,0,0.55);
248     display:flex;
249     flex-direction:column;
250 }
251 .modalHead{
252     display:flex;
253     align-items:center;
254     justify-content:space-between;
255     padding:12px 14px;
256     border-bottom:1px solid rgba(255,255,255,0.10);
257     gap:10px;
258     flex: 0 0 auto;
259 }
260 .modalTitle{ font-weight:950; }
261 .modalBody{
262     padding:14px;
263     color:var(--text-primary);
264     line-height:1.45;
265     overflow:auto;
266     -webkit-overflow-scrolling: touch;
267     flex: 1 1 auto;
268     min-height: 0;
269 }
270 .iconBtn{
271     width:32px;
272     height:32px;
273     border-radius:12px;
274     border:1px solid rgba(255,255,255,0.14);
275     background:rgba(255,255,255,0.06);
276     color:var(--text-primary);
277     cursor:pointer;
278     padding:0;
279     display:inline-flex;
280     align-items:center;
281     justify-content:center;
282     flex:0 0 auto;
283 }
284 .iconBtn:hover{ border-color:rgba(255,255,255,0.22); }
285
286 /* Neutralize Tailwind color blocks inside documentation so it matches Roughness docs */
287 .rvDoc{ color: var(--text-primary); }
288 .rvDoc .text-blue-400,
289 .rvDoc .text-blue-300,
290 .rvDoc .text-white,
291 .rvDoc .text-slate-300{ color: var(--text-primary) !important; }
292 .rvDoc .text-slate-400,
293 .rvDoc .text-slate-500,
294 .rvDoc .text-slate-600{ color: var(--muted) !important; }
295 .rvDoc a{ color: var(--accent-primary) !important; }
296 .rvDoc .bg-slate-900\50{ background: transparent !important; }
297 .rvDoc .border-slate-700{ border-color: var(--line) !important; }
298
299 .quickstart-modal {
300     position: fixed;
301     top: 0;
302     left: 0;
303     right: 0;
304     bottom: 0;
305     background: rgba(0, 0, 0, 0.55);
306     backdrop-filter: blur(2px);
307     display: flex;
308     align-items: center;
309     justify-content: center;
310     z-index: 900;
311     animation: fadeIn 0.2s ease;
312 }
313
314 .quickstart-content {
315     background: var(--bg-secondary);
316     border-radius: 16px;
317     padding: 20px;
318     width: min(420px, 92vw);
319     border: 1px solid rgba(255,255,255,0.1);
320     box-shadow: 0 20px 60px rgba(0,0,0,0.5);
321     animation: slideUp 0.3s ease;
322 }

```

```

323
324 .quickstart-step {
325     display: flex;
326     align-items: center;
327     gap: 10px;
328     padding: 10px 12px;
329     border-radius: 12px;
330     border: 1px solid rgba(255,255,255,0.08);
331     background: rgba(0,0,0,0.18);
332 }
333
334 .quickstart-step.current {
335     border-color: rgba(59, 130, 246, 0.45);
336 }
337
338 .quickstart-step.done {
339     opacity: 0.9;
340 }
341
342 .quickstart-check {
343     width: 18px;
344     height: 18px;
345     border-radius: 999px;
346     border: 2px solid rgba(148, 163, 184, 0.8);
347     flex: 0 0 auto;
348 }
349
350 .quickstart-step.done .quickstart-check {
351     border-color: rgba(34, 197, 94, 0.9);
352     background: rgba(34, 197, 94, 0.25);
353 }
354
355 @keyframes fadeIn {
356     from { opacity: 0; }
357     to { opacity: 1; }
358 }
359
360 .processing-content {
361     background: var(--bg-secondary);
362     border-radius: 16px;
363     padding: 32px;
364     max-width: 400px;
365     width: 90%;
366     border: 1px solid rgba(255,255,255,0.1);
367     box-shadow: 0 20px 60px rgba(0,0,0,0.5);
368     animation: slideUp 0.3s ease;
369 }
370
371 @keyframes slideUp {
372     from { transform: translateY(20px); opacity: 0; }
373     to { transform: translateY(0); opacity: 1; }
374 }
375
376 .spinner {
377     width: 48px;
378     height: 48px;
379     border: 4px solid rgba(59, 130, 246, 0.2);
380     border-top-color: #3b82f6;
381     border-radius: 50%;
382     animation: spin 0.8s linear infinite;
383     margin: 0 auto 20px;
384 }
385
386 @keyframes spin {
387     to { transform: rotate(360deg); }
388 }
389
390 .progress-bar-container {
391     width: 100%;
392     height: 8px;
393     background: rgba(0,0,0,0.3);
394     border-radius: 4px;
395     overflow: hidden;
396     margin-top: 16px;
397 }
398
399 .progress-bar {
400     height: 100%;
401     background: linear-gradient(90deg, #3b82f6, #60a5fa);
402     border-radius: 4px;
403     transition: width 0.3s ease;

```

```

404     box-shadow: 0 0 10px rgba(59, 130, 246, 0.5);
405 }
406 </style>
407 <style>@view-transition { navigation: auto; }</style>
408 </head>
409 <body class="h-full overflow-auto">
410   <div class="min-h-[100svh] flex flex-col"><!-- Header -->
411     <header class="bg-slate-800/80 backdrop-blur border-b border-slate-700 px-4 py-3 flex items-center justify-between flex-shrink-0">
412       <div class="flex items-center gap-3">
413         <div class="w-10 h-10 rounded-xl bg-gradient-to-br from-blue-500 to-cyan-400 flex items-center justify-center">
414           <svg class="w-6 h-6 text-white" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-linecap="round"
415             stroke-linejoin="round" stroke-width="2" d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0zM10 7v3m0 0v3m0-3h3m-3 0H7" />
416         </div>
417         <div>
418           <h1 id="appTitle" class="text-lg font-bold text-white">MicroMeasure</h1>
419           <p class="text-xs text-slate-400">Image Analysis</p>
420         </div>
421       </div>
422       <div class="flex items-center gap-3">
423         <button id="homeBtn" class="btn-secondary w-10 h-10 rounded-lg text-sm font-medium inline-flex items-center justify-cent
424           title="Go to i-Nano Research Facility home page" aria-label="Go to i-Nano Research Facility home page">
425           <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
426             <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M3 10.5 12 3l9 7.5"/>
427             <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M5.25 9.75V21h13.5V9.75"/>
428             <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M9.75 21v-6h4.5v6"/>
429           </svg>
430         </button>
431         <button id="loadDemoBtn" class="btn-secondary w-10 h-10 rounded-lg text-sm font-medium inline-flex items-center
432           justify-center" title="Load a demo photo" aria-label="Load a demo photo">
433           <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
434             <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4 16l4.586-4.586a2 2 0 0 1 2.828 0L16 16"/>
435             <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M14 14l1.586-1.586a2 2 0 0 1 2.828 0L20 14"/>
436             <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4 6h16v12H4z"/>
437             <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M8.5 10.5h.01"/>
438           </svg>
439         </button>
440         <button id="headerResetBtn" class="btn-danger w-10 h-10 rounded-lg text-sm font-medium inline-flex items-center
441           justify-center" title="Reset MicroMeasure" aria-label="Reset MicroMeasure">
442           <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
443             <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4.5 12a7.5 7.5 0 0 1 12.78-5.303"/>
444             <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M19.5 12a7.5 7.5 0 0 1 -12.78 5.303"/>
445             <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M16.5 6.75V3.75h-3"/>
446             <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M7.5 17.25v3h3"/>
447           </svg>
448         </button>
449         <button id="helpBtn" class="btn-secondary w-10 h-10 rounded-lg text-sm font-medium inline-flex items-center justify-cente
450           title="Documentation" aria-label="Documentation">
451           <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-linecap="round"
452             stroke-linejoin="round" stroke-width="2" d="M8.228 9c.549-1.165 2.03-2 3.772-2 2.21 0 4 1.343 4 3 0 1.4-1.278 2.575-3.0
453             2.907-.542 1.04-.994 1.093m0 3h.01M21 12a9 9 0 11-18 0 9 9 0 0118 0z" />
454         </svg></button> <button id="saveScreenshot" class="btn-save-image w-10 h-10 rounded-lg text-sm font-medium inline-flex
455           items-center justify-center" title="Save Image" aria-label="Save Image" disabled>
456         <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-linecap="round"
457             stroke-linejoin="round" stroke-width="2" d="M3 9a2 2 0 012-2h.93a2 2 0 001.664-.89l.812-1.22A2 2 0 0110.07 4h3.86a2 2 0
458             011.664.89l.812 1.22A2 2 0 0018.07 7H19a2 2 0 012 2v9a2 2 0 01-2 2V9z" /> <path stroke-linecap="round"
459             stroke-linejoin="round" stroke-width="2" d="M15 13a3 3 0 11-6 0 3 3 0 016 0z" />
460         </svg></button>
461       </div>
462     </header><!-- Main Content -->
463     <div class="flex-1 flex flex-col lg:flex-row gap-4 p-4 overflow-auto"><!-- Left Panel - Image Viewer -->
464       <div class="lg:w-2/3 flex flex-col gap-3 min-h-0 flex-1"><!-- Image Controls -->
465         <div class="tool-card p-3 flex flex-wrap items-center gap-4"><label class="btn-primary px-4 py-2 rounded-lg cursor-point
466           text-sm font-medium flex items-center gap-2">
467           <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-linecap="round"
468             stroke-linejoin="round" stroke-width="2" d="M4 16l4.586-4.586a2 2 0 012.828 0L16 16m-2-2l1.586-1.586a2 2 0 012.828 0L20
469             14m-6-6h.01M6 20h12a2 2 0 002-2V6a2 2 0 00-2-2H6a2 2 0 00-2 2v12a2 2 0 002 2z" />
470           </svg> Load Image <input type="file" id="imageInput" accept="image/*,.tif,.tiff" class="hidden"> </label>
471         <div class="flex items-center gap-3 bg-slate-900/30 rounded-lg px-3 py-2 border border-slate-700/50"><span class="text-
472           text-slate-400">Zoom:</span>
473         <div class="bg-slate-800 rounded px-3 py-1.5 flex items-center gap-2"><input type="range" id="zoomSlider" min="50"
474           max="500" value="100" class="w-32"> <span id="zoomValue" class="text-xs mono w-10">100%</span>
475         </div>
476         <div id="imageDimensions" class="text-xs text-slate-400 mono hidden"><span id="imgWidth">0</span> × <span
477           id="imgHeight">0</span> px
478         </div><button id="fitToView" class="btn-secondary px-3 py-1.5 rounded-lg text-xs hidden">Fit to View</button> <!-- Scal
479         Calibration Section -->
480         <div class="flex items-center gap-2 bg-slate-900/30 rounded-lg px-3 py-2 border border-slate-700/50">
481           <svg class="w-4 h-4 text-blue-400 flex-shrink-0" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path

```

```

stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M19 11H5m14 0a2 2 0 012 2v6a2 2 0 01-2 2H5a2 2 0
01-2-2v-6a2 2 0 012-2m14 0V9a2 2 0 00-2-2M5 11V9a2 2 0 012-2m0 0V5a2 2 0 012-2h6a2 2 0 012 2v2M7 7h10" />
466 </svg><span class="text-xs text-slate-400 whitespace-nowrap">Scale:</span> <button id="setScaleBtn" class="btn-primary
px-3 py-1.5 rounded-lg text-xs font-medium disabled:opacity-50 whitespace-nowrap" disabled> Set Scale </button>
467 <div id="scaleInfo" class="hidden flex items-center gap-2"><span id="scaleValue" class="mono text-green-400 text-xs
whitespace-nowrap">-</span> <button id="resetScaleBtn" class="btn-secondary px-2 py-1 rounded-lg text-xs" title="Reset
Scale">Reset</button>
468 </div>
469 </div>
470 <div class="flex items-center gap-3 bg-slate-900/30 rounded-lg px-3 py-2 border border-slate-700/50 ml-auto hidden"
id="panControls"><span class="text-xs text-slate-400">Pan:</span>
471 <div class="flex items-center gap-1"><button id="panLeft" class="btn-secondary px-2 py-1 rounded text-xs" title="Pan
Left"></button>
472 <div class="flex flex-col gap-1"><button id="panUp" class="btn-secondary px-2 py-0.5 rounded text-xs leading-none"
title="Pan Up">↑</button> <button id="panDown" class="btn-secondary px-2 py-0.5 rounded text-xs leading-none" title="Pa
Down">↓</button>
473 </div><button id="panRight" class="btn-secondary px-2 py-1 rounded text-xs" title="Pan Right">→</button>
474 </div><button id="resetPan" class="btn-secondary px-2 py-1 rounded text-xs" title="Reset Pan">■</button>
475 </div>
476 </div><!-- Calibration Inputs Panel -->
477 <div id="calibrationInputs" class="hidden tool-card p-3">
478 <div class="grid grid-cols-2 gap-3">
479 <div><label class="text-xs text-slate-400 block mb-1">Known Length</label> <input type="number" id="knownLength"
step="any" min="0" class="w-full bg-slate-900 border border-slate-600 rounded-lg px-3 py-2 text-sm" placeholder="e.g.,
100">
480 </div>
481 <div><label class="text-xs text-slate-400 block mb-1">Unit</label> <select id="unitSelect" class="w-full bg-slate-900
border border-slate-600 rounded-lg px-3 py-2 text-sm"> <option value="nm">nm</option> <option value="µm"
selected>µm</option> <option value="mm">mm</option> </select>
482 </div>
483 <div class="col-span-2 flex gap-2"><button id="saveScale" class="btn-primary px-4 py-2 rounded-lg text-sm flex-1">Save
Scale</button> <button id="cancelScale" class="btn-secondary px-4 py-2 rounded-lg text-sm">Cancel</button>
484 </div>
485 </div>
486 </div><!-- Canvas Container -->
487 <div class="flex-1 tool-card overflow-hidden min-h-[60vh] lg:min-h-0">
488 <div id="canvasWrapper" class="canvas-container w-full min-h-[60vh] lg:h-full flex items-center justify-center">
489 <div id="noImagePlaceholder" class="text-center p-8 pointer-events-none">
490 <svg class="w-16 h-16 mx-auto text-slate-600 mb-4" fill="none" stroke="currentColor" viewBox="0 0 24 24"><path
stroke-linecap="round" stroke-linejoin="round" stroke-width="1.5" d="M4 16l4.586-4.586a2 2 0 012.828 0L16
16m-2-2l1.586-1.586a2 2 0 012.828 0L20 14m-6-6h.01M6 20h12a2 2 0 002-2V6a2 2 0 00-2-2h12a2 2 0 002 2z" /
491 </svg>
492 <p class="text-slate-500 text-sm">Load an image to begin analysis</p>
493 <p class="text-slate-600 text-xs mt-1">Supports TIFF (first page), PNG, JPG, WebP, and other browser-supported image
formats</p>
494 <p class="text-slate-500 text-xs mt-3">↓ Drag & drop an image here</p>
495 </div>
496 <canvas id="mainCanvas" class="hidden"></canvas>
497 <canvas id="overlayCanvas" class="hidden"></canvas>
498 </div>
499 </div>
500 </div><!-- Right Panel - Tools -->
501 <div class="lg:w-1/3 flex flex-col gap-3 min-h-0"><!-- Tool Tabs -->
502 <div class="tool-card p-2 flex gap-1 flex-wrap"><button class="tab-btn active px-3 py-2 rounded-lg text-xs font-medium
flex-1 min-w-[60px]" data-tab="length">Length</button> <button class="tab-btn px-3 py-2 rounded-lg text-xs font-medium
flex-1 min-w-[60px]" data-tab="angle">Angle</button> <button class="tab-btn px-3 py-2 rounded-lg text-xs font-medium
flex-1 min-w-[60px]" data-tab="area">Area</button> <button class="tab-btn px-3 py-2 rounded-lg text-xs font-medium flex
min-w-[60px]" data-tab="particles">Particles</button> <button class="tab-btn px-3 py-2 rounded-lg text-xs font-medium
flex-1 min-w-[60px]" data-tab="holes">Holes</button>
503 </div><!-- Tool Content -->
504 <div class="tool-card flex-1 p-4 overflow-y-auto scrollbar-thin min-h-[200px]"><!-- Length Tab -->
505 <div id="lengthTab" class="tab-content">
506 <p class="text-xs text-slate-400 mb-3">Click two points on the image to measure distance.</p>
507 <div id="editLengthModeControls" class="hidden bg-yellow-500/20 border border-yellow-500/50 rounded-lg p-3 mb-3">
508 <p class="text-xs text-yellow-300 mb-2 font-medium">■ Editing Length <span id="editingLengthLabel"></span></p>
509 <p class="text-xs text-slate-400 mb-3">Drag the endpoints to adjust the line.</p>
510 <div class="flex gap-2"><button id="saveLengthEdit" class="btn-primary px-3 py-1.5 rounded-lg text-xs flex-1">Save
Changes</button> <button id="cancelLengthEdit" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1">Cancel</but
511 </div>
512 </div>
513 <div id="lengthMeasurements" class="scrollbar-thin mb-3"></div>
514 <div class="flex gap-2">
515 <button id="lengthStatsBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Descriptive
Statistics</button>
516 <button id="lengthHistBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Histogram</button>
517 <button id="clearLengths" class="btn-danger px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Clear All Lengths</button>
518 </div>
519 <div id="lengthExportRow" class="flex gap-2 mt-2 hidden">
520 <button id="copyLengthTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1" disabled>Copy TSV</button>
521 <button id="downloadLengthTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1" disabled>Download
CSV</button>

```



```

522     </div>
523 </div><!-- Angle Tab -->
524 <div id="angleTab" class="tab-content hidden">
525   <p class="text-xs text-slate-400 mb-3">Click three points on the image to measure an angle. The second point is the
vertex (middle point).</p>
526   <div id="editAngleModeControls" class="hidden bg-yellow-500/20 border border-yellow-500/50 rounded-lg p-3 mb-3">
527     <p class="text-xs text-yellow-300 mb-2 font-medium">■ Editing Angle <span id="editingAngleLabel"></span></p>
528     <p class="text-xs text-slate-400 mb-3">Drag any of the three points to adjust the angle.</p>
529     <div class="flex gap-2"><button id="saveAngleEdit" class="btn-primary px-3 py-1.5 rounded-lg text-xs flex-1">Save
Changes</button> <button id="cancelAngleEdit" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1">Cancel</button>
530   </div>
531 </div>
532 <div id="angleMeasurements" class="scrollbar-thin mb-3"></div>
533 <div class="flex gap-2">
534   <button id="angleStatsBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Descriptive
Statistics</button>
535   <button id="angleHistBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Histogram</button>
536   <button id="clearAngles" class="btn-danger px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Clear All Angles</button>
537 </div>
538 <div id="angleExportRow" class="flex gap-2 mt-2 hidden">
539   <button id="copyAngleTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1" disabled>Copy TSV</button>
540   <button id="downloadAngleTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1" disabled>Download
CSV</button>
541 </div>
542 </div><!-- Area Tab -->
543 <div id="areaTab" class="tab-content hidden">
544   <div class="flex gap-2 mb-3">
545     <button id="areaModePolygonBtn" class="btn-primary px-3 py-1.5 rounded-lg text-xs flex-1"
type="button">Polygon</button>
546     <button id="areaModeCircleBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1"
type="button">Circle</button>
547   </div>
548   <p id="areaInstructions" class="text-xs text-slate-400 mb-3">Polygon: click points to create polygon.</p>
549   <button id="closePolygonBtn" class="hidden btn-primary px-4 py-2 rounded-lg text-sm w-full mb-3">Close Polygon (3+
points)</button>
550   <div id="editModeControls" class="hidden bg-yellow-500/20 border border-yellow-500/50 rounded-lg p-3 mb-3">
551     <p class="text-xs text-yellow-300 mb-2 font-medium">■ Editing Area <span id="editingAreaLabel"></span></p>
552     <p class="text-xs text-slate-400 mb-3">Drag points to adjust shape. Click edge to add point. Right-click point to
delete.</p>
553     <div class="flex gap-2"><button id="saveAreaEdit" class="btn-primary px-3 py-1.5 rounded-lg text-xs flex-1">Save
Changes</button> <button id="cancelAreaEdit" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1">Cancel</button>
554   </div>
555 </div>
556 <div id="areaMeasurements" class="scrollbar-thin mb-3"></div>
557 <div class="flex gap-2">
558   <button id="areaStatsBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Descriptive
Statistics</button>
559   <button id="areaHistBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Histogram</button>
560   <button id="clearAreas" class="btn-danger px-3 py-1.5 rounded-lg text-xs flex-1 hidden">Clear All Areas</button>
561 </div>
562 <div id="areaExportRow" class="flex gap-2 mt-2 hidden">
563   <button id="copyAreaTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1" disabled>Copy TSV</button>
564   <button id="downloadAreaTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1" disabled>Download CSV</button>
565 </div>
566 </div><!-- Particles Tab -->
567 <div id="particlesTab" class="tab-content hidden">
568   <div class="space-y-3"><!-- ROI Selection -->
569     <div class="bg-slate-900/50 rounded-lg p-3 border border-slate-700">
570       <h4 class="text-xs font-semibold mb-2 text-blue-400">Region of Interest</h4>
571       <div id="particleROIControls"><button id="selectParticleROI" class="btn-secondary px-3 py-1.5 rounded-lg text-xs w-f
mb-2 disabled:opacity-50" disabled> ■ Select ROI (Draw Rectangle) </button>
572       <div id="particleROIInfo" class="hidden">
573         <div class="text-xs text-slate-400 mb-2"><span class="font-medium text-green-400">ROI Active:</span> <span
id="particleROIDimensions" class="mono ml-1"></span>
574       </div><button id="clearParticleROI" class="btn-danger px-2 py-1 rounded-lg text-xs w-full">Clear ROI (Use Full
Image)</button>
575     </div>
576   </div>
577 </div><!-- Preprocessing Section -->
578 <div class="bg-slate-900/50 rounded-lg p-3 border border-slate-700">
579   <h4 class="text-xs font-semibold mb-2 text-blue-400">Preprocessing</h4><label class="flex items-center gap-2 text-xs
cursor-pointer mb-2"> <input type="checkbox" id="backgroundSubtraction" class="accent-blue-500"> Background subtraction
</label>
580   <div id="backgroundOptions" class="ml-5 mb-2 hidden"><label class="text-xs text-slate-400">Rolling ball radius
(px)</label> <input type="number" id="rollingBallRadius" value="50" min="10" max="200" step="10" class="w-full
bg-slate-900 border border-slate-600 rounded-lg px-2 py-1 text-xs mt-1">
581 </div>
582   <div class="mb-2"><label class="text-xs text-slate-400 block mb-1">Smoothing</label> <select id="smoothingMethod"
class="w-full bg-slate-900 border border-slate-600 rounded-lg px-2 py-1.5 text-xs"> <option value="none">None</option>
<option value="gaussian3">Gaussian 3×3</option> <option value="median3">Median 3×3</option> <option value="median5">Med
5×5</option> </select>

```

```

583     </div><label class="flex items-center gap-2 text-xs cursor-pointer"> <input type="checkbox" id="morphologicalOpening
class="accent-blue-500"> Morphological opening (remove small objects) </label>
584 </div><!-- Thresholding Section -->
585 <div class="bg-slate-900/50 rounded-lg p-3 border border-slate-700">
586   <h4 class="text-xs font-semibold mb-2 text-blue-400">Thresholding</h4>
587   <div class="flex items-center gap-2 mb-2"><button id="autoThresholdBtn" class="btn-secondary px-3 py-1.5 rounded-lg
text-xs flex-1">Auto (Otsu)</button> <label class="flex items-center gap-2 text-xs cursor-pointer"> <input type="checkbox"
id="adaptiveThreshold" class="accent-blue-500"> Adaptive </label>
588   </div>
589   <div id="manualThresholdControls"><label class="text-xs text-slate-400 flex justify-between"> <span>Manual
Threshold</span> <span id="thresholdValue" class="mono">128</span> </label> <input type="range" id="particleThreshold"
min="0" max="255" value="128" class="w-full mt-1">
590   </div>
591   <div id="adaptiveThresholdControls" class="hidden mt-2"><label class="text-xs text-slate-400">Block size (px)</label>
<input type="number" id="adaptiveBlockSize" value="32" min="8" max="128" step="8" class="w-full bg-slate-900 border
border-slate-600 rounded-lg px-2 py-1 text-xs mt-1">
592   </div>
593   </div>
594   <div class="flex items-center gap-2"><label class="flex items-center gap-2 text-xs cursor-pointer"> <input type="radio"
name="particlePolarity" value="bright" checked class="accent-blue-500"> Particles are bright </label> <label class="flex
items-center gap-2 text-xs cursor-pointer"> <input type="radio" name="particlePolarity" value="dark"
class="accent-blue-500"> Particles are dark </label>
595   </div>
596   <div><label class="text-xs text-slate-400" id="minParticleAreaLabel">Min Area (px2)</label> <input type="number"
id="minParticleArea" value="10" min="0.001" step="any" class="w-full bg-slate-900 border border-slate-600 rounded-lg px-2
py-2 text-sm mt-1" disabled>
597   <p class="text-xs text-slate-500 mt-1">Set scale first to enable</p>
598   </div><button id="analyzeParticles" class="btn-primary px-4 py-2 rounded-lg text-sm w-full disabled:opacity-50"
disabled>Analyze Particles</button>
599   <div id="particleResults" class="hidden">
600     <div class="bg-slate-900/50 rounded-lg p-3 mt-3">
601       <h4 class="text-xs font-semibold mb-2">Summary Statistics</h4>
602       <div class="grid grid-cols-2 gap-2 text-xs">
603         <div>
604           <span class="text-slate-400">Count:</span> <span id="particleCount" class="mono">-</span>
605         </div>
606         <div>
607           <span class="text-slate-400">Mean Area:</span> <span id="particleMeanArea" class="mono">-</span>
608         </div>
609         <div>
610           <span class="text-slate-400">Mean ECD:</span> <span id="particleMeanECD" class="mono">-</span>
611         </div>
612         <div>
613           <span class="text-slate-400">Std Dev:</span> <span id="particleStdDev" class="mono">-</span>
614         </div>
615       </div>
616     </div>
617     <div class="mt-3">
618       <h4 class="text-xs font-semibold mb-2">ECD Histogram</h4>
619       <canvas id="particleHistogram" class="histogram-canvas w-full h-24"></canvas>
620     </div>
621   </div>
622   <div class="flex gap-2 flex-wrap">
623     <button id="particleStatsBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[120px]
hidden">Descriptive Statistics</button>
624     <button id="particleHistBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[90px]
hidden">Histogram</button>
625     <button id="clearParticles" class="btn-danger px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[110px] hidden">Clear
Particles</button>
626   </div>
627   <div id="particlesExportRow" class="flex gap-2 flex-wrap hidden">
628     <button id="copyParticlesTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[90px]" disabled>C
TSV</button>
629     <button id="downloadParticlesTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[110px]"
disabled>Download CSV</button>
630   </div>
631 </div>
632 </div><!-- Holes Tab -->
633 <div id="holesTab" class="tab-content hidden">
634   <div class="space-y-3"><!-- ROI Selection -->
635     <div class="bg-slate-900/50 rounded-lg p-3 border border-slate-700">
636       <h4 class="text-xs font-semibold mb-2 text-blue-400">Region of Interest</h4>
637       <div id="holeROIControls"><button id="selectHoleROI" class="btn-secondary px-3 py-1.5 rounded-lg text-xs w-full mb-2
disabled:opacity-50" disabled> ■ Select ROI (Draw Rectangle) </button>
638       <div id="holeROIInfo" class="hidden">
639         <div class="text-xs text-slate-400 mb-2"><span class="font-medium text-green-400">ROI Active:</span> <span
id="holeROIDimensions" class="mono ml-1"></span>
640       </div><button id="clearHoleROI" class="btn-danger px-2 py-1 rounded-lg text-xs w-full">Clear ROI (Use Full
Image)</button>
641     </div>
642   </div>

```

```

643     </div><!-- Preprocessing Section -->
644     <div class="bg-slate-900/50 rounded-lg p-3 border border-slate-700">
645         <h4 class="text-xs font-semibold mb-2 text-blue-400">Preprocessing</h4><label class="flex items-center gap-2 text-xs
cursor-pointer mb-2"> <input type="checkbox" id="holeBackgroundSubtraction" class="accent-blue-500"> Background
subtraction </label>
646         <div id="holeBackgroundOptions" class="ml-5 mb-2 hidden"><label class="text-xs text-slate-400">Rolling ball radius
(px)</label> <input type="number" id="holeRollingBallRadius" value="50" min="10" max="200" step="10" class="w-full
bg-slate-900 border border-slate-600 rounded-lg px-2 py-1 text-xs mt-1">
647         </div>
648         <div class="mb-2"><label class="text-xs text-slate-400 block mb-1">Smoothing</label> <select id="holeSmoothingMethod
class="w-full bg-slate-900 border border-slate-600 rounded-lg px-2 py-1.5 text-xs"> <option value="none">None</option>
<option value="gaussian3">Gaussian 3x3</option> <option value="median3">Median 3x3</option> <option value="median5">Med
5x5</option> </select>
649         </div><label class="flex items-center gap-2 text-xs cursor-pointer"> <input type="checkbox"
id="holeMorphologicalOpening" class="accent-blue-500"> Morphological opening (remove small objects) </label>
650     </div><!-- Thresholding Section -->
651     <div class="bg-slate-900/50 rounded-lg p-3 border border-slate-700">
652         <h4 class="text-xs font-semibold mb-2 text-blue-400">Thresholding</h4>
653         <div class="flex items-center gap-2 mb-2"><button id="autoThresholdHoleBtn" class="btn-secondary px-3 py-1.5 rounded
text-xs flex-1">Auto (Otsu)</button> <label class="flex items-center gap-2 text-xs cursor-pointer"> <input type="checkbox"
id="adaptiveThresholdHole" class="accent-blue-500"> Adaptive </label>
654         </div>
655         <div id="manualThresholdHoleControls"><label class="text-xs text-slate-400 flex justify-between"> <span>Manual
Threshold</span> <span id="holeThresholdValue" class="mono">128</span> </label> <input type="range" id="holeThreshold"
min="0" max="255" value="128" class="w-full mt-1">
656         </div>
657         <div id="adaptiveThresholdHoleControls" class="hidden mt-2"><label class="text-xs text-slate-400">Block size
(px)</label> <input type="number" id="holeAdaptiveBlockSize" value="32" min="8" max="128" step="8" class="w-full
bg-slate-900 border border-slate-600 rounded-lg px-2 py-1 text-xs mt-1">
658         </div>
659         </div>
660         <div class="flex items-center gap-2"><label class="flex items-center gap-2 text-xs cursor-pointer"> <input type="radio"
name="holePolarity" value="dark" checked class="accent-blue-500"> Holes are dark </label> <label class="flex items-cent
gap-2 text-xs cursor-pointer"> <input type="radio" name="holePolarity" value="bright" class="accent-blue-500"> Holes ar
bright </label>
661         </div>
662         <div><label class="text-xs text-slate-400" id="minHoleAreaLabel">Min Area (px2)</label> <input type="number"
id="minHoleArea" value="10" min="0.001" step="any" class="w-full bg-slate-900 border border-slate-600 rounded-lg px-3 p
text-sm mt-1" disabled>
663         <p class="text-xs text-slate-500 mt-1">Set scale first to enable</p>
664         </div><button id="analyzeHoles" class="btn-primary px-4 py-2 rounded-lg text-sm w-full disabled:opacity-50"
disabled>Analyze Holes</button>
665         <div id="holeResults" class="hidden">
666             <div class="bg-slate-900/50 rounded-lg p-3 mt-3">
667                 <h4 class="text-xs font-semibold mb-2">Summary Statistics</h4>
668                 <div class="grid grid-cols-2 gap-2 text-xs">
669                     <div>
670                         <span class="text-slate-400">Count:</span> <span id="holeCount" class="mono">-</span>
671                     </div>
672                     <div>
673                         <span class="text-slate-400">Mean Area:</span> <span id="holeMeanArea" class="mono">-</span>
674                     </div>
675                     <div>
676                         <span class="text-slate-400">Mean ECD:</span> <span id="holeMeanECD" class="mono">-</span>
677                     </div>
678                     <div>
679                         <span class="text-slate-400">Std Dev:</span> <span id="holeStdDev" class="mono">-</span>
680                     </div>
681                 </div>
682             </div>
683             <div class="mt-3">
684                 <h4 class="text-xs font-semibold mb-2">ECD Histogram</h4>
685                 <canvas id="holeHistogram" class="histogram-canvas w-full h-24"></canvas>
686             </div>
687         </div>
688         <div class="flex gap-2 flex-wrap">
689             <button id="holeStatsBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[120px] hidden">Descripti
Statistics</button>
690             <button id="holeHistBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[90px]
hidden">Histogram</button>
691             <button id="clearHoles" class="btn-danger px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[95px] hidden">Clear
Holes</button>
692         </div>
693         <div id="holesExportRow" class="flex gap-2 flex-wrap hidden">
694             <button id="copyHolesTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[90px]" disabled>Copy
TSV</button>
695             <button id="downloadHolesTSVBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs flex-1 min-w-[110px]"
disabled>Download CSV</button>
696         </div>
697     </div>
698 </div>

```

```

699     </div>
700 </div>
701 </div>
702 </div>
703 <!-- Quick Start Overlay -->
704 <div id="quickStartOverlay" class="quickstart-modal hidden" role="dialog" aria-modal="true" aria-label="Quick Start">
705   <div class="quickstart-content">
706     <h3 class="text-lg font-semibold text-white mb-3">Quick Start</h3>
707     <ol class="space-y-2 text-sm text-slate-300">
708       <li id="quickStartStep1" class="quickstart-step">
709         <span class="quickstart-check" aria-hidden="true"></span>
710         <span>1. Load Image</span>
711       </li>
712       <li id="quickStartStep2" class="quickstart-step">
713         <span class="quickstart-check" aria-hidden="true"></span>
714         <span>2. Set Scale</span>
715       </li>
716       <li id="quickStartStep3" class="quickstart-step">
717         <span class="quickstart-check" aria-hidden="true"></span>
718         <span>3. Start measurement</span>
719       </li>
720     </ol>
721     <div class="mt-4">
722       <button id="quickStartOk" class="btn-primary px-6 py-2 rounded-lg text-sm w-full">OK</button>
723     </div>
724   </div>
725 </div><!-- Processing Modal -->
726 <div id="processingModal" class="processing-modal hidden">
727   <div class="processing-content">
728     <div class="spinner"></div>
729     <h3 id="processingTitle" class="text-lg font-semibold text-center mb-2">Processing Image</h3>
730     <p id="processingStep" class="text-sm text-slate-400 text-center mb-1">Initializing...</p>
731     <p id="processingTime" class="text-xs text-slate-500 text-center mono">Estimating time...</p>
732     <div class="progress-bar-container">
733       <div id="progressBar" class="progress-bar" style="width: 0%></div>
734     </div>
735   </div>
736 </div><!-- Documentation Modal -->
737 <div id="docModal" class="overlay hidden">
738   <div class="modal" role="dialog" aria-modal="true" aria-labelledby="docTitle">
739     <div class="modalHead">
740       <div id="docTitle" class="modalTitle">MicroMeasure Documentation</div>
741       <button id="closeDocBtn" class="iconBtn" type="button" aria-label="Close documentation">X</button>
742     </div>
743     <div class="modalBody rvDoc"><!-- Project Information -->
744     <section>
745       <h3 class="text-xl font-bold text-blue-400 mb-3">Web-Based Image Analysis and Measurement Tool</h3>
746       <div class="bg-slate-900/50 rounded-lg p-4 border border-slate-700 space-y-2">
747         <p><strong>Version:</strong> 1.2.0</p>
748         <p><strong>Website:</strong> <a href="https://www.inanolab.com/micromesasure.html" target="_blank" rel="noopener noreferrer" class="text-blue-400 hover:text-blue-300 underline">https://www.inanolab.com/micromesasure.html</a></p>
749         <p><strong>Developer / Programmer:</strong> Dr. James Salveo Olarve</p>
750         <p><strong>Affiliation:</strong> i-Nano Research Facility, De La Salle University Manila</p>
751       </div>
752     </section><!-- Citation Recommendation -->
753     <section>
754       <h3 class="text-lg font-semibold text-white mb-2">Citation Recommendation</h3>
755       <p class="text-slate-300 mb-2">If you use MicroMeasure in academic work:</p>
756       <div class="bg-slate-900/50 rounded-lg p-4 border border-slate-700">
757         <p class="text-slate-300 text-sm font-mono leading-relaxed">Olarve, J. S. (2026). MicroMeasure: A Web-Based Image Measurement Tool (v1.2.0). Zenodo. https://doi.org/10.5281/zenodo.19418861</p>
758       </div>
759     </section><!-- Overview -->
760     <section>
761       <h3 class="text-lg font-semibold text-white mb-2">Overview</h3>
762       <p class="text-slate-300 mb-2">MicroMeasure is a browser-based image analysis application designed for quantitative measurements of scientific images, particularly microscopy images such as scanning electron microscopy (SEM) micrograph. The application provides intuitive tools for measuring length, angle, area, particle size, and hole size, without requiring software installation.</p>
763       <p class="text-slate-300">MicroMeasure is inspired by widely used scientific image analysis platforms (e.g., ImageJ) but optimized for fast, web-based workflows and educational use.</p>
764     </section><!-- Intended Users -->
765     <section>
766       <h3 class="text-lg font-semibold text-white mb-2">Intended Users</h3>
767       <p class="text-slate-300 mb-2">MicroMeasure is intended for:</p>
768       <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
769         <li>Researchers and laboratory personnel</li>
770         <li>Graduate and undergraduate students</li>
771         <li>Educators and instructors</li>
772         <li>Materials science, physics, chemistry, biology, and engineering users</li>
773       </ul>

```

```

774     </section><!-- Quick Start -->
775     <section>
776         <h3 class="text-lg font-semibold text-white mb-2">Quick Start</h3>
777         <ol class="list-decimal list-inside space-y-2 text-slate-300">
778             <li><strong>Load an image</strong> - Click "Load Image" or drag & drop an image file</li>
779             <li><strong>Set the scale</strong> - Click "Set Scale", draw a line of known length, then enter the real-world
780             measurement</li>
781             <li><strong>Make measurements</strong> - Use the tabs to measure lengths, angles, areas, particles, or holes</li>
782             <li><strong>Export results</strong> - Use the <strong>Copy TSV</strong> / <strong>Download CSV</strong> buttons at the
783             bottom of each measurement tab</li>
784         </ol>
785     </section><!-- Scale Calibration -->
786     <section>
787         <h3 class="text-lg font-semibold text-white mb-2">Scale Calibration</h3>
788         <p class="text-slate-300 mb-2">Before making real-world measurements, you must calibrate the scale:</p>
789         <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
790             <li>Click the <strong>Set Scale</strong> button in the image controls</li>
791             <li>Click two points on a feature of known length (e.g., a scale bar)</li>
792             <li>Enter the known length and select the unit (nm,  $\mu$ m, mm)</li>
793             <li>Click <strong>Save Scale</strong> to apply calibration</li>
794             <li>All measurements will now show both pixel and real-world values</li>
795             <li>Use <strong>Reset</strong> to clear calibration if needed</li>
796         </ul>
797     </section><!-- Length Measurements -->
798     <section>
799         <h3 class="text-lg font-semibold text-white mb-2">Length Measurements</h3>
800         <p class="text-slate-300 mb-2">Measure distances between two points:</p>
801         <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
802             <li>Switch to the <strong>Length</strong> tab</li>
803             <li>Click two points on the image to create a measurement</li>
804             <li>Each measurement is labeled (L1, L2, etc.)</li>
805             <li>Use the <strong>Edit (—■)</strong> button to adjust endpoints by dragging</li>
806             <li>Use the <strong>Eye (■)</strong> button to show/hide measurements</li>
807             <li>Click the <strong>×</strong> button to delete a measurement</li>
808         </ul>
809     </section><!-- Area Measurements -->
810     <section>
811         <h3 class="text-lg font-semibold text-white mb-2">Angle Measurements</h3>
812         <p class="text-slate-300 mb-2">Measure the angle formed by two rays meeting at a vertex:</p>
813         <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
814             <li>Switch to the <strong>Angle</strong> tab</li>
815             <li>Click <strong>three</strong> points on the image</li>
816             <li>The <strong>second</strong> point is the <strong>vertex</strong> (middle point)</li>
817             <li>Each angle is labeled (Ang1, Ang2, etc.) and reported in <strong>degrees (deg)</strong></li>
818             <li>Use the <strong>Edit (—■)</strong> button to adjust by dragging any of the three points, then <strong>Save
819             Changes</strong> or <strong>Cancel</strong></li>
820             <li>Use the <strong>Eye (■)</strong> button to show/hide, and <strong>×</strong> to delete</li>
821         </ul>
822     </section><!-- Area Measurements -->
823     <section>
824         <h3 class="text-lg font-semibold text-white mb-2">Area Measurements</h3>
825         <p class="text-slate-300 mb-2">Measure polygon areas by defining vertices:</p>
826         <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
827             <li>Switch to the <strong>Area</strong> tab</li>
828             <li>Click points to create a polygon outline</li>
829             <li>Click <strong>Close Polygon</strong> when you have 3+ points, or double-click to close</li>
830             <li>Each area is labeled (A1, A2, etc.)</li>
831             <li>Use <strong>Edit</strong> to adjust vertices by dragging points</li>
832             <li>While editing, right-click a point to delete it, or click an edge to add a point</li>
833             <li>Use <strong>Eye</strong> to show/hide, <strong>×</strong> to delete</li>
834         </ul>
835     </section><!-- Particle Analysis -->
836     <section>
837         <h3 class="text-lg font-semibold text-white mb-2">Particle Analysis</h3>
838         <p class="text-slate-300 mb-2">Automatically detect and measure particles:</p>
839         <h4 class="font-semibold text-blue-300 mt-3 mb-1">Region of Interest (ROI)</h4>
840         <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
841             <li>Click <strong>Select ROI</strong> to restrict analysis to a specific region</li>
842             <li>Click and drag to draw a rectangular region</li>
843             <li>Click <strong>Clear ROI</strong> to analyze the full image</li>
844         </ul>
845         <h4 class="font-semibold text-blue-300 mt-3 mb-1">Preprocessing Options</h4>
846         <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
847             <li><strong>Background subtraction</strong> - Removes uneven illumination (rolling ball method)</li>
848             <li><strong>Smoothing</strong> - Reduces noise using Gaussian or Median filters</li>
849             <li><strong>Morphological opening</strong> - Removes small noise objects</li>
850         </ul>
851         <h4 class="font-semibold text-blue-300 mt-3 mb-1">Thresholding</h4>
852         <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
853             <li><strong>Auto (Otsu)</strong> - Automatically calculates optimal threshold</li>
854             <li><strong>Manual</strong> - Adjust threshold slider (0-255)</li>
855         </ul>

```

```

852     <li><strong>Adaptive</strong> - Local thresholding for uneven backgrounds</li>
853     <li>Select whether particles are <strong>bright</strong> or <strong>dark</strong></li>
854 </ul>
855 <h4 class="font-semibold text-blue-300 mt-3 mb-1">Analysis</h4>
856 <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
857     <li>Set <strong>Min Area</strong> to filter out small noise (in real units if calibrated)</li>
858     <li>Click <strong>Analyze Particles</strong></li>
859     <li>View count, mean area, mean ECD (Equivalent Circular Diameter), and size distribution</li>
860     <li>Each detected particle is outlined in orange on the image</li>
861 </ul>
862 </section><!-- Hole Analysis -->
863 <section>
864     <h3 class="text-lg font-semibold text-white mb-2">■ Hole Analysis</h3>
865     <p class="text-slate-300 mb-2">Automatically detect and measure holes/voids in materials:</p>
866     <h4 class="font-semibold text-blue-300 mt-3 mb-1">Region of Interest (ROI)</h4>
867     <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
868         <li>Click <strong>Select ROI</strong> to restrict analysis to a specific region</li>
869         <li>Click and drag to draw a rectangular region</li>
870         <li>Click <strong>Clear ROI</strong> to analyze the full image</li>
871     </ul>
872     <h4 class="font-semibold text-blue-300 mt-3 mb-1">Preprocessing Options</h4>
873     <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
874         <li><strong>Background subtraction</strong> - Removes uneven illumination (rolling ball method)</li>
875         <li><strong>Smoothing</strong> - Reduces noise using Gaussian or Median filters</li>
876         <li><strong>Morphological opening</strong> - Removes small noise objects</li>
877     </ul>
878     <h4 class="font-semibold text-blue-300 mt-3 mb-1">Thresholding</h4>
879     <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
880         <li><strong>Auto (Otsu)</strong> - Automatically calculates optimal threshold</li>
881         <li><strong>Manual</strong> - Adjust threshold slider (0-255)</li>
882         <li><strong>Adaptive</strong> - Local thresholding for uneven backgrounds</li>
883         <li>Select whether holes are <strong>dark</strong> or <strong>bright</strong></li>
884     </ul>
885     <h4 class="font-semibold text-blue-300 mt-3 mb-1">Analysis</h4>
886     <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
887         <li>Set <strong>Min Area</strong> to filter out small noise (in real units if calibrated)</li>
888         <li>Click <strong>Analyze Holes</strong></li>
889         <li>View count, mean area, mean ECD (Equivalent Circular Diameter), and size distribution</li>
890         <li>Each detected hole is outlined in purple on the image</li>
891     </ul>
892 </section><!-- Navigation & Controls -->
893 <section>
894     <h3 class="text-lg font-semibold text-white mb-2">■ Descriptive Statistics & Histogram</h3>
895     <p class="text-slate-300 mb-2">Summarize and visualize your measurements:</p>
896     <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
897         <li><strong>Descriptive Statistics</strong> shows mean, median, standard deviation, quartiles, etc.</li>
898         <li><strong>Histogram</strong> shows the distribution of values (adjust bin count in the histogram window).</li>
899         <li>Buttons appear when there is enough data (shown when <strong>n ≥ 4</strong> for the selected feature).</li>
900         <li>For <strong>Length/Area/Particles/Holes</strong>, values are shown in pixels until scale is calibrated (then real
901         units are shown).</li>
902         <li><strong>Angle</strong> is always in <strong>degrees</strong> and does not require calibration.</li>
903         <li>In the histogram window, <strong>Particles/Holes</strong> can be plotted by <strong>ECD</strong> or
904         <strong>Area</strong> (metric selector).</li>
905     </ul>
906 </section><!-- Results & Export -->
907 <section>
908     <h3 class="text-lg font-semibold text-white mb-2">■ Navigation & Controls</h3>
909     <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
910         <li><strong>Zoom</strong> - Use the slider, mouse wheel, or pinch gesture</li>
911         <li><strong>Pan</strong> - Middle-click drag, Alt+drag, two-finger drag, or use arrow buttons/keys</li>
912         <li><strong>Fit to View</strong> - Automatically adjusts zoom to fit the image</li>
913         <li><strong>Drag & Drop</strong> - Drag image files directly onto the canvas</li>
914     </ul>
915 </section><!-- Results & Export -->
916 <section>
917     <h3 class="text-lg font-semibold text-white mb-2">■ Results & Export</h3>
918     <p class="text-slate-300 mb-2">Export data from within each measurement tab:</p>
919     <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
920         <li><strong>Save Image</strong> - Export image with all measurements overlaid (PNG)</li>
921         <li><strong>Copy TSV</strong> - Copy the current tab's data as tab-separated values (paste into spreadsheets)</li>
922         <li><strong>Download CSV</strong> - Download the current tab's data as a .csv file</li>
923         <li><strong>Note</strong>: Export buttons appear only when the current tab has at least one measurement/object.</li>
924         <li><strong>Reset</strong> - Use the reset control to clear measurements and restart</li>
925     </ul>
926 </section><!-- Tips & Tricks -->
927 <section>
928     <h3 class="text-lg font-semibold text-white mb-2">■ Tips & Tricks</h3>
929     <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
930         <li>Always calibrate scale first for accurate real-world measurements</li>
931         <li>Use ROI selection to speed up particle/hole analysis on large images</li>
932         <li>Try Auto (Otsu) threshold first, then adjust manually if needed</li>

```

```

931     <li>Adaptive thresholding works best for images with uneven lighting</li>
932     <li>Background subtraction is useful for removing gradients in microscopy images</li>
933     <li>Increase smoothing to reduce noise, but be careful not to blur features</li>
934     <li>Morphological opening removes small speckles without affecting larger particles</li>
935     <li>The Save Image button is only enabled when you have measurements on the image</li>
936   </ul>
937 </section><!-- Keyboard Shortcuts -->
938 <section>
939   <h3 class="text-lg font-semibold text-white mb-2">Keyboard Shortcuts</h3>
940   <ul class="list-disc list-inside space-y-1 text-slate-300 ml-4">
941     <li><strong>Arrow Keys</strong> - Pan the image</li>
942     <li><strong>Mouse Wheel</strong> - Zoom in/out</li>
943     <li><strong>Alt + Left Click</strong> - Pan the image</li>
944   </ul>
945 </section><!-- Disclaimer -->
946 <section>
947   <h3 class="text-lg font-semibold text-white mb-2">Disclaimer</h3>
948   <div class="bg-slate-900/50 rounded-lg p-4 border border-slate-700 space-y-2">
949     <p class="text-slate-300 text-sm">MicroMeasure is provided for research and educational purposes only.</p>
950     <p class="text-slate-300 text-sm">While every effort has been made to ensure accuracy, the developer makes no guarantee
    regarding absolute measurement correctness for all imaging conditions. Users are solely responsible for verifying
    calibration accuracy and validating results for their specific applications.</p>
951     <p class="text-slate-300 text-sm">MicroMeasure does not replace certified metrology software or instrument calibration
    standards.</p>
952   </div>
953 </section>
954 </div>
955 </div>
956 </div>
957 <!-- Descriptive Statistics Modal (publish-ready, light) -->
958 <div id="descriptiveStatsModal" class="processing-modal hidden" role="dialog" aria-modal="true" aria-label="Descriptive
    Statistics">
959   <div class="max-w-3xl w-[92vw] max-h-[90vh] overflow-hidden rounded-2xl bg-white text-slate-900 border border-slate-200
    shadow-2xl">
960     <div class="px-6 py-4 border-b border-slate-200 flex items-start justify-between gap-4">
961       <div>
962         <h3 id="statsModalTitle" class="text-lg font-semibold">Descriptive Statistics</h3>
963         <p id="statsModalSubtitle" class="text-xs text-slate-500 mt-1"></p>
964       </div>
965       <button id="closeStatsModalBtn" class="text-slate-500 hover:text-slate-900 text-2xl leading-none"
        aria-label="Close">×</button>
966     </div>
967     <div class="px-6 py-4 overflow-y-auto" style="max-height: calc(90vh - 140px)">
968       <div class="flex gap-2 flex-wrap mb-4">
969         <button class="stats-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
          hover:bg-slate-100" data-stats-type="length">Length</button>
970         <button class="stats-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
          hover:bg-slate-100" data-stats-type="angle">Angle</button>
971         <button class="stats-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
          hover:bg-slate-100" data-stats-type="area">Area</button>
972         <button class="stats-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
          hover:bg-slate-100" data-stats-type="particles">Particles</button>
973         <button class="stats-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
          hover:bg-slate-100" data-stats-type="holes">Holes</button>
974       </div>
975       <div id="statsModalContent" class="space-y-4"></div>
976     </div>
977     <div class="px-6 py-4 border-t border-slate-200 flex gap-2">
978       <button id="copyStatsTSVBtn" class="btn-secondary px-4 py-2 rounded-lg text-sm flex-1 text-white">Copy TSV</button>
979       <button id="closeStatsModalBtnBottom" class="btn-primary px-4 py-2 rounded-lg text-sm flex-1 text-white">Close</button>
980     </div>
981   </div>
982 </div>
983 <!-- Histogram Modal (publish-ready, light) -->
984 <div id="histogramModal" class="processing-modal hidden" role="dialog" aria-modal="true" aria-label="Histogram">
985   <div class="max-w-4xl w-[94vw] max-h-[92vh] overflow-hidden rounded-2xl bg-white text-slate-900 border border-slate-200
    shadow-2xl">
986     <div class="px-6 py-4 border-b border-slate-200 flex items-start justify-between gap-4">
987       <div>
988         <h3 id="histModalTitle" class="text-lg font-semibold">Histogram</h3>
989         <p id="histModalSubtitle" class="text-xs text-slate-500 mt-1"></p>
990       </div>
991       <button id="closeHistModalBtn" class="text-slate-500 hover:text-slate-900 text-2xl leading-none"
        aria-label="Close">×</button>
992     </div>
993     <div class="px-6 py-4 overflow-y-auto" style="max-height: calc(92vh - 160px)">
994       <div class="flex gap-2 flex-wrap mb-4">
995         <button class="hist-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
          hover:bg-slate-100" data-hist-type="length">Length</button>
996         <button class="hist-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
          hover:bg-slate-100" data-hist-type="angle">Angle</button>

```

```

997 <button class="hist-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
    hover:bg-slate-100" data-hist-type="area">Area</button>
998 <button class="hist-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
    hover:bg-slate-100" data-hist-type="particles">Particles</button>
999 <button class="hist-type-btn px-3 py-1.5 rounded-full text-xs font-medium border border-slate-200 bg-slate-50
    hover:bg-slate-100" data-hist-type="holes">Holes</button>
1000 </div>
1001 <div class="grid grid-cols-1 lg:grid-cols-3 gap-4 items-start mb-4">
1002   <div class="rounded-xl border border-slate-200 bg-slate-50 p-4">
1003     <div class="flex items-center justify-between gap-3">
1004       <label for="histBinsNumber" class="text-xs font-medium text-slate-700">Bins</label>
1005       <input id="histBinsNumber" type="number" min="3" max="80" step="1" value="15" class="w-24 bg-white border
border-slate-300 rounded-lg px-2 py-1 text-xs mono text-slate-900" />
1006     </div>
1007     <div class="flex items-center justify-between gap-3 mt-3">
1008       <label for="histXTicksNumber" class="text-xs font-medium text-slate-700">X-axis divisions</label>
1009       <input id="histXTicksNumber" type="number" min="2" max="12" step="1" value="6" class="w-24 bg-white border
border-slate-300 rounded-lg px-2 py-1 text-xs mono text-slate-900" />
1010     </div>
1011     <div class="flex items-center justify-between gap-3 mt-3">
1012       <label for="histYTicksNumber" class="text-xs font-medium text-slate-700">Y-axis divisions</label>
1013       <input id="histYTicksNumber" type="number" min="2" max="12" step="1" value="6" class="w-24 bg-white border
border-slate-300 rounded-lg px-2 py-1 text-xs mono text-slate-900" />
1014     </div>
1015     <div class="mt-3">
1016       <label for="histMetricSelect" class="text-xs font-medium text-slate-700">Metric</label>
1017       <select id="histMetricSelect" class="w-full mt-1 bg-white border border-slate-300 rounded-lg px-2 py-1.5 text-xs
text-slate-900">
1018         <option value="auto" selected>Auto</option>
1019         <option value="area">Area</option>
1020         <option value="ecd">ECD</option>
1021       </select>
1022       <p class="text-[11px] text-slate-500 mt-1">Metric applies to Particles/Holes (Length/Area ignore it).</p>
1023     </div>
1024   </div>
1025   <div class="lg:col-span-2 rounded-xl border border-slate-200 bg-white p-4">
1026     <div class="flex items-center justify-between flex-wrap gap-2 mb-2">
1027       <div class="text-xs text-slate-600">
1028         <span class="font-medium">n:</span> <span id="histN" class="mono">-</span>
1029         <span class="ml-3 font-medium">Range:</span> <span id="histRange" class="mono">-</span>
1030       </div>
1031       <button id="downloadHistSvgBtn" class="btn-secondary px-3 py-1.5 rounded-lg text-xs text-white">Download SVG</button>
1032     </div>
1033     <div class="overflow-x-auto">
1034       <svg id="histSvg" class="w-full" viewBox="0 0 900 520" preserveAspectRatio="xMidYMid meet" role="img"
aria-label="Histogram chart"></svg>
1035     </div>
1036     <div id="histEmptyNote" class="hidden mt-3 rounded-lg border border-slate-200 bg-slate-50 p-3">
1037       <p class="text-xs text-slate-700 font-medium">Not enough data</p>
1038       <p class="text-xs text-slate-600 mt-1">Add measurements/objects to generate a histogram.</p>
1039     </div>
1040   </div>
1041 </div>
1042 </div>
1043 <div class="px-6 py-4 border-t border-slate-200 flex gap-2">
1044   <button id="closeHistModalBtnBottom" class="btn-primary px-4 py-2 rounded-lg text-sm flex-1 text-white">Close</button>
1045 </div>
1046 </div>
1047 </div>
1048 <!-- Reset Confirmation Modal -->
1049 <div id="resetConfirmModal" class="processing-modal hidden" role="dialog" aria-modal="true" aria-label="Reset confirmation"
1050   <div class="processing-content max-w-lg">
1051     <h3 class="text-lg font-semibold text-center mb-2">Reset MicroMeasure?</h3>
1052     <p class="text-sm text-slate-300 text-center mb-4">
1053       This will clear the loaded image, scale calibration, ROI selection, and all measurements/results (Length, Angle, Area,
        Particles, Holes). This cannot be undone.
1054     </p>
1055     <div class="flex gap-2">
1056       <button id="confirmResetYes" class="btn-danger px-4 py-2 rounded-lg text-sm flex-1">Yes</button>
1057       <button id="confirmResetNo" class="btn-secondary px-4 py-2 rounded-lg text-sm flex-1">No</button>
1058     </div>
1059   </div>
1060 </div>
1061 <!-- Save Image Confirmation Modal -->
1062 <div id="saveImageConfirmModal" class="processing-modal hidden" role="dialog" aria-modal="true" aria-label="Save image
confirmation">
1063   <div class="processing-content max-w-lg">
1064     <h3 class="text-lg font-semibold text-center mb-2">Save Image?</h3>
1065     <p class="text-sm text-slate-300 text-center mb-4">
1066       This will export the current image as a PNG with all visible measurements and overlays included. Continue to download the
        image.

```



```

1067     </p>
1068     <div class="flex gap-2">
1069         <button id="confirmSaveImageYes" class="btn-primary px-4 py-2 rounded-lg text-sm flex-1">Yes</button>
1070         <button id="confirmSaveImageNo" class="btn-secondary px-4 py-2 rounded-lg text-sm flex-1">No</button>
1071     </div>
1072 </div>
1073 </div>
1074 <!-- Home Redirect Confirmation Modal -->
1075 <div id="homeConfirmModal" class="processing-modal hidden" role="dialog" aria-modal="true" aria-label="Home page
confirmation">
1076     <div class="processing-content max-w-lg">
1077         <h3 class="text-lg font-semibold text-center mb-2">Go to Home Page?</h3>
1078         <p class="text-sm text-slate-300 text-center mb-4">
1079             This will redirect you to the i-Nano Research Facility home page at DLSU Manila. Continue to open
https://www.inanolab.com/#extras.
1080         </p>
1081         <div class="flex gap-2">
1082             <button id="confirmHomeYes" class="btn-primary px-4 py-2 rounded-lg text-sm flex-1">Yes</button>
1083             <button id="confirmHomeNo" class="btn-secondary px-4 py-2 rounded-lg text-sm flex-1">No</button>
1084         </div>
1085     </div>
1086 </div>
1087 <!-- Demo Image Confirmation Modal -->
1088 <div id="demoImageConfirmModal" class="processing-modal hidden" role="dialog" aria-modal="true" aria-label="Demo image
confirmation">
1089     <div class="processing-content max-w-lg">
1090         <h3 class="text-lg font-semibold text-center mb-2">Load Demo Photo?</h3>
1091         <p class="text-sm text-slate-300 text-center mb-4">
1092             This will load the bundled sample SEM image so you can try the app without uploading your own file. If you already have a
loaded image or measurements, they will be replaced.
1093         </p>
1094         <div class="flex gap-2">
1095             <button id="confirmDemoImageYes" class="btn-primary px-4 py-2 rounded-lg text-sm flex-1">Yes</button>
1096             <button id="confirmDemoImageNo" class="btn-secondary px-4 py-2 rounded-lg text-sm flex-1">No</button>
1097         </div>
1098     </div>
1099 </div>
1100 <script>
1101     // ===== STATE =====
1102     const state = {
1103         image: null,
1104         imageData: null,
1105         zoom: 100,
1106         panX: 0,
1107         panY: 0,
1108         isPanning: false,
1109         lastPanX: 0,
1110         lastPanY: 0,
1111
1112         // Guidance
1113         quickStartDismissed: true,
1114
1115         // Calibration
1116         isCalibrated: false,
1117         unitsPerPixel: 0,
1118         unit: 'µm',
1119         calibrationLine: null,
1120         isSettingScale: false,
1121         scalePoints: [],
1122
1123         // Current tool
1124         currentTool: 'length',
1125
1126         // Length measurements
1127         lengthPoints: [],
1128         lengths: [],
1129         editingLengthIndex: null,
1130         draggedLengthPoint: null, // 'p1' or 'p2'
1131
1132         // Angle measurements
1133         anglePoints: [],
1134         angles: [], // { p1, p2, p3, angleDeg, visible? }
1135         editingAngleIndex: null,
1136         draggedAnglePointIndex: null, // 0|1|2
1137         angleEditBackup: null,
1138
1139         // Area measurements
1140         areaMode: 'polygon', // 'polygon' | 'circle'
1141         areaPoints: [],
1142         isDrawingCircle: false,
1143         circleDraft: null, // { center: {x,y}, radiusPx }

```

```

1144     areas: [],
1145     editingAreaIndex: null,
1146     draggedPointIndex: null,
1147
1148     // Particles
1149     particles: [],
1150
1151     // Holes
1152     holes: [],
1153
1154     // ROI (Region of Interest)
1155     roi: null, // {x, y, width, height}
1156     isDrawingROI: false,
1157     roiStart: null,
1158     roiMode: null // 'particles' or 'holes'
1159 };
1160
1161 // ===== DOM ELEMENTS =====
1162 const elements = {
1163     imageInput: document.getElementById('imageInput'),
1164     mainCanvas: document.getElementById('mainCanvas'),
1165     overlayCanvas: document.getElementById('overlayCanvas'),
1166     canvasWrapper: document.getElementById('canvasWrapper'),
1167     noImagePlaceholder: document.getElementById('noImagePlaceholder'),
1168     zoomSlider: document.getElementById('zoomSlider'),
1169     zoomValue: document.getElementById('zoomValue'),
1170     imageDimensions: document.getElementById('imageDimensions'),
1171     imgWidth: document.getElementById('imgWidth'),
1172     imgHeight: document.getElementById('imgHeight'),
1173     fitToView: document.getElementById('fitToView'),
1174     panLeft: document.getElementById('panLeft'),
1175     panRight: document.getElementById('panRight'),
1176     panUp: document.getElementById('panUp'),
1177     panDown: document.getElementById('panDown'),
1178     resetPan: document.getElementById('resetPan'),
1179     setScaleBtn: document.getElementById('setScaleBtn'),
1180     calibrationInputs: document.getElementById('calibrationInputs'),
1181     knownLength: document.getElementById('knownLength'),
1182     unitSelect: document.getElementById('unitSelect'),
1183     saveScale: document.getElementById('saveScale'),
1184     cancelScale: document.getElementById('cancelScale'),
1185     scaleInfo: document.getElementById('scaleInfo'),
1186     scaleValue: document.getElementById('scaleValue'),
1187     callLineLength: document.getElementById('callLineLength'),
1188     resetScaleBtn: document.getElementById('resetScaleBtn'),
1189     quickStartOverlay: document.getElementById('quickStartOverlay'),
1190     quickStartOk: document.getElementById('quickStartOk'),
1191     quickStartStep1: document.getElementById('quickStartStep1'),
1192     quickStartStep2: document.getElementById('quickStartStep2'),
1193     quickStartStep3: document.getElementById('quickStartStep3'),
1194     appTitle: document.getElementById('appTitle'),
1195     loadDemoBtn: document.getElementById('loadDemoBtn'),
1196     headerResetBtn: document.getElementById('headerResetBtn'),
1197
1198     // Tabs
1199     tabBtns: document.querySelectorAll('.tab-btn'),
1200     tabContents: document.querySelectorAll('.tab-content'),
1201
1202     // Length
1203     lengthMeasurements: document.getElementById('lengthMeasurements'),
1204     clearLengths: document.getElementById('clearLengths'),
1205     lengthStatsBtn: document.getElementById('lengthStatsBtn'),
1206     lengthHistBtn: document.getElementById('lengthHistBtn'),
1207     lengthExportRow: document.getElementById('lengthExportRow'),
1208     copyLengthTSVBtn: document.getElementById('copyLengthTSVBtn'),
1209     downloadLengthTSVBtn: document.getElementById('downloadLengthTSVBtn'),
1210     editLengthModeControls: document.getElementById('editLengthModeControls'),
1211     editingLengthLabel: document.getElementById('editingLengthLabel'),
1212     saveLengthEdit: document.getElementById('saveLengthEdit'),
1213     cancelLengthEdit: document.getElementById('cancelLengthEdit'),
1214
1215     // Angle
1216     angleMeasurements: document.getElementById('angleMeasurements'),
1217     clearAngles: document.getElementById('clearAngles'),
1218     angleStatsBtn: document.getElementById('angleStatsBtn'),
1219     angleHistBtn: document.getElementById('angleHistBtn'),
1220     angleExportRow: document.getElementById('angleExportRow'),
1221     copyAngleTSVBtn: document.getElementById('copyAngleTSVBtn'),
1222     downloadAngleTSVBtn: document.getElementById('downloadAngleTSVBtn'),
1223     editAngleModeControls: document.getElementById('editAngleModeControls'),
1224     editingAngleLabel: document.getElementById('editingAngleLabel'),

```

```

1225     saveAngleEdit: document.getElementById('saveAngleEdit'),
1226     cancelAngleEdit: document.getElementById('cancelAngleEdit'),
1227
1228     // Area
1229     areaMeasurements: document.getElementById('areaMeasurements'),
1230     clearAreas: document.getElementById('clearAreas'),
1231     areaStatsBtn: document.getElementById('areaStatsBtn'),
1232     areaHistBtn: document.getElementById('areaHistBtn'),
1233     areaExportRow: document.getElementById('areaExportRow'),
1234     copyAreaTSVBtn: document.getElementById('copyAreaTSVBtn'),
1235     downloadAreaTSVBtn: document.getElementById('downloadAreaTSVBtn'),
1236     areaModePolygonBtn: document.getElementById('areaModePolygonBtn'),
1237     areaModeCircleBtn: document.getElementById('areaModeCircleBtn'),
1238     areaInstructions: document.getElementById('areaInstructions'),
1239     closePolygonBtn: document.getElementById('closePolygonBtn'),
1240     editModeControls: document.getElementById('editModeControls'),
1241     editingAreaLabel: document.getElementById('editingAreaLabel'),
1242     saveAreaEdit: document.getElementById('saveAreaEdit'),
1243     cancelAreaEdit: document.getElementById('cancelAreaEdit'),
1244
1245     // Particles
1246     backgroundSubtraction: document.getElementById('backgroundSubtraction'),
1247     backgroundOptions: document.getElementById('backgroundOptions'),
1248     rollingBallRadius: document.getElementById('rollingBallRadius'),
1249     smoothingMethod: document.getElementById('smoothingMethod'),
1250     morphologicalOpening: document.getElementById('morphologicalOpening'),
1251     autoThresholdBtn: document.getElementById('autoThresholdBtn'),
1252     adaptiveThreshold: document.getElementById('adaptiveThreshold'),
1253     manualThresholdControls: document.getElementById('manualThresholdControls'),
1254     adaptiveThresholdControls: document.getElementById('adaptiveThresholdControls'),
1255     adaptiveBlockSize: document.getElementById('adaptiveBlockSize'),
1256     particleThreshold: document.getElementById('particleThreshold'),
1257     thresholdValue: document.getElementById('thresholdValue'),
1258     minParticleArea: document.getElementById('minParticleArea'),
1259     minParticleAreaLabel: document.getElementById('minParticleAreaLabel'),
1260     analyzeParticles: document.getElementById('analyzeParticles'),
1261     particleResults: document.getElementById('particleResults'),
1262     particleCount: document.getElementById('particleCount'),
1263     particleMeanArea: document.getElementById('particleMeanArea'),
1264     particleMeanECD: document.getElementById('particleMeanECD'),
1265     particleStdDev: document.getElementById('particleStdDev'),
1266     particleHistogram: document.getElementById('particleHistogram'),
1267     particleStatsBtn: document.getElementById('particleStatsBtn'),
1268     particleHistBtn: document.getElementById('particleHistBtn'),
1269     clearParticles: document.getElementById('clearParticles'),
1270     particlesExportRow: document.getElementById('particlesExportRow'),
1271     copyParticlesTSVBtn: document.getElementById('copyParticlesTSVBtn'),
1272     downloadParticlesTSVBtn: document.getElementById('downloadParticlesTSVBtn'),
1273
1274     // ROI controls
1275     selectParticleROI: document.getElementById('selectParticleROI'),
1276     clearParticleROI: document.getElementById('clearParticleROI'),
1277     particleROIInfo: document.getElementById('particleROIInfo'),
1278     particleROIDimensions: document.getElementById('particleROIDimensions'),
1279     selectHoleROI: document.getElementById('selectHoleROI'),
1280     clearHoleROI: document.getElementById('clearHoleROI'),
1281     holeROIInfo: document.getElementById('holeROIInfo'),
1282     holeROIDimensions: document.getElementById('holeROIDimensions'),
1283
1284     // Holes
1285     holeThreshold: document.getElementById('holeThreshold'),
1286     holeThresholdValue: document.getElementById('holeThresholdValue'),
1287     minHoleArea: document.getElementById('minHoleArea'),
1288     minHoleAreaLabel: document.getElementById('minHoleAreaLabel'),
1289     smoothHoles: document.getElementById('smoothHoles'),
1290     analyzeHoles: document.getElementById('analyzeHoles'),
1291     holeResults: document.getElementById('holeResults'),
1292     holeCount: document.getElementById('holeCount'),
1293     holeMeanArea: document.getElementById('holeMeanArea'),
1294     holeMeanECD: document.getElementById('holeMeanECD'),
1295     holeStdDev: document.getElementById('holeStdDev'),
1296     holeHistogram: document.getElementById('holeHistogram'),
1297     holeStatsBtn: document.getElementById('holeStatsBtn'),
1298     holeHistBtn: document.getElementById('holeHistBtn'),
1299     clearHoles: document.getElementById('clearHoles'),
1300     holesExportRow: document.getElementById('holesExportRow'),
1301     copyHolesTSVBtn: document.getElementById('copyHolesTSVBtn'),
1302     downloadHolesTSVBtn: document.getElementById('downloadHolesTSVBtn'),
1303
1304     // Save Image
1305     saveScreenshot: document.getElementById('saveScreenshot'),

```

```

1306     homeBtn: document.getElementById('homeBtn'),
1307
1308     // Processing modal
1309     processingModal: document.getElementById('processingModal'),
1310     processingTitle: document.getElementById('processingTitle'),
1311     processingStep: document.getElementById('processingStep'),
1312     processingTime: document.getElementById('processingTime'),
1313     progressBar: document.getElementById('progressBar'),
1314
1315     // Descriptive statistics modal
1316     descriptiveStatsModal: document.getElementById('descriptiveStatsModal'),
1317     statsModalTitle: document.getElementById('statsModalTitle'),
1318     statsModalSubtitle: document.getElementById('statsModalSubtitle'),
1319     statsModalContent: document.getElementById('statsModalContent'),
1320     statsTypeBtns: document.querySelectorAll('.stats-type-btn'),
1321     closeStatsModalBtn: document.getElementById('closeStatsModalBtn'),
1322     closeStatsModalBtnBottom: document.getElementById('closeStatsModalBtnBottom'),
1323     copyStatsTSVBtn: document.getElementById('copyStatsTSVBtn'),
1324
1325     // Histogram modal
1326     histogramModal: document.getElementById('histogramModal'),
1327     histModalTitle: document.getElementById('histModalTitle'),
1328     histModalSubtitle: document.getElementById('histModalSubtitle'),
1329     histTypeBtns: document.querySelectorAll('.hist-type-btn'),
1330     closeHistModalBtn: document.getElementById('closeHistModalBtn'),
1331     closeHistModalBtnBottom: document.getElementById('closeHistModalBtnBottom'),
1332     histBinsNumber: document.getElementById('histBinsNumber'),
1333     histXTicksNumber: document.getElementById('histXTicksNumber'),
1334     histYTicksNumber: document.getElementById('histYTicksNumber'),
1335     histMetricSelect: document.getElementById('histMetricSelect'),
1336     histSvg: document.getElementById('histSvg'),
1337     histN: document.getElementById('histN'),
1338     histRange: document.getElementById('histRange'),
1339     histEmptyNote: document.getElementById('histEmptyNote'),
1340     downloadHistSvgBtn: document.getElementById('downloadHistSvgBtn'),
1341     resetConfirmModal: document.getElementById('resetConfirmModal'),
1342     confirmResetYes: document.getElementById('confirmResetYes'),
1343     confirmResetNo: document.getElementById('confirmResetNo'),
1344     saveImageConfirmModal: document.getElementById('saveImageConfirmModal'),
1345     confirmSaveImageYes: document.getElementById('confirmSaveImageYes'),
1346     confirmSaveImageNo: document.getElementById('confirmSaveImageNo'),
1347     homeConfirmModal: document.getElementById('homeConfirmModal'),
1348     confirmHomeYes: document.getElementById('confirmHomeYes'),
1349     confirmHomeNo: document.getElementById('confirmHomeNo'),
1350     demoImageConfirmModal: document.getElementById('demoImageConfirmModal'),
1351     confirmDemoImageYes: document.getElementById('confirmDemoImageYes'),
1352     confirmDemoImageNo: document.getElementById('confirmDemoImageNo')
1353 };
1354
1355 const mainCtx = elements.mainCanvas.getContext('2d');
1356 const overlayCtx = elements.overlayCanvas.getContext('2d');
1357
1358 // ===== CONFIG =====
1359 const defaultConfig = {
1360     app_title: 'MicroMeasure'
1361 };
1362
1363 let config = { ...defaultConfig };
1364
1365 // ===== ELEMENT SDK =====
1366 if (window.elementSdk) {
1367     window.elementSdk.init({
1368         defaultConfig,
1369         onConfigChange: async (newConfig) => {
1370             config = { ...defaultConfig, ...newConfig };
1371             elements.appTitle.textContent = config.app_title || defaultConfig.app_title;
1372         },
1373         mapToCapabilities: () => ({
1374             recolorables: [],
1375             borderables: [],
1376             fontEditable: undefined,
1377             fontSizeable: undefined
1378         }),
1379         mapToEditPanelValues: (cfg) => new Map([
1380             ['app_title', cfg.app_title || defaultConfig.app_title]
1381         ])
1382     });
1383 }
1384
1385 // ===== IMAGE LOADING =====
1386 const TIFF_MIME_TYPES = new Set(['image/tiff', 'image/tif', 'image/x-tiff', 'application/tiff']);

```

```

1387
1388 function isTiffFile(file) {
1389     if (!file) return false;
1390     const fileType = (file.type || '').toLowerCase();
1391     return Tiff_MIME_TYPES.has(fileType) || /\.(\tif|tiff)$/i.test(file.name || '');
1392 }
1393
1394 function isSupportedImageFile(file) {
1395     if (!file) return false;
1396     const fileType = (file.type || '').toLowerCase();
1397     return fileType.startsWith('image/') || isTiffFile(file);
1398 }
1399
1400 function showImageLoadError(message) {
1401     hideProcessingModal();
1402     window.alert(message);
1403 }
1404
1405 function applyLoadedImage(img) {
1406     state.image = img;
1407     state.imageData = null;
1408
1409     elements.mainCanvas.width = img.width;
1410     elements.mainCanvas.height = img.height;
1411     elements.overlayCanvas.width = img.width;
1412     elements.overlayCanvas.height = img.height;
1413
1414     elements.imgWidth.textContent = img.width;
1415     elements.imgHeight.textContent = img.height;
1416     elements.imageDimensions.classList.remove('hidden');
1417     elements.fitToView.classList.remove('hidden');
1418     document.getElementById('panControls').classList.remove('hidden');
1419
1420     elements.noImagePlaceholder.classList.add('hidden');
1421     elements.mainCanvas.classList.remove('hidden');
1422     elements.overlayCanvas.classList.remove('hidden');
1423
1424     elements.setScaleBtn.disabled = false;
1425     elements.analyzeParticles.disabled = false;
1426     elements.analyzeHoles.disabled = false;
1427     elements.selectParticleROI.disabled = false;
1428     elements.selectHoleROI.disabled = false;
1429
1430     // If the user was mid-calibration and loads a new image, reset the
1431     // calibration drawing state so auto-start can run reliably.
1432     state.isSettingScale = false;
1433     state.scalePoints = [];
1434
1435     // Reset pan and defer initial fit-to-view for placement
1436     state.panX = 0;
1437     state.panY = 0;
1438     requestAnimationFrame(() => {
1439         fitToView();
1440         // Auto-start scale setting as soon as an image is loaded
1441         // so the user is guided into the calibration step immediately.
1442         setScale();
1443         drawMainCanvas();
1444         drawOverlay();
1445         updateGuidance();
1446     });
1447 }
1448
1449 function loadBrowserImage(file) {
1450     return new Promise((resolve, reject) => {
1451         const objectUrl = URL.createObjectURL(file);
1452         const img = new Image();
1453
1454         img.onload = () => {
1455             URL.revokeObjectURL(objectUrl);
1456             resolve(img);
1457         };
1458
1459         img.onerror = () => {
1460             URL.revokeObjectURL(objectUrl);
1461             reject(new Error('The selected image could not be decoded by this browser.'));
1462         };
1463
1464         img.src = objectUrl;
1465     });
1466 }
1467

```

```

1468     async function loadTiffImage(file) {
1469         showProcessingModal('Loading TIFF Image');
1470         updateProgress('Loading TIFF decoder...', 15);
1471
1472         try {
1473             const decoderLoaded = await tiffDecoderReady;
1474             if (!decoderLoaded || !window.UTIF) {
1475                 throw new Error('TIFF support is unavailable because the TIFF decoder could not be loaded.');
```

```

1548     const wrapperH = wrapper.clientHeight;
1549
1550     // Guard against zero-size wrapper (mobile layout settling)
1551     if (!wrapperW || !wrapperH) {
1552         requestAnimationFrame(fitToView);
1553         return;
1554     }
1555
1556     const scaleX = wrapperW / state.image.width;
1557     const scaleY = wrapperH / state.image.height;
1558     const scale = Math.min(scaleX, scaleY) * 0.95;
1559
1560     state.zoom = Math.max(50, Math.min(500, Math.round(scale * 100)));
1561     elements.zoomSlider.value = state.zoom;
1562     elements.zoomValue.textContent = state.zoom + '%';
1563
1564     state.panX = 0;
1565     state.panY = 0;
1566
1567     updateCanvasTransform();
1568 }
1569
1570 function updateCanvasTransform() {
1571     const scale = state.zoom / 100;
1572     // Center in wrapper using translate, then apply pan in screen pixels
1573     const wrapper = elements.canvasWrapper;
1574     const wrapperW = wrapper.clientWidth;
1575     const wrapperH = wrapper.clientHeight;
1576     const canvasW = state.image.width * scale;
1577     const canvasH = state.image.height * scale;
1578
1579     const centerX = (wrapperW - canvasW) / 2;
1580     const centerY = (wrapperH - canvasH) / 2;
1581
1582     // Ensure absolute position baseline is 0,0 and use a single transform
1583     elements.mainCanvas.style.left = '0px';
1584     elements.mainCanvas.style.top = '0px';
1585     elements.overlayCanvas.style.left = '0px';
1586     elements.overlayCanvas.style.top = '0px';
1587
1588     const transform = `translate(${centerX + state.panX}px, ${centerY + state.panY}px) scale(${scale})`;
1589     elements.mainCanvas.style.transformOrigin = 'top left';
1590     elements.overlayCanvas.style.transformOrigin = 'top left';
1591     elements.mainCanvas.style.transform = transform;
1592     elements.overlayCanvas.style.transform = transform;
1593 }
1594
1595 function drawMainCanvas() {
1596     mainCtx.clearRect(0, 0, elements.mainCanvas.width, elements.mainCanvas.height);
1597     mainCtx.drawImage(state.image, 0, 0);
1598 }
1599
1600 // ===== OVERLAY DRAWING =====
1601 function drawOverlay() {
1602     overlayCtx.clearRect(0, 0, elements.overlayCanvas.width, elements.overlayCanvas.height);
1603
1604     // Draw calibration line
1605     if (state.calibrationLine) {
1606         drawLine(state.calibrationLine.p1, state.calibrationLine.p2, '#bbbf24', 2, true);
1607     }
1608
1609     // Draw scale setting points
1610     if (state.isSettingScale && state.scalePoints.length > 0) {
1611         drawPoint(state.scalePoints[0], '#bbbf24');
1612         if (state.scalePoints.length === 2) {
1613             drawLine(state.scalePoints[0], state.scalePoints[1], '#bbbf24', 2);
1614         }
1615     }
1616
1617     // Draw length measurements
1618     state.lengths.forEach((len, i) => {
1619         if (len.visible !== false) {
1620             const isEditing = state.editingLengthIndex === i;
1621             drawLine(len.p1, len.p2, isEditing ? '#bbbf24' : '#3b82f6', 2);
1622
1623             // Draw edit handles for editing length
1624             if (isEditing) {
1625                 overlayCtx.fillStyle = '#bbbf24';
1626                 overlayCtx.beginPath();
1627                 overlayCtx.arc(len.p1.x, len.p1.y, 8, 0, Math.PI * 2);
1628                 overlayCtx.fill();

```

```

1629     overlayCtx.strokeStyle = '#fff';
1630     overlayCtx.lineWidth = 2;
1631     overlayCtx.stroke();
1632
1633     overlayCtx.beginPath();
1634     overlayCtx.arc(len.p2.x, len.p2.y, 8, 0, Math.PI * 2);
1635     overlayCtx.fill();
1636     overlayCtx.stroke();
1637 } else {
1638     drawPoint(len.p1, '#3b82f6');
1639     drawPoint(len.p2, '#3b82f6');
1640 }
1641
1642 // Create measurement text only for non-editing lengths
1643 if (!isEditing) {
1644     const pixelText = `${formatNumber(len.pixelLength)} px`;
1645     const realText = state.isCalibrated ? `${formatNumber(len.realLength)} ${state.unit}` : '';
1646     const labelText = realText ? `L${i+1}: ${realText}` : `L${i+1}: ${pixelText}`;
1647
1648     drawMeasurementLabel(midpoint(len.p1, len.p2), labelText, '#3b82f6');
1649 }
1650 }
1651 });
1652
1653 // Draw current length points
1654 if (state.currentTool === 'length' && state.lengthPoints.length > 0) {
1655     drawPoint(state.lengthPoints[0], '#3b82f6');
1656 }
1657
1658 // Draw angle measurements
1659 state.angles.forEach((ang, i) => {
1660     if (ang.visible === false) return;
1661
1662     const isEditing = state.editingAngleIndex === i;
1663     const color = isEditing ? '#fbbf24' : '#06b6d4';
1664
1665     // Rays from vertex (p2)
1666     drawLine(ang.p2, ang.p1, color, 2);
1667     drawLine(ang.p2, ang.p3, color, 2);
1668
1669     // Arc inside the angle (minor or reflex)
1670     const arcInfo = drawAngleArc(ang.p1, ang.p2, ang.p3, !!ang.reflex, color);
1671
1672     if (isEditing) {
1673         [ang.p1, ang.p2, ang.p3].forEach(p => {
1674             overlayCtx.fillStyle = '#fbbf24';
1675             overlayCtx.beginPath();
1676             overlayCtx.arc(p.x, p.y, 8, 0, Math.PI * 2);
1677             overlayCtx.fill();
1678             overlayCtx.strokeStyle = '#fff';
1679             overlayCtx.lineWidth = 2;
1680             overlayCtx.stroke();
1681         });
1682     } else {
1683         drawPoint(ang.p1, color);
1684         drawPoint(ang.p2, color);
1685         drawPoint(ang.p3, color);
1686
1687         // Put label near the arc mid-angle (looks better than always above the vertex)
1688         let labelPos = { x: ang.p2.x, y: ang.p2.y - 18 };
1689         if (arcInfo && Number.isFinite(arcInfo.sweep)) {
1690             const mid = arcInfo.start + arcInfo.sweep / 2;
1691             const rr = (arcInfo.r || 24) + 16;
1692             labelPos = {
1693                 x: ang.p2.x + rr * Math.cos(mid),
1694                 y: ang.p2.y + rr * Math.sin(mid)
1695             };
1696         }
1697         drawMeasurementLabel(labelPos, `Ang${i + 1}: ${formatNumber(ang.angleDeg)}°`, color);
1698     }
1699 });
1700
1701 // Draw current angle points (in-progress)
1702 if (state.currentTool === 'angle' && state.anglePoints.length > 0) {
1703     const color = '#06b6d4';
1704     state.anglePoints.forEach(p => drawPoint(p, color));
1705     if (state.anglePoints.length >= 2) drawLine(state.anglePoints[1], state.anglePoints[0], color, 2, true);
1706     if (state.anglePoints.length === 3) drawLine(state.anglePoints[1], state.anglePoints[2], color, 2, true);
1707 }
1708
1709 // Draw area measurements (polygons + circles)

```



```

1710 state.areas.forEach((area, i) => {
1711     if (area.visible === false) return;
1712
1713     const isEditing = state.editingAreaIndex === i;
1714
1715     if (area.type === 'circle') {
1716         drawCircle(area.center, area.radiusPx, isEditing ? '#fbbf24' : '#22c55e');
1717
1718         if (!isEditing && area.center) {
1719             const pixelText = `${formatNumber(area.pixelArea)} px2`;
1720             const realText = state.isCalibrated ? `${formatNumber(area.realArea)} ${state.unit}2` : '';
1721             const labelText = realText ? `A${i+1}: ${realText}` : `A${i+1}: ${pixelText}`;
1722             drawMeasurementLabel(area.center, labelText, '#22c55e');
1723         }
1724
1725         return;
1726     }
1727
1728     // Polygon (default)
1729     drawPolygon(area.points, isEditing ? '#fbbf24' : '#22c55e');
1730
1731     // Draw edit handles for editing area
1732     if (isEditing) {
1733         area.points.forEach((p) => {
1734             overlayCtx.fillStyle = '#fbbf24';
1735             overlayCtx.beginPath();
1736             overlayCtx.arc(p.x, p.y, 8, 0, Math.PI * 2);
1737             overlayCtx.fill();
1738             overlayCtx.strokeStyle = '#fff';
1739             overlayCtx.lineWidth = 2;
1740             overlayCtx.stroke();
1741         });
1742     } else {
1743         // Create measurement text only for non-editing areas
1744         const pixelText = `${formatNumber(area.pixelArea)} px2`;
1745         const realText = state.isCalibrated ? `${formatNumber(area.realArea)} ${state.unit}2` : '';
1746         const labelText = realText ? `A${i+1}: ${realText}` : `A${i+1}: ${pixelText}`;
1747
1748         drawMeasurementLabel(centroid(area.points), labelText, '#22c55e');
1749     }
1750 });
1751
1752 // Draw current polygon area points (in-progress)
1753 if (state.currentTool === 'area' && state.areaMode === 'polygon' && state.areaPoints.length > 0) {
1754     overlayCtx.strokeStyle = '#22c55e';
1755     overlayCtx.lineWidth = 2;
1756     overlayCtx.setLineDash([5, 5]);
1757     overlayCtx.beginPath();
1758     overlayCtx.moveTo(state.areaPoints[0].x, state.areaPoints[0].y);
1759     state.areaPoints.forEach(p => overlayCtx.lineTo(p.x, p.y));
1760     overlayCtx.stroke();
1761     overlayCtx.setLineDash([]);
1762     state.areaPoints.forEach(p => drawPoint(p, '#22c55e'));
1763 }
1764
1765 // Draw in-progress circle (Area tool)
1766 if (state.currentTool === 'area' && state.areaMode === 'circle' && state.circleDraft) {
1767     drawCircle(state.circleDraft.center, state.circleDraft.radiusPx, '#22c55e', true);
1768     if (state.circleDraft.center) drawPoint(state.circleDraft.center, '#22c55e');
1769 }
1770
1771 // Draw particles
1772 state.particles.forEach((p, i) => {
1773     drawBoundingBox(p.bbox, '#f97316');
1774 });
1775
1776 // Draw holes
1777 state.holes.forEach((h, i) => {
1778     drawBoundingBox(h.bbox, '#a855f7');
1779 });
1780
1781 // Draw ROI rectangle
1782 if (state.roi) {
1783     overlayCtx.strokeStyle = '#fbbf24';
1784     overlayCtx.lineWidth = 3;
1785     overlayCtx.setLineDash([10, 5]);
1786     overlayCtx.strokeRect(state.roi.x, state.roi.y, state.roi.width, state.roi.height);
1787     overlayCtx.setLineDash([]);
1788
1789     // Draw ROI label
1790     const roiLabel = `ROI: ${Math.round(state.roi.width)}x${Math.round(state.roi.height)} px`;

```

```

1791     overlayCtx.font = 'bold 13px Inter, sans-serif';
1792     overlayCtx.fillStyle = '#fbbf24';
1793     overlayCtx.textAlign = 'left';
1794     overlayCtx.textBaseline = 'bottom';
1795
1796     const padding = 4;
1797     const metrics = overlayCtx.measureText(roiLabel);
1798     const labelX = state.roi.x;
1799     const labelY = state.roi.y - 5;
1800
1801     overlayCtx.fillStyle = 'rgba(0,0,0,0.85)';
1802     overlayCtx.fillRect(labelX, labelY - 16, metrics.width + padding * 2, 18);
1803
1804     overlayCtx.fillStyle = '#fbbf24';
1805     overlayCtx.fillText(roiLabel, labelX + padding, labelY);
1806   }
1807 }
1808
1809 function drawLine(p1, p2, color, width, dashed = false) {
1810   overlayCtx.strokeStyle = color;
1811   overlayCtx.lineWidth = width;
1812   overlayCtx.setLineDash(dashed ? [8, 4] : []);
1813   overlayCtx.beginPath();
1814   overlayCtx.moveTo(p1.x, p1.y);
1815   overlayCtx.lineTo(p2.x, p2.y);
1816   overlayCtx.stroke();
1817   overlayCtx.setLineDash([]);
1818 }
1819
1820 function drawPoint(p, color) {
1821   overlayCtx.fillStyle = color;
1822   overlayCtx.beginPath();
1823   overlayCtx.arc(p.x, p.y, 5, 0, Math.PI * 2);
1824   overlayCtx.fill();
1825   overlayCtx.strokeStyle = '#fff';
1826   overlayCtx.lineWidth = 1;
1827   overlayCtx.stroke();
1828 }
1829
1830 function drawCircle(center, radiusPx, color, dashed = false) {
1831   if (!center) return;
1832   if (!Number.isFinite(center.x) || !Number.isFinite(center.y)) return;
1833   if (!Number.isFinite(radiusPx) || radiusPx <= 0) return;
1834
1835   overlayCtx.fillStyle = color + '40';
1836   overlayCtx.strokeStyle = color;
1837   overlayCtx.lineWidth = 2;
1838   overlayCtx.setLineDash(dashed ? [8, 4] : []);
1839   overlayCtx.beginPath();
1840   overlayCtx.arc(center.x, center.y, radiusPx, 0, Math.PI * 2);
1841   overlayCtx.closePath();
1842   overlayCtx.fill();
1843   overlayCtx.stroke();
1844   overlayCtx.setLineDash([]);
1845 }
1846
1847 function drawPolygon(points, color) {
1848   if (points.length < 3) return;
1849   overlayCtx.fillStyle = color + '40';
1850   overlayCtx.strokeStyle = color;
1851   overlayCtx.lineWidth = 2;
1852   overlayCtx.beginPath();
1853   overlayCtx.moveTo(points[0].x, points[0].y);
1854   points.forEach(p => overlayCtx.lineTo(p.x, p.y));
1855   overlayCtx.closePath();
1856   overlayCtx.fill();
1857   overlayCtx.stroke();
1858 }
1859
1860 function drawBoundingBox(bbox, color) {
1861   overlayCtx.strokeStyle = color;
1862   overlayCtx.lineWidth = 1;
1863   overlayCtx.strokeRect(bbox.x, bbox.y, bbox.w, bbox.h);
1864 }
1865
1866 function drawLabel(p, text, color) {
1867   overlayCtx.font = '12px Inter, sans-serif';
1868   overlayCtx.fillStyle = color;
1869   overlayCtx.textAlign = 'center';
1870   overlayCtx.textBaseline = 'middle';
1871

```

```

1872     const padding = 4;
1873     const metrics = overlayCtx.measureText(text);
1874     const w = metrics.width + padding * 2;
1875     const h = 16;
1876
1877     overlayCtx.fillStyle = 'rgba(0,0,0,0.7)';
1878     overlayCtx.fillRect(p.x - w/2, p.y - h/2, w, h);
1879
1880     overlayCtx.fillStyle = color;
1881     overlayCtx.fillText(text, p.x, p.y);
1882 }
1883
1884 function formatNumber(num) {
1885     return num.toLocaleString('en-US', { maximumFractionDigits: 2 });
1886 }
1887
1888 function drawMeasurementLabel(p, text, color) {
1889     overlayCtx.font = 'bold 13px Inter, sans-serif';
1890     overlayCtx.textAlign = 'center';
1891     overlayCtx.textBaseline = 'middle';
1892
1893     const padding = 6;
1894     const metrics = overlayCtx.measureText(text);
1895     const w = metrics.width + padding * 2;
1896     const h = 20;
1897
1898     // Draw background with border
1899     overlayCtx.fillStyle = 'rgba(0,0,0,0.85)';
1900     overlayCtx.fillRect(p.x - w/2, p.y - h/2, w, h);
1901
1902     overlayCtx.strokeStyle = color;
1903     overlayCtx.lineWidth = 2;
1904     overlayCtx.strokeRect(p.x - w/2, p.y - h/2, w, h);
1905
1906     // Draw text
1907     overlayCtx.fillStyle = 'ffffff';
1908     overlayCtx.fillText(text, p.x, p.y);
1909 }
1910
1911 function drawAngleArc(p1, p2, p3, reflex, color) {
1912     const v1x = p1.x - p2.x;
1913     const v1y = p1.y - p2.y;
1914     const v2x = p3.x - p2.x;
1915     const v2y = p3.y - p2.y;
1916     const mag1 = Math.hypot(v1x, v1y);
1917     const mag2 = Math.hypot(v2x, v2y);
1918     if (mag1 === 0 || mag2 === 0) return null;
1919
1920     const a1 = Math.atan2(v1y, v1x);
1921     const a2 = Math.atan2(v2y, v2x);
1922     const TWO_PI = Math.PI * 2;
1923     const diff = (a2 - a1 + TWO_PI) % TWO_PI;
1924
1925     const minorIsA1toA2 = diff <= Math.PI;
1926     const useA1toA2 = reflex ? !minorIsA1toA2 : minorIsA1toA2;
1927
1928     const start = useA1toA2 ? a1 : a2;
1929     const end = useA1toA2 ? a2 : a1;
1930     const sweep = (end - start + TWO_PI) % TWO_PI;
1931
1932     // Radius scales with ray lengths but is clamped for consistency
1933     const r = Math.min(44, Math.max(16, Math.min(mag1, mag2) * 0.28));
1934
1935     overlayCtx.save();
1936
1937     // Subtle filled wedge to make the interior obvious
1938     overlayCtx.globalAlpha = 0.10;
1939     overlayCtx.fillStyle = color;
1940     overlayCtx.beginPath();
1941     overlayCtx.moveTo(p2.x, p2.y);
1942     overlayCtx.arc(p2.x, p2.y, r, start, end, false);
1943     overlayCtx.closePath();
1944     overlayCtx.fill();
1945
1946     // Dashed arc stroke
1947     overlayCtx.globalAlpha = 1;
1948     overlayCtx.strokeStyle = color;
1949     overlayCtx.lineWidth = 2;
1950     overlayCtx.setLineDash([6, 4]);
1951     overlayCtx.beginPath();
1952     overlayCtx.arc(p2.x, p2.y, r, start, end, false);

```

```

1953     overlayCtx.stroke();
1954     overlayCtx.setLineDash([]);
1955
1956     overlayCtx.restore();
1957
1958     return { r, start, sweep };
1959 }
1960
1961 function midpoint(p1, p2) {
1962     return { x: (p1.x + p2.x) / 2, y: (p1.y + p2.y) / 2 };
1963 }
1964
1965 function centroid(points) {
1966     const n = points.length;
1967     const sum = points.reduce((acc, p) => ({ x: acc.x + p.x, y: acc.y + p.y }), { x: 0, y: 0 });
1968     return { x: sum.x / n, y: sum.y / n };
1969 }
1970
1971 // ===== COORDINATE CONVERSION =====
1972 function canvasCoords(e) {
1973     const rect = elements.overlayCanvas.getBoundingClientRect();
1974     const scale = state.zoom / 100;
1975     return {
1976         x: (e.clientX - rect.left) / scale,
1977         y: (e.clientY - rect.top) / scale
1978     };
1979 }
1980
1981 // ===== GUIDANCE SYSTEM =====
1982 function hasAnyMeasurement() {
1983     return state.lengths.length > 0 || state.angles.length > 0 || state.areas.length > 0 || state.particles.length > 0 ||
1984         state.holes.length > 0;
1985 }
1986
1987 function updateGuidance() {
1988     if (!elements.quickStartOverlay) return;
1989
1990     const step1 = !!state.image;
1991     const step2 = !!state.isCalibrated;
1992     const step3 = hasAnyMeasurement();
1993     const allDone = step1 && step2 && step3;
1994
1995     if (elements.quickStartStep1) elements.quickStartStep1.classList.toggle('done', step1);
1996     if (elements.quickStartStep2) elements.quickStartStep2.classList.toggle('done', step2);
1997     if (elements.quickStartStep3) elements.quickStartStep3.classList.toggle('done', step3);
1998
1999     const steps = [elements.quickStartStep1, elements.quickStartStep2, elements.quickStartStep3].filter(Boolean);
2000     steps.forEach(el => el.classList.remove('current'));
2001
2002     const current = !step1
2003         ? elements.quickStartStep1
2004         : (!step2 ? elements.quickStartStep2 : (!step3 ? elements.quickStartStep3 : null));
2005     if (current) current.classList.add('current');
2006
2007     const shouldShow = !state.quickStartDismissed && !allDone;
2008     elements.quickStartOverlay.classList.toggle('hidden', !shouldShow);
2009 }
2010
2011 // Update the Save Image button state
2012 function updateSaveButtonState() {
2013     const hasMeasurements = state.lengths.length > 0 || state.angles.length > 0 || state.areas.length > 0 ||
2014         state.particles.length > 0 || state.holes.length > 0;
2015     const canSave = state.image && hasMeasurements;
2016     elements.saveScreenshot.disabled = !canSave;
2017 }
2018
2019 // ===== CALIBRATION =====
2020 function setScale() {
2021     if (!state.image || state.isSettingScale) return;
2022     state.isSettingScale = true;
2023     state.scalePoints = [];
2024     elements.calibrationInputs.classList.remove('hidden');
2025     elements.setScaleBtn.textContent = 'Drawing...';
2026     elements.setScaleBtn.disabled = true;
2027 }
2028
2029 function saveScale() {
2030     const knownLength = parseFloat(elements.knownLength.value);
2031     if (!knownLength || knownLength <= 0 || state.scalePoints.length !== 2) return;

```

```

2032     const p2 = state.scalePoints[1];
2033     const pixelLength = Math.sqrt(Math.pow(p2.x - p1.x, 2) + Math.pow(p2.y - p1.y, 2));
2034
2035     state.unitsPerPixel = knownLength / pixelLength;
2036     state.unit = elements.unitSelect.value;
2037     state.isCalibrated = true;
2038     state.calibrationLine = { p1, p2 };
2039     state.isSettingScale = false;
2040     state.scalePoints = [];
2041
2042     elements.calibrationInputs.classList.add('hidden');
2043     elements.scaleInfo.classList.remove('hidden');
2044     elements.scaleValue.textContent = state.unitsPerPixel.toFixed(4) + ' ' + state.unit + '/px';
2045     elements.setScaleBtn.textContent = 'Set Scale';
2046     elements.setScaleBtn.disabled = false;
2047
2048     // Enable min area inputs and update labels to use real units
2049     elements.minParticleArea.disabled = false;
2050     elements.minHoleArea.disabled = false;
2051     elements.minParticleAreaLabel.textContent = `Min Area (${state.unit})2`;
2052     elements.minHoleAreaLabel.textContent = `Min Area (${state.unit})2`;
2053
2054     // Convert current pixel values to real units
2055     const currentParticleAreaPx = parseFloat(elements.minParticleArea.value) || 10;
2056     const currentHoleAreaPx = parseFloat(elements.minHoleArea.value) || 10;
2057     const areaConversion = Math.pow(state.unitsPerPixel, 2);
2058
2059     elements.minParticleArea.value = (currentParticleAreaPx * areaConversion).toFixed(4);
2060     elements.minHoleArea.value = (currentHoleAreaPx * areaConversion).toFixed(4);
2061
2062     // Update placeholder text
2063     elements.minParticleArea.nextElementSibling.textContent = `Value in (${state.unit})2`;
2064     elements.minHoleArea.nextElementSibling.textContent = `Value in (${state.unit})2`;
2065
2066     drawOverlay();
2067     updateMeasurementDisplays();
2068     updateGuidance();
2069 }
2070
2071 function cancelScale() {
2072     state.isSettingScale = false;
2073     state.scalePoints = [];
2074     elements.calibrationInputs.classList.add('hidden');
2075     elements.setScaleBtn.textContent = 'Set Scale';
2076     elements.setScaleBtn.disabled = false;
2077     drawOverlay();
2078 }
2079
2080 function resetScale() {
2081     state.isCalibrated = false;
2082     state.calibrationLine = null;
2083     state.unitsPerPixel = 0;
2084
2085     elements.scaleInfo.classList.add('hidden');
2086     elements.setScaleBtn.disabled = false;
2087     elements.setScaleBtn.textContent = 'Set Scale';
2088
2089     // Reset min area inputs back to pixels
2090     elements.minParticleArea.disabled = true;
2091     elements.minHoleArea.disabled = true;
2092     elements.minParticleAreaLabel.textContent = 'Min Area (px2)';
2093     elements.minHoleAreaLabel.textContent = 'Min Area (px2)';
2094
2095     // Convert current real values back to pixels (or reset to default)
2096     const currentParticleAreaReal = parseFloat(elements.minParticleArea.value);
2097     const currentHoleAreaReal = parseFloat(elements.minHoleArea.value);
2098
2099     // If we had a previous calibration, convert back; otherwise use default
2100     if (state.unitsPerPixel > 0) {
2101         const areaConversion = Math.pow(state.unitsPerPixel, 2);
2102         elements.minParticleArea.value = Math.round(currentParticleAreaReal / areaConversion);
2103         elements.minHoleArea.value = Math.round(currentHoleAreaReal / areaConversion);
2104     } else {
2105         elements.minParticleArea.value = 10;
2106         elements.minHoleArea.value = 10;
2107     }
2108
2109     // Update placeholder text
2110     elements.minParticleArea.nextElementSibling.textContent = 'Set scale first to enable';
2111     elements.minHoleArea.nextElementSibling.textContent = 'Set scale first to enable';
2112

```

```

2113 // Recalculate measurements to remove real values
2114 state.lengths.forEach(len => {
2115   len.realLength = null;
2116 });
2117 state.areas.forEach(area => {
2118   area.realArea = null;
2119 });
2120 state.particles.forEach(p => {
2121   p.areaReal = null;
2122   p.ecdReal = null;
2123 });
2124 state.holes.forEach(h => {
2125   h.areaReal = null;
2126   h.ecdReal = null;
2127 });
2128
2129 updateLengthDisplay();
2130 updateAreaDisplay();
2131 if (state.particles.length) updateParticleResults();
2132 if (state.holes.length) updateHoleResults();
2133 drawOverlay();
2134 updateGuidance();
2135 }
2136
2137 // ===== LENGTH MEASUREMENT =====
2138 function measureLength(p1, p2) {
2139   const pixelLength = Math.sqrt(Math.pow(p2.x - p1.x, 2) + Math.pow(p2.y - p1.y, 2));
2140   const realLength = state.isCalibrated ? pixelLength * state.unitsPerPixel : null;
2141   return { p1, p2, pixelLength, realLength };
2142 }
2143
2144 function addLengthMeasurement(len) {
2145   state.lengths.push(len);
2146   updateLengthDisplay();
2147   drawOverlay();
2148   updateGuidance();
2149   updateSaveButtonState();
2150 }
2151
2152 function updateLengthDisplay() {
2153   elements.lengthMeasurements.innerHTML = state.lengths.map((len, i) => `
2154     <div class="measurement-item">
2155       <div>
2156         <span class="font-medium">L${i+1}</span>
2157         <span class="mono">${formatNumber(len.pixelLength)} px</span>
2158         ${state.isCalibrated ? `<span class="mono text-blue-400 ml-2">${formatNumber(len.realLength)} ${state.unit}</span>` : ''}
2159       </div>
2160       <div class="flex items-center gap-2">
2161         <button onclick="toggleLengthVisibility(${i})" class="text-slate-400 hover:text-blue-400 text-sm"
2162           title="${len.visible !== false ? 'Hide' : 'Show'}">
2163           ${len.visible !== false ? '■' : '■'}
2164         </button>
2165         <button onclick="editLengthMeasurement(${i})" class="text-slate-400 hover:text-yellow-400 text-sm"
2166           title="Edit">✎</button>
2167         <button onclick="deleteLengthMeasurement(${i})" class="text-red-400 hover:text-red-300 text-lg
2168           leading-none">&times;</button>
2169       </div>
2170     </div>
2171   `).join('');
2172   elements.clearLengths.classList.toggle('hidden', state.lengths.length === 0);
2173   updateStatsButtonsVisibility();
2174 }
2175
2176 window.deleteLengthMeasurement = function(index) {
2177   state.lengths.splice(index, 1);
2178   updateLengthDisplay();
2179   drawOverlay();
2180   updateSaveButtonState();
2181 };
2182
2183 window.toggleLengthVisibility = function(index) {
2184   state.lengths[index].visible = state.lengths[index].visible === false ? true : false;
2185   updateLengthDisplay();
2186   drawOverlay();
2187 };
2188
2189 window.editLengthMeasurement = function(index) {
2190   state.editingLengthIndex = index;
2191   state.currentTool = 'length';

```

```

2190 // Switch to length tab UI
2191 elements.tabBtns.forEach(b => b.classList.remove('active'));
2192 document.querySelector('[data-tab="length"]').classList.add('active');
2193 document.querySelectorAll('.tab-content').forEach(content => content.classList.add('hidden'));
2194 document.getElementById('lengthTab').classList.remove('hidden');
2195
2196 // Show edit mode controls
2197 elements.editLengthModeControls.classList.remove('hidden');
2198 elements.editingLengthLabel.textContent = `L${index + 1}`;
2199
2200 drawOverlay();
2201 };
2202
2203 function saveLengthEdit() {
2204   if (state.editingLengthIndex !== null) {
2205     const len = state.lengths[state.editingLengthIndex];
2206     const pixelLength = Math.sqrt(Math.pow(len.p2.x - len.p1.x, 2) + Math.pow(len.p2.y - len.p1.y, 2));
2207     len.pixelLength = pixelLength;
2208     len.realLength = state.isCalibrated ? pixelLength * state.unitsPerPixel : null;
2209
2210     exitLengthEditMode();
2211     updateLengthDisplay();
2212   }
2213 }
2214
2215 function cancelLengthEdit() {
2216   exitLengthEditMode();
2217 }
2218
2219 function exitLengthEditMode() {
2220   state.editingLengthIndex = null;
2221   state.draggedLengthPoint = null;
2222   elements.editLengthModeControls.classList.add('hidden');
2223   drawOverlay();
2224 }
2225
2226 function clearAllLengths() {
2227   state.lengths = [];
2228   state.lengthPoints = [];
2229   updateLengthDisplay();
2230   drawOverlay();
2231   updateSaveButtonState();
2232 }
2233
2234 // ===== ANGLE MEASUREMENT =====
2235 function computeAngleMetrics(p1, p2, p3) {
2236   const v1x = p1.x - p2.x;
2237   const v1y = p1.y - p2.y;
2238   const v2x = p3.x - p2.x;
2239   const v2y = p3.y - p2.y;
2240   const mag1 = Math.hypot(v1x, v1y);
2241   const mag2 = Math.hypot(v2x, v2y);
2242   if (mag1 === 0 || mag2 === 0) {
2243     return { minorDeg: 0, majorDeg: 0 };
2244   }
2245
2246   // Use atan2(cross, dot) so we can derive both minor + reflex angles reliably.
2247   const dot = v1x * v2x + v1y * v2y;
2248   const cross = v1x * v2y - v1y * v2x;
2249   let angleRad = Math.atan2(Math.abs(cross), dot); // [0, π]
2250   if (!Number.isFinite(angleRad)) angleRad = 0;
2251
2252   const minorDeg = angleRad * (180 / Math.PI);
2253   // Reflex is 360 - minor; suppress degenerate 360 when minor is 0.
2254   const majorDeg = minorDeg < 1e-9 ? 0 : (360 - minorDeg);
2255   return { minorDeg, majorDeg };
2256 }
2257
2258 function computeAngleDeg(p1, p2, p3) {
2259   // Backwards-compatible: returns minor angle (0..180)
2260   return computeAngleMetrics(p1, p2, p3).minorDeg;
2261 }
2262
2263 function computeSelectedAngleDeg(p1, p2, p3, reflex) {
2264   const m = computeAngleMetrics(p1, p2, p3);
2265   return reflex ? m.majorDeg : m.minorDeg;
2266 }
2267
2268 function addAngleMeasurement(p1, p2, p3) {
2269   const reflex = false;
2270   const angleDeg = computeSelectedAngleDeg(p1, p2, p3, reflex);

```

```

2271     state.angles.push({ p1, p2, p3, angleDeg, reflex });
2272     updateAngleDisplay();
2273     drawOverlay();
2274     updateGuidance();
2275     updateSaveButtonState();
2276 }
2277
2278 function updateAngleDisplay() {
2279     if (!elements.angleMeasurements) return;
2280
2281     elements.angleMeasurements.innerHTML = state.angles.map((ang, i) => `
2282         <div class="measurement-item">
2283             <div>
2284                 <span class="font-medium">Ang${i + 1}</span>
2285                 <span class="mono">${formatNumber(ang.angleDeg)}°</span>
2286                 ${ang.reflex ? `<span class="text-xs text-slate-400 ml-2">(reflex)</span>` : ``}
2287             </div>
2288             <div class="flex items-center gap-2">
2289                 <button onclick="toggleAngleVisibility(${i})" class="text-slate-400 hover:text-cyan-400 text-sm"
2290                     title="${ang.visible !== false ? 'Hide' : 'Show'}">
2291                     ${ang.visible !== false ? '■' : '■'}
2292                 </button>
2293                 <button onclick="toggleAngleReflex(${i})" class="text-slate-400 hover:text-blue-300 text-sm" title="Toggle
2294                     minor/reflex angle">■</button>
2295                 <button onclick="editAngleMeasurement(${i})" class="text-slate-400 hover:text-yellow-400 text-sm"
2296                     title="Edit">■</button>
2297                 <button onclick="deleteAngleMeasurement(${i})" class="text-red-400 hover:text-red-300 text-lg
2298                     leading-none">&times;</button>
2299             </div>
2300         </div>
2301     `).join('');
2302
2303     if (elements.clearAngles) elements.clearAngles.classList.toggle('hidden', state.angles.length === 0);
2304     updateStatsButtonsVisibility();
2305 }
2306
2307 window.deleteAngleMeasurement = function(index) {
2308     state.angles.splice(index, 1);
2309     updateAngleDisplay();
2310     drawOverlay();
2311     updateSaveButtonState();
2312 };
2313
2314 window.toggleAngleVisibility = function(index) {
2315     state.angles[index].visible = state.angles[index].visible === false ? true : false;
2316     updateAngleDisplay();
2317     drawOverlay();
2318 };
2319
2320 window.toggleAngleReflex = function(index) {
2321     const ang = state.angles[index];
2322     if (!ang) return;
2323     ang.reflex = !ang.reflex;
2324     ang.angleDeg = computeSelectedAngleDeg(ang.p1, ang.p2, ang.p3, !!ang.reflex);
2325     updateAngleDisplay();
2326     drawOverlay();
2327 };
2328
2329 window.editAngleMeasurement = function(index) {
2330     state.editingAngleIndex = index;
2331     state.draggedAnglePointIndex = null;
2332     state.currentTool = 'angle';
2333
2334     const a = state.angles[index];
2335     state.angleEditBackup = {
2336         p1: { ...a.p1 },
2337         p2: { ...a.p2 },
2338         p3: { ...a.p3 },
2339         angleDeg: a.angleDeg,
2340         reflex: !!a.reflex
2341     };
2342
2343     // Switch to angle tab UI
2344     elements.tabBtns.forEach(b => b.classList.remove('active'));
2345     const angleBtn = document.querySelector('[data-tab="angle"]');
2346     if (angleBtn) angleBtn.classList.add('active');
2347     document.querySelectorAll('.tab-content').forEach(content => content.classList.add('hidden'));
2348     const tab = document.getElementById('angleTab');
2349     if (tab) tab.classList.remove('hidden');
2350
2351     if (elements.editAngleModeControls) elements.editAngleModeControls.classList.remove('hidden');

```



```

2348     if (elements.editingAngleLabel) elements.editingAngleLabel.textContent = `Ang${index + 1}`;
2349
2350     drawOverlay();
2351 };
2352
2353 function saveAngleEdit() {
2354     if (state.editingAngleIndex === null) return;
2355     const a = state.angles[state.editingAngleIndex];
2356     a.angleDeg = computeSelectedAngleDeg(a.p1, a.p2, a.p3, !!a.reflex);
2357     exitAngleEditMode();
2358     updateAngleDisplay();
2359 }
2360
2361 function cancelAngleEdit() {
2362     if (state.editingAngleIndex === null) return;
2363     if (state.angleEditBackup) {
2364         const a = state.angles[state.editingAngleIndex];
2365         a.p1 = { ...state.angleEditBackup.p1 };
2366         a.p2 = { ...state.angleEditBackup.p2 };
2367         a.p3 = { ...state.angleEditBackup.p3 };
2368         a.angleDeg = state.angleEditBackup.angleDeg;
2369         a.reflex = !!state.angleEditBackup.reflex;
2370     }
2371     exitAngleEditMode();
2372     updateAngleDisplay();
2373 }
2374
2375 function exitAngleEditMode() {
2376     state.editingAngleIndex = null;
2377     state.draggedAnglePointIndex = null;
2378     state.angleEditBackup = null;
2379     if (elements.editAngleModeControls) elements.editAngleModeControls.classList.add('hidden');
2380     drawOverlay();
2381 }
2382
2383 function clearAllAngles() {
2384     state.angles = [];
2385     state.anglePoints = [];
2386     exitAngleEditMode();
2387     updateAngleDisplay();
2388     drawOverlay();
2389     updateSaveButtonState();
2390 }
2391
2392 // ===== AREA MEASUREMENT =====
2393 function measurePolygonArea(points) {
2394     if (points.length < 3) return 0;
2395     let area = 0;
2396     const n = points.length;
2397     for (let i = 0; i < n; i++) {
2398         const j = (i + 1) % n;
2399         area += points[i].x * points[j].y;
2400         area -= points[j].x * points[i].y;
2401     }
2402     return Math.abs(area / 2);
2403 }
2404
2405 function measureCircleArea(radiusPx) {
2406     const r = Math.max(0, radiusPx || 0);
2407     return Math.PI * r * r;
2408 }
2409
2410 function addAreaMeasurement(points) {
2411     const pixelArea = measurePolygonArea(points);
2412     const realArea = state.isCalibrated ? pixelArea * Math.pow(state.unitsPerPixel, 2) : null;
2413     state.areas.push({ points: [...points], pixelArea, realArea });
2414     updateAreaDisplay();
2415     drawOverlay();
2416     updateGuidance();
2417     updateSaveButtonState();
2418 }
2419
2420 function addCircleAreaMeasurement(center, radiusPx) {
2421     const pixelArea = measureCircleArea(radiusPx);
2422     const realArea = state.isCalibrated ? pixelArea * Math.pow(state.unitsPerPixel, 2) : null;
2423     state.areas.push({ type: 'circle', center: { ...center }, radiusPx, pixelArea, realArea });
2424     updateAreaDisplay();
2425     drawOverlay();
2426     updateGuidance();
2427     updateSaveButtonState();
2428 }

```

```

2429
2430 function updateAreaDisplay() {
2431     elements.areaMeasurements.innerHTML = state.areas.map((area, i) => `
2432         <div class="measurement-item">
2433             <div>
2434                 <span class="font-medium">A${i+1}</span>
2435                 <span class="mono">${formatNumber(area.pixelArea)} px2</span>
2436                 ${state.isCalibrated ? `<span class="mono text-green-400 ml-2">${formatNumber(area.realArea)} ${state.unit}2</span>` : ''}
2437             </div>
2438             <div class="flex items-center gap-2">
2439                 <button onclick="toggleAreaVisibility(${i})" class="text-slate-400 hover:text-green-400 text-sm"
2440                     title="${area.visible !== false ? 'Hide' : 'Show'}">
2441                     ${area.visible !== false ? '■' : '■'}
2442                 </button>
2443                 ${area.type === 'circle' ? '' : `<button onclick="editAreaMeasurement(${i})" class="text-slate-400
2444                     hover:text-yellow-400 text-sm" title="Edit">—■</button>`}
2445                 <button onclick="deleteAreaMeasurement(${i})" class="text-red-400 hover:text-red-300 text-lg
2446                     leading-none">&times;</button>
2447             </div>
2448             </div>
2449             `).join('');
2450     elements.clearAreas.classList.toggle('hidden', state.areas.length === 0);
2451     updateStatsButtonsVisibility();
2452 }
2453
2454 window.deleteAreaMeasurement = function(index) {
2455     state.areas.splice(index, 1);
2456     updateAreaDisplay();
2457     drawOverlay();
2458     updateSaveButtonState();
2459 };
2460
2461 window.toggleAreaVisibility = function(index) {
2462     state.areas[index].visible = state.areas[index].visible === false ? true : false;
2463     updateAreaDisplay();
2464     drawOverlay();
2465 };
2466
2467 window.editAreaMeasurement = function(index) {
2468     const a = state.areas[index];
2469     if (!a || a.type === 'circle') return;
2470
2471     state.editingAreaIndex = index;
2472     state.currentTool = 'area';
2473
2474     // Switch to area tab UI
2475     elements.tabBtns.forEach(b => b.classList.remove('active'));
2476     document.querySelector('[data-tab="area"]').classList.add('active');
2477     document.querySelectorAll('.tab-content').forEach(content => content.classList.add('hidden'));
2478     document.getElementById('areaTab').classList.remove('hidden');
2479
2480     // Show edit mode controls
2481     elements.areaInstructions.classList.add('hidden');
2482     elements.editModeControls.classList.remove('hidden');
2483     elements.editingAreaLabel.textContent = `A${index + 1}`;
2484
2485     drawOverlay();
2486 };
2487
2488 function saveAreaEdit() {
2489     if (state.editingAreaIndex !== null) {
2490         const area = state.areas[state.editingAreaIndex];
2491         if (!area || !Array.isArray(area.points)) {
2492             exitAreaEditMode();
2493             updateAreaDisplay();
2494             return;
2495         }
2496         area.pixelArea = measurePolygonArea(area.points);
2497         area.realArea = state.isCalibrated ? area.pixelArea * Math.pow(state.unitsPerPixel, 2) : null;
2498
2499         exitAreaEditMode();
2500         updateAreaDisplay();
2501     }
2502 }
2503
2504 function cancelAreaEdit() {
2505     exitAreaEditMode();
2506 }
2507
2508 function exitAreaEditMode() {

```

```

2506     state.editingAreaIndex = null;
2507     state.draggedPointIndex = null;
2508     elements.areaInstructions.classList.remove('hidden');
2509     elements.editModeControls.classList.add('hidden');
2510     drawOverlay();
2511 }
2512
2513 function clearAllAreas() {
2514     exitAreaEditMode();
2515     state.areas = [];
2516     state.areaPoints = [];
2517     state.isDrawingCircle = false;
2518     state.circleDraft = null;
2519     elements.closePolygonBtn.classList.add('hidden');
2520     updateAreaDisplay();
2521     drawOverlay();
2522     updateSaveButtonState();
2523 }
2524
2525 function setAreaMode(mode) {
2526     if (state.editingAreaIndex !== null) return;
2527     if (mode !== 'polygon' && mode !== 'circle') return;
2528
2529     state.areaMode = mode;
2530     state.areaPoints = [];
2531     state.isDrawingCircle = false;
2532     state.circleDraft = null;
2533
2534     elements.closePolygonBtn.classList.add('hidden');
2535
2536     if (mode === 'polygon') {
2537         if (elements.areaInstructions) elements.areaInstructions.textContent = 'Polygon: click points to create polygon.';
2538         if (elements.areaModePolygonBtn) {
2539             elements.areaModePolygonBtn.classList.add('btn-primary');
2540             elements.areaModePolygonBtn.classList.remove('btn-secondary');
2541         }
2542         if (elements.areaModeCircleBtn) {
2543             elements.areaModeCircleBtn.classList.add('btn-secondary');
2544             elements.areaModeCircleBtn.classList.remove('btn-primary');
2545         }
2546     } else {
2547         if (elements.areaInstructions) elements.areaInstructions.textContent = 'Circle: click and drag to draw a circle.';
2548         if (elements.areaModePolygonBtn) {
2549             elements.areaModePolygonBtn.classList.add('btn-secondary');
2550             elements.areaModePolygonBtn.classList.remove('btn-primary');
2551         }
2552         if (elements.areaModeCircleBtn) {
2553             elements.areaModeCircleBtn.classList.add('btn-primary');
2554             elements.areaModeCircleBtn.classList.remove('btn-secondary');
2555         }
2556     }
2557
2558     drawOverlay();
2559 }
2560
2561 // ===== PARTICLE/HOLE ANALYSIS =====
2562 function getImageData() {
2563     if (!state.imageData) {
2564         const canvas = document.createElement('canvas');
2565         canvas.width = state.image.width;
2566         canvas.height = state.image.height;
2567         const ctx = canvas.getContext('2d');
2568         ctx.drawImage(state.image, 0, 0);
2569         state.imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);
2570     }
2571     return state.imageData;
2572 }
2573
2574 function toGrayscale(imageData) {
2575     const data = imageData.data;
2576     const gray = new Uint8Array(imageData.width * imageData.height);
2577     for (let i = 0; i < gray.length; i++) {
2578         const idx = i * 4;
2579         gray[i] = Math.round(0.299 * data[idx] + 0.587 * data[idx + 1] + 0.114 * data[idx + 2]);
2580     }
2581     return gray;
2582 }
2583
2584 function blur3x3(gray, width, height) {
2585     const result = new Uint8Array(gray.length);
2586     const kernel = [1, 2, 1, 2, 4, 2, 1, 2, 1];

```

```

2587     const kernelSum = 16;
2588
2589     for (let y = 1; y < height - 1; y++) {
2590         for (let x = 1; x < width - 1; x++) {
2591             let sum = 0;
2592             let ki = 0;
2593             for (let ky = -1; ky <= 1; ky++) {
2594                 for (let kx = -1; kx <= 1; kx++) {
2595                     sum += gray[(y + ky) * width + (x + kx)] * kernel[ki++];
2596                 }
2597             }
2598             result[y * width + x] = Math.round(sum / kernelSum);
2599         }
2600     }
2601     return result;
2602 }
2603
2604 function medianFilter(gray, width, height, kernelSize) {
2605     const result = new Uint8Array(gray.length);
2606     const half = Math.floor(kernelSize / 2);
2607     const values = [];
2608
2609     for (let y = 0; y < height; y++) {
2610         for (let x = 0; x < width; x++) {
2611             values.length = 0;
2612
2613             for (let ky = -half; ky <= half; ky++) {
2614                 for (let kx = -half; kx <= half; kx++) {
2615                     const ny = y + ky;
2616                     const nx = x + kx;
2617                     if (ny >= 0 && ny < height && nx >= 0 && nx < width) {
2618                         values.push(gray[ny * width + nx]);
2619                     }
2620                 }
2621             }
2622
2623             values.sort((a, b) => a - b);
2624             result[y * width + x] = values[Math.floor(values.length / 2)];
2625         }
2626     }
2627
2628     return result;
2629 }
2630
2631 function backgroundSubtractRollingBall(gray, width, height, radius) {
2632     // Gaussian blur approximation of rolling ball
2633     const background = new Uint8Array(gray.length);
2634     const result = new Uint8Array(gray.length);
2635
2636     // Create large Gaussian kernel
2637     const kernelSize = radius * 2 + 1;
2638     const half = radius;
2639     const sigma = radius / 3;
2640     const kernel = [];
2641     let kernelSum = 0;
2642
2643     for (let y = -half; y <= half; y++) {
2644         for (let x = -half; x <= half; x++) {
2645             const weight = Math.exp(-(x * x + y * y) / (2 * sigma * sigma));
2646             kernel.push(weight);
2647             kernelSum += weight;
2648         }
2649     }
2650
2651     // Normalize kernel
2652     for (let i = 0; i < kernel.length; i++) {
2653         kernel[i] /= kernelSum;
2654     }
2655
2656     // Apply large blur to estimate background
2657     for (let y = 0; y < height; y++) {
2658         for (let x = 0; x < width; x++) {
2659             let sum = 0;
2660             let ki = 0;
2661
2662             for (let ky = -half; ky <= half; ky++) {
2663                 for (let kx = -half; kx <= half; kx++) {
2664                     const ny = y + ky;
2665                     const nx = x + kx;
2666                     if (ny >= 0 && ny < height && nx >= 0 && nx < width) {
2667                         sum += gray[ny * width + nx] * kernel[ki];

```

```

2668     }
2669     ki++;
2670 }
2671 }
2672
2673     background[y * width + x] = Math.round(sum);
2674 }
2675 }
2676
2677 // Subtract background
2678 for (let i = 0; i < gray.length; i++) {
2679     const val = gray[i] - background[i] + 128;
2680     result[i] = Math.max(0, Math.min(255, val));
2681 }
2682
2683 return result;
2684 }
2685
2686 function otsuThreshold(gray) {
2687     // Compute histogram
2688     const histogram = new Array(256).fill(0);
2689     for (let i = 0; i < gray.length; i++) {
2690         histogram[gray[i]]++;
2691     }
2692
2693     const total = gray.length;
2694     let sum = 0;
2695     for (let i = 0; i < 256; i++) {
2696         sum += i * histogram[i];
2697     }
2698
2699     let sumB = 0;
2700     let wB = 0;
2701     let wF = 0;
2702     let maxVariance = 0;
2703     let threshold = 0;
2704
2705     for (let t = 0; t < 256; t++) {
2706         wB += histogram[t];
2707         if (wB === 0) continue;
2708
2709         wF = total - wB;
2710         if (wF === 0) break;
2711
2712         sumB += t * histogram[t];
2713
2714         const mB = sumB / wB;
2715         const mF = (sum - sumB) / wF;
2716
2717         const variance = wB * wF * (mB - mF) * (mB - mF);
2718
2719         if (variance > maxVariance) {
2720             maxVariance = variance;
2721             threshold = t;
2722         }
2723     }
2724
2725     return threshold;
2726 }
2727
2728 function adaptiveThreshold(gray, width, height, blockSize, brightIsObject) {
2729     const binary = new Uint8Array(gray.length);
2730     const half = Math.floor(blockSize / 2);
2731
2732     for (let y = 0; y < height; y++) {
2733         for (let x = 0; x < width; x++) {
2734             // Calculate local mean
2735             let sum = 0;
2736             let count = 0;
2737
2738             for (let ky = -half; ky <= half; ky++) {
2739                 for (let kx = -half; kx <= half; kx++) {
2740                     const ny = y + ky;
2741                     const nx = x + kx;
2742                     if (ny >= 0 && ny < height && nx >= 0 && nx < width) {
2743                         sum += gray[ny * width + nx];
2744                         count++;
2745                     }
2746                 }
2747             }
2748

```

```

2749     const localMean = sum / count;
2750     const threshold = localMean - 5; // Small constant
2751
2752     const i = y * width + x;
2753     if (brightIsObject) {
2754         binary[i] = gray[i] >= threshold ? 1 : 0;
2755     } else {
2756         binary[i] = gray[i] < threshold ? 1 : 0;
2757     }
2758 }
2759 }
2760
2761 return binary;
2762 }
2763
2764 function morphologicalErode(binary, width, height) {
2765     const result = new Uint8Array(binary.length);
2766
2767     for (let y = 0; y < height; y++) {
2768         for (let x = 0; x < width; x++) {
2769             const i = y * width + x;
2770
2771             if (binary[i] === 0) {
2772                 result[i] = 0;
2773                 continue;
2774             }
2775
2776             // Check 8-connected neighbors
2777             let isEdge = false;
2778             for (let ky = -1; ky <= 1; ky++) {
2779                 for (let kx = -1; kx <= 1; kx++) {
2780                     const ny = y + ky;
2781                     const nx = x + kx;
2782                     if (ny >= 0 && ny < height && nx >= 0 && nx < width) {
2783                         if (binary[ny * width + nx] === 0) {
2784                             isEdge = true;
2785                             break;
2786                         }
2787                     }
2788                 }
2789                 if (isEdge) break;
2790             }
2791
2792             result[i] = isEdge ? 0 : 1;
2793         }
2794     }
2795
2796     return result;
2797 }
2798
2799 function morphologicalDilate(binary, width, height) {
2800     const result = new Uint8Array(binary.length);
2801
2802     for (let y = 0; y < height; y++) {
2803         for (let x = 0; x < width; x++) {
2804             const i = y * width + x;
2805
2806             if (binary[i] === 1) {
2807                 result[i] = 1;
2808                 continue;
2809             }
2810
2811             // Check 8-connected neighbors
2812             let hasNeighbor = false;
2813             for (let ky = -1; ky <= 1; ky++) {
2814                 for (let kx = -1; kx <= 1; kx++) {
2815                     const ny = y + ky;
2816                     const nx = x + kx;
2817                     if (ny >= 0 && ny < height && nx >= 0 && nx < width) {
2818                         if (binary[ny * width + nx] === 1) {
2819                             hasNeighbor = true;
2820                             break;
2821                         }
2822                     }
2823                 }
2824                 if (hasNeighbor) break;
2825             }
2826
2827             result[i] = hasNeighbor ? 1 : 0;
2828         }
2829     }

```

```

2830     return result;
2831 }
2832 }
2833
2834 function morphologicalOpening(binary, width, height) {
2835     const eroded = morphologicalErode(binary, width, height);
2836     return morphologicalDilate(eroded, width, height);
2837 }
2838
2839 function thresholdAndLabelComponents(gray, width, height, threshold, brightIsObject, minArea, useAdaptive, blockSize, roi) {
2840     // Default ROI is the entire image
2841     if (!roi) {
2842         roi = { x: 0, y: 0, width: width, height: height };
2843     }
2844
2845     // Ensure ROI is within bounds
2846     const roiX = Math.max(0, Math.floor(roi.x));
2847     const roiY = Math.max(0, Math.floor(roi.y));
2848     const roiWidth = Math.min(width - roiX, Math.ceil(roi.width));
2849     const roiHeight = Math.min(height - roiY, Math.ceil(roi.height));
2850
2851     let binary;
2852
2853     if (useAdaptive) {
2854         binary = adaptiveThreshold(gray, width, height, blockSize, brightIsObject);
2855     } else {
2856         // Standard global threshold
2857         binary = new Uint8Array(gray.length);
2858         for (let i = 0; i < gray.length; i++) {
2859             if (brightIsObject) {
2860                 binary[i] = gray[i] >= threshold ? 1 : 0;
2861             } else {
2862                 binary[i] = gray[i] < threshold ? 1 : 0;
2863             }
2864         }
2865     }
2866
2867     // Apply morphological opening if requested
2868     if (elements.morphologicalOpening && elements.morphologicalOpening.checked) {
2869         binary = morphologicalOpening(binary, width, height);
2870     }
2871
2872     // Connected component labeling (8-connected) - only within ROI
2873     const labels = new Int32Array(gray.length);
2874     let currentLabel = 0;
2875     const equivalences = new Map();
2876
2877     function find(label) {
2878         if (!equivalences.has(label)) return label;
2879         let root = label;
2880         while (equivalences.has(root)) root = equivalences.get(root);
2881         // Path compression
2882         let l = label;
2883         while (equivalences.has(l)) {
2884             const next = equivalences.get(l);
2885             equivalences.set(l, root);
2886             l = next;
2887         }
2888         return root;
2889     }
2890
2891     function union(a, b) {
2892         const rootA = find(a);
2893         const rootB = find(b);
2894         if (rootA !== rootB) {
2895             equivalences.set(Math.max(rootA, rootB), Math.min(rootA, rootB));
2896         }
2897     }
2898
2899     // First pass - only within ROI
2900     for (let y = roiY; y < roiY + roiHeight; y++) {
2901         for (let x = roiX; x < roiX + roiWidth; x++) {
2902             const i = y * width + x;
2903             if (binary[i] === 0) continue;
2904
2905             const neighbors = [];
2906             if (y > roiY && x > roiX && labels[(y-1) * width + (x-1)]) neighbors.push(labels[(y-1) * width + (x-1)]);
2907             if (y > roiY && labels[(y-1) * width + x]) neighbors.push(labels[(y-1) * width + x]);
2908             if (y > roiY && x < roiX + roiWidth - 1 && labels[(y-1) * width + (x+1)]) neighbors.push(labels[(y-1) * width + (x+1)]);
2909             if (x > roiX && labels[y * width + (x-1)]) neighbors.push(labels[y * width + (x-1)]);

```

```

2910
2911     if (neighbors.length === 0) {
2912         labels[i] = ++currentLabel;
2913     } else {
2914         const minLabel = Math.min(...neighbors);
2915         labels[i] = minLabel;
2916         neighbors.forEach(n => union(minLabel, n));
2917     }
2918 }
2919 }
2920
2921 // Second pass - resolve equivalences and compute properties
2922 const components = new Map();
2923
2924 for (let y = roiY; y < roiY + roiHeight; y++) {
2925     for (let x = roiX; x < roiX + roiWidth; x++) {
2926         const i = y * width + x;
2927         if (labels[i] === 0) continue;
2928
2929         const root = find(labels[i]);
2930         labels[i] = root;
2931
2932         if (!components.has(root)) {
2933             components.set(root, {
2934                 area: 0,
2935                 minX: width, maxX: 0, minY: height, maxY: 0,
2936                 pixels: []
2937             });
2938         }
2939
2940         const comp = components.get(root);
2941         comp.area++;
2942         comp.minX = Math.min(comp.minX, x);
2943         comp.maxX = Math.max(comp.maxX, x);
2944         comp.minY = Math.min(comp.minY, y);
2945         comp.maxY = Math.max(comp.maxY, y);
2946     }
2947 }
2948
2949 // Filter by min area and compute final properties
2950 const result = [];
2951 components.forEach((comp, label) => {
2952     if (comp.area >= minArea) {
2953         const ecd = 2 * Math.sqrt(comp.area / Math.PI);
2954         result.push({
2955             id: result.length + 1,
2956             areaPx: comp.area,
2957             areaReal: state.isCalibrated ? comp.area * Math.pow(state.unitsPerPixel, 2) : null,
2958             ecdPx: ecd,
2959             ecdReal: state.isCalibrated ? ecd * state.unitsPerPixel : null,
2960             bbox: {
2961                 x: comp.minX,
2962                 y: comp.minY,
2963                 w: comp.maxX - comp.minX + 1,
2964                 h: comp.maxY - comp.minY + 1
2965             }
2966         });
2967     }
2968 });
2969
2970 return result;
2971 }
2972
2973 function computeStats(components, key) {
2974     if (components.length === 0) return { count: 0, mean: 0, std: 0 };
2975
2976     const values = components.map(c => c[key]);
2977     const mean = values.reduce((a, b) => a + b, 0) / values.length;
2978     const variance = values.reduce((a, b) => a + Math.pow(b - mean, 2), 0) / values.length;
2979
2980     return {
2981         count: components.length,
2982         mean,
2983         std: Math.sqrt(variance)
2984     };
2985 }
2986
2987 // ===== DESCRIPTIVE STATISTICS (PUBLISH MODAL) =====
2988 let currentStatsType = 'length';
2989 let lastStatsTSV = '';
2990 let statsCopyIdleText = null;

```



```

2991 let statsCopyTimer = null;
2992
2993 function quantile(sortedValues, p) {
2994   const n = sortedValues.length;
2995   if (n === 0) return NaN;
2996   if (n === 1) return sortedValues[0];
2997   const idx = (n - 1) * p;
2998   const lo = Math.floor(idx);
2999   const hi = Math.ceil(idx);
3000   const h = idx - lo;
3001   return sortedValues[lo] + (sortedValues[hi] - sortedValues[lo]) * h;
3002 }
3003
3004 function computeDescriptiveStats(values) {
3005   const clean = values.filter(v => Number.isFinite(v));
3006   const n = clean.length;
3007   if (!n) return null;
3008   const sorted = [...clean].sort((a, b) => a - b);
3009   const sum = clean.reduce((acc, v) => acc + v, 0);
3010   const mean = sum / n;
3011   const median = (n % 2 === 1) ? sorted[(n - 1) / 2] : (sorted[n / 2 - 1] + sorted[n / 2]) / 2;
3012   const min = sorted[0];
3013   const max = sorted[n - 1];
3014   const q1 = quantile(sorted, 0.25);
3015   const q3 = quantile(sorted, 0.75);
3016   const iqr = q3 - q1;
3017
3018   let std = 0;
3019   if (n > 1) {
3020     const ss = clean.reduce((acc, v) => acc + Math.pow(v - mean, 2), 0);
3021     std = Math.sqrt(ss / (n - 1)); // sample std dev
3022   }
3023   const sem = std / Math.sqrt(n);
3024   const cv = mean !== 0 ? (std / mean) * 100 : 0;
3025
3026   return { n, sum, mean, median, std, sem, min, max, q1, q3, iqr, cv };
3027 }
3028
3029 function statsUnitLabel(kind) {
3030   if (kind === 'length') return state.isCalibrated ? state.unit : 'px';
3031   if (kind === 'angle') return 'deg';
3032   if (kind === 'area') return state.isCalibrated ? `${state.unit}^2` : 'px^2';
3033   if (kind === 'ecd') return state.isCalibrated ? state.unit : 'px';
3034   if (kind === 'componentArea') return state.isCalibrated ? `${state.unit}^2` : 'px^2';
3035   return '';
3036 }
3037
3038 function getStatsSectionsForType(type) {
3039   if (type === 'length') {
3040     return [{
3041       label: 'Length',
3042       unit: statsUnitLabel('length'),
3043       values: state.lengths.map(l => state.isCalibrated ? l.realLength : l.pixelLength)
3044     }];
3045   }
3046   if (type === 'angle') {
3047     return [{
3048       label: 'Angle',
3049       unit: statsUnitLabel('angle'),
3050       values: state.angles.map(a => a.angleDeg)
3051     }];
3052   }
3053   if (type === 'area') {
3054     return [{
3055       label: 'Area',
3056       unit: statsUnitLabel('area'),
3057       values: state.areas.map(a => state.isCalibrated ? a.realArea : a.pixelArea)
3058     }];
3059   }
3060   if (type === 'particles') {
3061     return [
3062       {
3063         label: 'Particle Area',
3064         unit: statsUnitLabel('componentArea'),
3065         values: state.particles.map(p => state.isCalibrated ? p.areaReal : p.areaPx)
3066       },
3067       {
3068         label: 'Particle ECD',
3069         unit: statsUnitLabel('ecd'),
3070         values: state.particles.map(p => state.isCalibrated ? p.ecdReal : p.ecdPx)
3071       }
3072     ];
3073   }
3074 }

```

```

3072     ];
3073   }
3074   if (type === 'holes') {
3075     return [
3076       {
3077         label: 'Hole Area',
3078         unit: statsUnitLabel('componentArea'),
3079         values: state.holes.map(h => state.isCalibrated ? h.areaReal : h.areaPx)
3080       },
3081       {
3082         label: 'Hole ECD',
3083         unit: statsUnitLabel('ecd'),
3084         values: state.holes.map(h => state.isCalibrated ? h.ecdReal : h.ecdPx)
3085       }
3086     ];
3087   }
3088   return [];
3089 }
3090
3091 function renderStatsTable(stats, unit) {
3092   const fmt = (x) => Number.isFinite(x) ? formatNumber(x) : '-';
3093   const row = (label, value) => `
3094     <tr class="border-b border-slate-200">
3095       <th class="text-left font-medium py-2 pr-3 align-top text-slate-700 w-[55%]">${label}</th>
3096       <td class="py-2 mono text-slate-900">${value}</td>
3097     </tr>
3098   `;
3099
3100   return `
3101     <div class="overflow-x-auto">
3102       <table class="w-full text-xs">
3103         <tbody>
3104           ${row('Count (n)', stats.n)}
3105           ${row('Mean', `${fmt(stats.mean)} ${unit}`)}
3106           ${row('Median', `${fmt(stats.median)} ${unit}`)}
3107           ${row('Std Dev (sample)', `${fmt(stats.std)} ${unit}`)}
3108           ${row('Std Error (SEM)', `${fmt(stats.sem)} ${unit}`)}
3109           ${row('Min', `${fmt(stats.min)} ${unit}`)}
3110           ${row('Q1 (25%)', `${fmt(stats.q1)} ${unit}`)}
3111           ${row('Q3 (75%)', `${fmt(stats.q3)} ${unit}`)}
3112           ${row('Max', `${fmt(stats.max)} ${unit}`)}
3113           ${row('IQR', `${fmt(stats.iqr)} ${unit}`)}
3114           ${row('Sum', `${fmt(stats.sum)} ${unit}`)}
3115           ${row('CV (%)', `${fmt(stats.cv)} %`)}
3116         </tbody>
3117       </table>
3118     </div>
3119   `;
3120 }
3121
3122 function buildStatsTSV(type, sections) {
3123   const lines = [];
3124   lines.push(`Type\tMetric\tUnit\t\n\tMean\tMedian\tStdDev(sample)\tSEM\tMin\tQ1\tQ3\tMax\tIQR\tSum\tCV(%)`);
3125   sections.forEach(sec => {
3126     const stats = computeDescriptiveStats(sec.values);
3127     if (!stats) return;
3128     const vals = [
3129       type,
3130       sec.label,
3131       sec.unit,
3132       stats.n,
3133       stats.mean,
3134       stats.median,
3135       stats.std,
3136       stats.sem,
3137       stats.min,
3138       stats.q1,
3139       stats.q3,
3140       stats.max,
3141       stats.iqr,
3142       stats.sum,
3143       stats.cv
3144     ];
3145     lines.push(vals.join('\t'));
3146   });
3147   return lines.join('\n');
3148 }
3149
3150 function setStatsType(type) {
3151   currentStatsType = type;
3152   if (elements.statsTypeBtns) {

```

```

3153     elements.statsTypeBtns.forEach(btn => {
3154         const active = btn.dataset.statsType === type;
3155         btn.classList.toggle('bg-blue-600', active);
3156         btn.classList.toggle('text-white', active);
3157         btn.classList.toggle('border-blue-600', active);
3158         btn.classList.toggle('bg-slate-50', !active);
3159         btn.classList.toggle('text-slate-800', !active);
3160         btn.classList.toggle('border-slate-200', !active);
3161     });
3162 }
3163 renderStatsModal();
3164 }
3165
3166 function renderStatsModal() {
3167     const type = currentStatsType;
3168     const titleMap = {
3169         length: 'Length – Descriptive Statistics',
3170         angle: 'Angle – Descriptive Statistics',
3171         area: 'Area – Descriptive Statistics',
3172         particles: 'Particles – Descriptive Statistics',
3173         holes: 'Holes – Descriptive Statistics'
3174     };
3175     if (elements.statsModalTitle) elements.statsModalTitle.textContent = titleMap[type] || 'Descriptive Statistics';
3176
3177     const sections = getStatsSectionsForType(type);
3178     const totalN = sections.length ? (sections[0].values.filter(v => Number.isFinite(v)).length) : 0;
3179     const unitNote = (type === 'angle')
3180         ? 'Degrees (calibration not required)'
3181         : (state.isCalibrated ? `Real units (${state.unit})` : 'Pixels (set scale for real units)');
3182     if (elements.statsModalSubtitle) {
3183         elements.statsModalSubtitle.textContent = `Based on ${totalN} values. ${unitNote}. (Shown only when n ≥ 4)`;
3184     }
3185
3186     // Button availability: only when at least one section has n>=4
3187     const hasEnough = sections.some(sec => sec.values.filter(v => Number.isFinite(v)).length >= 4);
3188     if (elements.copyStatsTSVBtn) elements.copyStatsTSVBtn.disabled = !hasEnough;
3189
3190     if (!elements.statsModalContent) return;
3191
3192     if (!hasEnough) {
3193         elements.statsModalContent.innerHTML = `
3194             <div class="rounded-xl border border-slate-200 bg-slate-50 p-4">
3195                 <p class="text-sm font-medium text-slate-800">Not enough data yet</p>
3196                 <p class="text-xs text-slate-600 mt-1">Add at least 4 measurements/objects in this category to show descriptive
3197             statistics.</p>
3198             </div>
3199             `;
3200         lastStatsTSV = '';
3201         return;
3202     }
3203     elements.statsModalContent.innerHTML = sections.map(sec => {
3204         const n = sec.values.filter(v => Number.isFinite(v)).length;
3205         if (n < 4) {
3206             return `
3207                 <div class="rounded-xl border border-slate-200 bg-slate-50 p-4">
3208                     <div class="flex items-baseline justify-between gap-2">
3209                         <h4 class="text-sm font-semibold text-slate-900">${sec.label}</h4>
3210                         <span class="text-xs text-slate-500 mono">n=${n}</span>
3211                     </div>
3212                     <p class="text-xs text-slate-600 mt-1">Need at least 4 values for this metric.</p>
3213                 </div>
3214             `;
3215         }
3216         const stats = computeDescriptiveStats(sec.values);
3217         return `
3218             <div class="rounded-xl border border-slate-200 bg-white p-4">
3219                 <div class="flex items-baseline justify-between gap-2 mb-3">
3220                     <h4 class="text-sm font-semibold text-slate-900">${sec.label}</h4>
3221                     <span class="text-xs text-slate-500 mono">n=${stats.n} • ${sec.unit}</span>
3222                 </div>
3223                 ${renderStatsTable(stats, sec.unit)}
3224             </div>
3225             `;
3226     }).join('');
3227     lastStatsTSV = buildStatsTSV(type, sections);
3228 }
3229
3230 function openStatsModal(type) {
3231     if (!elements.descriptiveStatsModal) return;
3232

```

```

3233     elements.descriptiveStatsModal.classList.remove('hidden');
3234     setStatsType(type);
3235 }
3236
3237 function closeStatsModal() {
3238     if (!elements.descriptiveStatsModal) return;
3239     elements.descriptiveStatsModal.classList.add('hidden');
3240 }
3241
3242 function setStatsCopyFeedback(text) {
3243     const btn = elements.copyStatsTSVBtn;
3244     if (!btn) return;
3245
3246     if (statsCopyIdleText === null) statsCopyIdleText = btn.textContent;
3247     btn.textContent = text;
3248
3249     // Keep the button from being spam-clicked; restore soon.
3250     btn.disabled = true;
3251     if (statsCopyTimer) clearTimeout(statsCopyTimer);
3252     statsCopyTimer = setTimeout(() => {
3253         btn.textContent = statsCopyIdleText;
3254         // Restore correct disabled state (depends on current dataset)
3255         renderStatsModal();
3256     }, 1200);
3257 }
3258
3259 function fallbackCopyText(text) {
3260     const textarea = document.createElement('textarea');
3261     textarea.value = text;
3262     textarea.setAttribute('readonly', '');
3263     textarea.style.position = 'fixed';
3264     textarea.style.left = '-9999px';
3265     textarea.style.top = '0';
3266     document.body.appendChild(textarea);
3267     textarea.select();
3268     textarea.setSelectionRange(0, textarea.value.length);
3269     const ok = document.execCommand('copy');
3270     document.body.removeChild(textarea);
3271     return ok;
3272 }
3273
3274 async function copyStatsTSV() {
3275     if (!lastStatsTSV) {
3276         setStatsCopyFeedback('No data');
3277         return;
3278     }
3279
3280     try {
3281         if (navigator.clipboard && navigator.clipboard.writeText) {
3282             await navigator.clipboard.writeText(lastStatsTSV);
3283             setStatsCopyFeedback('Copied');
3284             return;
3285         }
3286     } catch (e) {
3287         // Fall back below
3288     }
3289
3290     const ok = fallbackCopyText(lastStatsTSV);
3291     setStatsCopyFeedback(ok ? 'Copied' : 'Copy failed');
3292 }
3293
3294 // ===== HISTOGRAM MODAL (publish-ready, light) =====
3295 let currentHistType = 'length';
3296
3297 function histogramUnitLabel(type, metric) {
3298     if (type === 'length') return statsUnitLabel('length');
3299     if (type === 'angle') return statsUnitLabel('angle');
3300     if (type === 'area') return statsUnitLabel('area');
3301     if (type === 'particles' || type === 'holes') {
3302         if (metric === 'area') return statsUnitLabel('componentArea');
3303         return statsUnitLabel('ecd');
3304     }
3305     return '';
3306 }
3307
3308 function getHistogramSeriesForType(type) {
3309     const metricPref = elements.histMetricSelect ? elements.histMetricSelect.value : 'auto';
3310     const metric = (type === 'particles' || type === 'holes')
3311         ? (metricPref === 'auto' ? 'ecd' : metricPref)
3312         : 'value';
3313 }

```

```

3314     let values = [];
3315     let label = '';
3316     let unit = '';
3317
3318     if (type === 'length') {
3319         values = state.lengths.map(l => state.isCalibrated ? l.realLength : l.pixelLength);
3320         unit = histogramUnitLabel('length');
3321         label = `Length (${unit})`;
3322     } else if (type === 'angle') {
3323         values = state.angles.map(a => a.angleDeg);
3324         unit = histogramUnitLabel('angle');
3325         label = `Angle (${unit})`;
3326     } else if (type === 'area') {
3327         values = state.areas.map(a => state.isCalibrated ? a.realArea : a.pixelArea);
3328         unit = histogramUnitLabel('area');
3329         label = `Area (${unit})`;
3330     } else if (type === 'particles') {
3331         if (metric === 'area') {
3332             values = state.particles.map(p => state.isCalibrated ? p.areaReal : p.areaPx);
3333             unit = histogramUnitLabel('particles', 'area');
3334             label = `Particle Area (${unit})`;
3335         } else {
3336             values = state.particles.map(p => state.isCalibrated ? p.ecdReal : p.ecdPx);
3337             unit = histogramUnitLabel('particles', 'ecd');
3338             label = `Particle ECD (${unit})`;
3339         }
3340     } else if (type === 'holes') {
3341         if (metric === 'area') {
3342             values = state.holes.map(h => state.isCalibrated ? h.areaReal : h.areaPx);
3343             unit = histogramUnitLabel('holes', 'area');
3344             label = `Hole Area (${unit})`;
3345         } else {
3346             values = state.holes.map(h => state.isCalibrated ? h.ecdReal : h.ecdPx);
3347             unit = histogramUnitLabel('holes', 'ecd');
3348             label = `Hole ECD (${unit})`;
3349         }
3350     }
3351
3352     const clean = values.filter(v => Number.isFinite(v));
3353     return { values: clean, label, unit, metric };
3354 }
3355
3356 function computeHistogram(values, binCount) {
3357     if (!values.length) return { bins: [], min: NaN, max: NaN };
3358     const min = Math.min(...values);
3359     const max = Math.max(...values);
3360     const range = (max - min) || 1;
3361     const k = Math.max(3, Math.min(80, Math.round(binCount)));
3362     const step = range / k;
3363
3364     const bins = new Array(k).fill(0);
3365     for (const v of values) {
3366         const idx = Math.min(k - 1, Math.max(0, Math.floor((v - min) / step)));
3367         bins[idx] += 1;
3368     }
3369     return { bins, min, max };
3370 }
3371
3372 function escapeXml(s) {
3373     return String(s)
3374         .replaceAll('&', '&amp;')
3375         .replaceAll('<', '&lt;')
3376         .replaceAll('>', '&gt;')
3377         .replaceAll('"', '&quot;')
3378         .replaceAll("'", '&apos;');
3379 }
3380
3381 function niceTicks(min, max, tickCount = 5) {
3382     if (!Number.isFinite(min) || !Number.isFinite(max)) return [];
3383     if (min === max) return [min];
3384     const span = max - min;
3385     const rawStep = span / Math.max(1, tickCount);
3386     const mag = Math.pow(10, Math.floor(Math.log10(rawStep)));
3387     const norm = rawStep / mag;
3388     const niceNorm = norm <= 1 ? 1 : (norm <= 2 ? 2 : (norm <= 5 ? 5 : 10));
3389     const step = niceNorm * mag;
3390     const start = Math.floor(min / step) * step;
3391     const end = Math.ceil(max / step) * step;
3392     const out = [];
3393     for (let v = start; v <= end + step / 2; v += step) out.push(v);
3394     return out;

```

```

3395 }
3396
3397 function renderHistogramSvg() {
3398     const svg = elements.histSvg;
3399     if (!svg) return;
3400
3401     const binsInput = elements.histBinsNumber ? parseInt(elements.histBinsNumber.value || '15', 10) : 15;
3402     const binCount = Number.isFinite(binsInput) ? binsInput : 15;
3403
3404     const xTicksInput = elements.histXTicksNumber ? parseInt(elements.histXTicksNumber.value || '6', 10) : 6;
3405     const xTickCount = Number.isFinite(xTicksInput) ? xTicksInput : 6;
3406
3407     const yTicksInput = elements.histYTicksNumber ? parseInt(elements.histYTicksNumber.value || '6', 10) : 6;
3408     const yTickCount = Number.isFinite(yTicksInput) ? yTicksInput : 6;
3409
3410     const { values, label } = getHistogramSeriesForType(currentHistType);
3411     const n = values.length;
3412
3413     if (elements.histN) elements.histN.textContent = n ? String(n) : '0';
3414
3415     if (!n) {
3416         svg.innerHTML = '';
3417         if (elements.histRange) elements.histRange.textContent = '-';
3418         if (elements.histEmptyNote) elements.histEmptyNote.classList.remove('hidden');
3419         if (elements.downloadHistSvgBtn) elements.downloadHistSvgBtn.disabled = true;
3420         return;
3421     }
3422
3423     if (elements.histEmptyNote) elements.histEmptyNote.classList.add('hidden');
3424     if (elements.downloadHistSvgBtn) elements.downloadHistSvgBtn.disabled = false;
3425
3426     const { bins, min, max } = computeHistogram(values, binCount);
3427     if (elements.histRange) elements.histRange.textContent = `${formatNumber(min)} to ${formatNumber(max)}`;
3428
3429     const W = 900, H = 520;
3430     const margin = { l: 76, r: 22, t: 52, b: 74 };
3431     const iw = W - margin.l - margin.r;
3432     const ih = H - margin.t - margin.b;
3433     const x0 = margin.l;
3434     const y0 = margin.t + ih;
3435
3436     const maxCount = Math.max(...bins, 1);
3437     const barW = iw / bins.length;
3438     const yScale = (c) => (margin.t + ih) - (c / maxCount) * ih;
3439
3440     const xTicks = niceTicks(min, max, Math.max(2, Math.min(12, xTickCount)));
3441     const yTicks = niceTicks(0, maxCount, Math.max(2, Math.min(12, yTickCount)));
3442
3443     const gridY = yTicks.map(v => {
3444         const y = yScale(v);
3445         return `

```

```

3474
3475     const xLabels = xTicks.map(v => {
3476         const t = (v - min) / ((max - min) || 1);
3477         const x = x0 + t * iw;
3478         return `
3479             <line x1="${x}" y1="${y0}" x2="${x}" y2="${y0 + 6}" stroke="#0f172a" stroke-width="1.25" />
3480             <text x="${x}" y="${y0 + 22}" text-anchor="middle" dominant-baseline="hanging" font-family="Inter, sans-serif"
font-size="12" fill="#0f172a">${escapeXml(formatNumber(v))}</text>
3481         `;
3482     }).join('\n');
3483
3484     const titleMap = {
3485         length: 'Length Histogram',
3486         angle: 'Angle Histogram',
3487         area: 'Area Histogram',
3488         particles: 'Particles Histogram',
3489         holes: 'Holes Histogram'
3490     };
3491     const title = titleMap[currentHistType] || 'Histogram';
3492
3493     svg.innerHTML = `
3494         <rect x="0" y="0" width="${W}" height="${H}" fill="#ffffff" />
3495         <text x="${margin.l}" y="28" font-family="Inter, sans-serif" font-size="18" font-weight="600"
fill="#0f172a">${escapeXml(title)}</text>
3496         <text x="${margin.l}" y="48" font-family="Inter, sans-serif" font-size="12" fill="#64748b">${escapeXml(label)} •
bins=${bins.length}</text>
3497         ${gridY}
3498         ${gridX}
3499         ${axis}
3500         ${bars}
3501         ${yLabels}
3502         ${xLabels}
3503         <text x="${margin.l + iw / 2}" y="${H - 26}" text-anchor="middle" font-family="Inter, sans-serif" font-size="13"
fill="#0f172a">${escapeXml(label)}</text>
3504         <text x="18" y="${margin.t + ih / 2}" text-anchor="middle" transform="rotate(-90 18 ${margin.t + ih / 2})"
font-family="Inter, sans-serif" font-size="13" fill="#0f172a">Count</text>
3505     `;
3506 }
3507
3508 function setHistType(type) {
3509     currentHistType = type;
3510     if (elements.histTypeBtns) {
3511         elements.histTypeBtns.forEach(btn => {
3512             const active = btn.dataset.histType === type;
3513             btn.classList.toggle('bg-blue-600', active);
3514             btn.classList.toggle('text-white', active);
3515             btn.classList.toggle('border-blue-600', active);
3516             btn.classList.toggle('bg-slate-50', !active);
3517             btn.classList.toggle('text-slate-800', !active);
3518             btn.classList.toggle('border-slate-200', !active);
3519         });
3520     }
3521     renderHistogramModal();
3522 }
3523
3524 function renderHistogramModal() {
3525     const titleMap = {
3526         length: 'Length – Histogram',
3527         angle: 'Angle – Histogram',
3528         area: 'Area – Histogram',
3529         particles: 'Particles – Histogram',
3530         holes: 'Holes – Histogram'
3531     };
3532     if (elements.histModalTitle) elements.histModalTitle.textContent = titleMap[currentHistType] || 'Histogram';
3533
3534     const unitNote = (currentHistType === 'angle')
3535         ? 'Degrees (calibration not required)'
3536         : (state.isCalibrated ? `Real units (${state.unit})` : `Pixels (set scale for real units)`);
3537     if (elements.histModalSubtitle) {
3538         elements.histModalSubtitle.textContent = `${unitNote}. Adjust bins to control resolution.`;
3539     }
3540
3541     const showMetric = (currentHistType === 'particles' || currentHistType === 'holes');
3542     if (elements.histMetricSelect) {
3543         elements.histMetricSelect.disabled = !showMetric;
3544         if (elements.histMetricSelect.parentElement) {
3545             elements.histMetricSelect.parentElement.classList.toggle('opacity-60', !showMetric);
3546         }
3547     }
3548
3549     renderHistogramSvg();

```

```

3550     }
3551
3552     function openHistogramModal(type) {
3553         if (!elements.histogramModal) return;
3554         elements.histogramModal.classList.remove('hidden');
3555         setHistType(type);
3556     }
3557
3558     function closeHistogramModal() {
3559         if (!elements.histogramModal) return;
3560         elements.histogramModal.classList.add('hidden');
3561     }
3562
3563     function downloadCurrentHistogramSvg() {
3564         const svg = elements.histSvg;
3565         if (!svg || !svg.innerHTML.trim()) return;
3566
3567         const serializer = new XMLSerializer();
3568         const svgText = serializer.serializeToString(svg);
3569         const blob = new Blob([svgText], { type: 'image/svg+xml;charset=utf-8' });
3570         const url = URL.createObjectURL(blob);
3571         const a = document.createElement('a');
3572         a.href = url;
3573         a.download = `micromeasure_histogram_${currentHistType}.svg`;
3574         a.click();
3575         URL.revokeObjectURL(url);
3576     }
3577
3578     function updateStatsButtonsVisibility() {
3579         const showLen = state.lengths.length >= 4;
3580         const showAngle = state.angles.length >= 4;
3581         const showArea = state.areas.length >= 4;
3582         const showParticles = state.particles.length >= 4;
3583         const showHoles = state.holes.length >= 4;
3584
3585         if (elements.lengthStatsBtn) elements.lengthStatsBtn.classList.toggle('hidden', !showLen);
3586         if (elements.lengthHistBtn) elements.lengthHistBtn.classList.toggle('hidden', !showLen);
3587         if (elements.angleStatsBtn) elements.angleStatsBtn.classList.toggle('hidden', !showAngle);
3588         if (elements.angleHistBtn) elements.angleHistBtn.classList.toggle('hidden', !showAngle);
3589         if (elements.areaStatsBtn) elements.areaStatsBtn.classList.toggle('hidden', !showArea);
3590         if (elements.areaHistBtn) elements.areaHistBtn.classList.toggle('hidden', !showArea);
3591         if (elements.particleStatsBtn) elements.particleStatsBtn.classList.toggle('hidden', !showParticles);
3592         if (elements.particleHistBtn) elements.particleHistBtn.classList.toggle('hidden', !showParticles);
3593         if (elements.holeStatsBtn) elements.holeStatsBtn.classList.toggle('hidden', !showHoles);
3594         if (elements.holeHistBtn) elements.holeHistBtn.classList.toggle('hidden', !showHoles);
3595
3596         // Per-feature actions (export + clear)
3597         updateFeatureActionButtons();
3598
3599         if (elements.histogramModal && !elements.histogramModal.classList.contains('hidden')) {
3600             renderHistogramModal();
3601         }
3602     }
3603
3604     function updateFeatureActionButtons() {
3605         const hasLen = state.lengths.length > 0;
3606         const hasAngle = state.angles.length > 0;
3607         const hasArea = state.areas.length > 0;
3608         const hasParticles = state.particles.length > 0;
3609         const hasHoles = state.holes.length > 0;
3610
3611         if (elements.lengthExportRow) elements.lengthExportRow.classList.toggle('hidden', !hasLen);
3612         if (elements.angleExportRow) elements.angleExportRow.classList.toggle('hidden', !hasAngle);
3613         if (elements.areaExportRow) elements.areaExportRow.classList.toggle('hidden', !hasArea);
3614         if (elements.particlesExportRow) elements.particlesExportRow.classList.toggle('hidden', !hasParticles);
3615         if (elements.holesExportRow) elements.holesExportRow.classList.toggle('hidden', !hasHoles);
3616
3617         if (elements.copyLengthTSVBtn) elements.copyLengthTSVBtn.disabled = !hasLen;
3618         if (elements.downloadLengthTSVBtn) elements.downloadLengthTSVBtn.disabled = !hasLen;
3619         if (elements.copyAngleTSVBtn) elements.copyAngleTSVBtn.disabled = !hasAngle;
3620         if (elements.downloadAngleTSVBtn) elements.downloadAngleTSVBtn.disabled = !hasAngle;
3621         if (elements.copyAreaTSVBtn) elements.copyAreaTSVBtn.disabled = !hasArea;
3622         if (elements.downloadAreaTSVBtn) elements.downloadAreaTSVBtn.disabled = !hasArea;
3623         if (elements.copyParticlesTSVBtn) elements.copyParticlesTSVBtn.disabled = !hasParticles;
3624         if (elements.downloadParticlesTSVBtn) elements.downloadParticlesTSVBtn.disabled = !hasParticles;
3625         if (elements.copyHolesTSVBtn) elements.copyHolesTSVBtn.disabled = !hasHoles;
3626         if (elements.downloadHolesTSVBtn) elements.downloadHolesTSVBtn.disabled = !hasHoles;
3627
3628         if (elements.clearParticles) elements.clearParticles.classList.toggle('hidden', !hasParticles);
3629         if (elements.clearHoles) elements.clearHoles.classList.toggle('hidden', !hasHoles);
3630     }

```



```

3631
3632 function drawHistogram(canvas, data, color) {
3633     const ctx = canvas.getContext('2d');
3634     const w = canvas.width = canvas.clientWidth * 2;
3635     const h = canvas.height = canvas.clientHeight * 2;
3636     ctx.scale(2, 2);
3637
3638     const width = w / 2;
3639     const height = h / 2;
3640
3641     ctx.clearRect(0, 0, width, height);
3642
3643     if (data.length === 0) {
3644         ctx.fillStyle = '#64748b';
3645         ctx.font = '11px Inter';
3646         ctx.textAlign = 'center';
3647         ctx.fillText('No data', width / 2, height / 2);
3648         return;
3649     }
3650
3651     // Create histogram bins
3652     const numBins = Math.min(20, Math.max(5, Math.ceil(Math.sqrt(data.length))));
3653     const min = Math.min(...data);
3654     const max = Math.max(...data);
3655     const range = max - min || 1;
3656     const binWidth = range / numBins;
3657
3658     const bins = new Array(numBins).fill(0);
3659     data.forEach(v => {
3660         const bin = Math.min(numBins - 1, Math.floor((v - min) / binWidth));
3661         bins[bin]++;
3662     });
3663
3664     const maxCount = Math.max(...bins);
3665     const barWidth = (width - 40) / numBins;
3666     const chartHeight = height - 20;
3667
3668     // Draw bars
3669     ctx.fillStyle = color;
3670     bins.forEach((count, i) => {
3671         const barHeight = (count / maxCount) * (chartHeight - 10);
3672         ctx.fillRect(30 + i * barWidth + 1, chartHeight - barHeight, barWidth - 2, barHeight);
3673     });
3674
3675     // Draw axis labels
3676     ctx.fillStyle = '#94a3b8';
3677     ctx.font = '9px JetBrains Mono';
3678     ctx.textAlign = 'center';
3679     ctx.fillText(min.toFixed(1), 30, height - 2);
3680     ctx.fillText(max.toFixed(1), width - 10, height - 2);
3681
3682     ctx.textAlign = 'right';
3683     ctx.fillText(maxCount.toString(), 25, 12);
3684     ctx.fillText('0', 25, chartHeight);
3685 }
3686
3687 // ===== ROI SELECTION =====
3688 function startROISelection(mode) {
3689     state.roiMode = mode;
3690     state.isDrawingROI = true;
3691     state.roi = null;
3692
3693     // Update UI
3694     if (mode === 'particles') {
3695         elements.selectParticleROI.textContent = '—■ Drawing ROI... (Click & Drag)';
3696         elements.selectParticleROI.classList.remove('btn-secondary');
3697         elements.selectParticleROI.classList.add('btn-primary');
3698     } else {
3699         elements.selectHoleROI.textContent = '—■ Drawing ROI... (Click & Drag)';
3700         elements.selectHoleROI.classList.remove('btn-secondary');
3701         elements.selectHoleROI.classList.add('btn-primary');
3702     }
3703
3704     elements.overlayCanvas.style.cursor = 'crosshair';
3705     drawOverlay();
3706 }
3707
3708 function finishROISelection() {
3709     state.isDrawingROI = false;
3710     state.roiStart = null;
3711

```

```

3712     if (!state.roi) {
3713         // User cancelled, reset buttons
3714         if (state.roiMode === 'particles') {
3715             elements.selectParticleROI.textContent = '■ Select ROI (Draw Rectangle)';
3716             elements.selectParticleROI.classList.add('btn-secondary');
3717             elements.selectParticleROI.classList.remove('btn-primary');
3718         } else {
3719             elements.selectHoleROI.textContent = '■ Select ROI (Draw Rectangle)';
3720             elements.selectHoleROI.classList.add('btn-secondary');
3721             elements.selectHoleROI.classList.remove('btn-primary');
3722         }
3723         state.roiMode = null;
3724         return;
3725     }
3726
3727     // Update UI to show ROI is active
3728     const roiText = `${Math.round(state.roi.width)}x${Math.round(state.roi.height)} px at (${Math.round(state.roi.x)},
3729     ${Math.round(state.roi.y)})`;
3730
3731     if (state.roiMode === 'particles') {
3732         elements.particleROIDimensions.textContent = roiText;
3733         elements.particleROIInfo.classList.remove('hidden');
3734         elements.selectParticleROI.classList.add('hidden');
3735     } else {
3736         elements.holeROIDimensions.textContent = roiText;
3737         elements.holeROIInfo.classList.remove('hidden');
3738         elements.selectHoleROI.classList.add('hidden');
3739     }
3740
3741     state.roiMode = null;
3742     elements.overlayCanvas.style.cursor = 'crosshair';
3743 }
3744
3745 function clearROI(mode) {
3746     state.roi = null;
3747
3748     if (mode === 'particles') {
3749         elements.particleROIInfo.classList.add('hidden');
3750         elements.selectParticleROI.classList.remove('hidden');
3751         elements.selectParticleROI.textContent = '■ Select ROI (Draw Rectangle)';
3752         elements.selectParticleROI.classList.add('btn-secondary');
3753         elements.selectParticleROI.classList.remove('btn-primary');
3754     } else {
3755         elements.holeROIInfo.classList.add('hidden');
3756         elements.selectHoleROI.classList.remove('hidden');
3757         elements.selectHoleROI.textContent = '■ Select ROI (Draw Rectangle)';
3758         elements.selectHoleROI.classList.add('btn-secondary');
3759         elements.selectHoleROI.classList.remove('btn-primary');
3760     }
3761
3762     drawOverlay();
3763 }
3764
3765 // ===== PROGRESS MODAL =====
3766 function showProcessingModal(title = 'Processing Image') {
3767     elements.processingTitle.textContent = title;
3768     elements.processingStep.textContent = 'Initializing...';
3769     elements.processingTime.textContent = 'Estimating time...';
3770     elements.progressBar.style.width = '0%';
3771     elements.processingModal.classList.remove('hidden');
3772 }
3773
3774 function updateProgress(step, progress, timeLeft = null) {
3775     elements.processingStep.textContent = step;
3776     elements.progressBar.style.width = progress + '%';
3777
3778     if (timeLeft !== null) {
3779         if (timeLeft < 1) {
3780             elements.processingTime.textContent = 'Less than 1 second remaining';
3781         } else if (timeLeft === 1) {
3782             elements.processingTime.textContent = '1 second remaining';
3783         } else {
3784             elements.processingTime.textContent = `${Math.ceil(timeLeft)} seconds remaining`;
3785         }
3786     }
3787 }
3788
3789 function hideProcessingModal() {
3790     elements.processingModal.classList.add('hidden');
3791 }

```

```

3792 // Async wrapper to allow UI updates during processing
3793 function sleep(ms) {
3794     return new Promise(resolve => setTimeout(resolve, ms));
3795 }
3796
3797 async function analyzeParticles() {
3798     const startTime = Date.now();
3799     showProcessingModal('Analyzing Particles');
3800
3801     await sleep(50); // Let modal render
3802
3803     const imageData = getImageData();
3804
3805     // Determine analysis region
3806     const roi = state.roi || { x: 0, y: 0, width: imageData.width, height: imageData.height };
3807
3808     const totalSteps = 4;
3809     let currentStep = 0;
3810
3811     // Step 1: Convert to grayscale
3812     updateProgress('Converting to grayscale...', (++currentStep / totalSteps) * 100, null);
3813     await sleep(10);
3814     let gray = toGrayscale(imageData);
3815
3816     // Step 2: Background subtraction
3817     if (elements.backgroundSubtraction.checked) {
3818         const elapsed = (Date.now() - startTime) / 1000;
3819         const estimated = elapsed * (totalSteps / currentStep) - elapsed;
3820         updateProgress('Subtracting background...', (++currentStep / totalSteps) * 100, estimated);
3821         await sleep(10);
3822         const radius = parseInt(elements.rollingBallRadius.value);
3823         gray = backgroundSubtractRollingBall(gray, imageData.width, imageData.height, radius);
3824     } else {
3825         currentStep++;
3826     }
3827
3828     // Step 3: Smoothing
3829     const smoothMethod = elements.smoothingMethod.value;
3830     if (smoothMethod !== 'none') {
3831         const elapsed = (Date.now() - startTime) / 1000;
3832         const estimated = elapsed * (totalSteps / currentStep) - elapsed;
3833         updateProgress('Applying smoothing filter...', (++currentStep / totalSteps) * 100, estimated);
3834         await sleep(10);
3835
3836         if (smoothMethod === 'gaussian3') {
3837             gray = blur3x3(gray, imageData.width, imageData.height);
3838         } else if (smoothMethod === 'median3') {
3839             gray = medianFilter(gray, imageData.width, imageData.height, 3);
3840         } else if (smoothMethod === 'median5') {
3841             gray = medianFilter(gray, imageData.width, imageData.height, 5);
3842         }
3843     } else {
3844         currentStep++;
3845     }
3846
3847     // Step 4: Thresholding and labeling
3848     const elapsed = (Date.now() - startTime) / 1000;
3849     const estimated = elapsed * (totalSteps / currentStep) - elapsed;
3850     updateProgress('Detecting particles...', (++currentStep / totalSteps) * 100, estimated);
3851     await sleep(10);
3852
3853     const threshold = parseInt(elements.particleThreshold.value);
3854     const minAreaInput = parseFloat(elements.minParticleArea.value) || 0.001;
3855     // If calibrated, convert from real units back to pixels
3856     const minArea = state.isCalibrated ? Math.round(minAreaInput / Math.pow(state.unitsPerPixel, 2)) :
3857     Math.round(minAreaInput);
3858     const brightIsObject = document.querySelector('input[name="particlePolarity"]:checked').value === 'bright';
3859     const useAdaptive = elements.adaptiveThreshold.checked;
3860     const blockSize = parseInt(elements.adaptiveBlockSize.value);
3861
3862     state.particles = thresholdAndLabelComponents(
3863         gray,
3864         imageData.width,
3865         imageData.height,
3866         threshold,
3867         brightIsObject,
3868         minArea,
3869         useAdaptive,
3870         blockSize,
3871         roi
3872     );

```

```

3872
3873 // Finalize
3874 updateProgress('Finalizing results...', 100, 0);
3875 await sleep(10);
3876
3877 updateParticleResults();
3878 drawOverlay();
3879 updateGuidance();
3880 updateSaveButtonState();
3881
3882 // Show completion briefly before hiding
3883 await sleep(300);
3884 hideProcessingModal();
3885 }
3886
3887 function updateParticleResults() {
3888   const stats = computeStats(state.particles, 'areaPx');
3889   const ecdStats = computeStats(state.particles, 'ecdPx');
3890
3891   elements.particleCount.textContent = stats.count;
3892   elements.particleMeanArea.textContent = formatNumber(stats.mean) + ' px²' +
3893     (state.isCalibrated ? ` (${formatNumber(stats.mean * Math.pow(state.unitsPerPixel, 2))} ${state.unit}²)` : '');
3894   elements.particleMeanECD.textContent = formatNumber(ecdStats.mean) + ' px' +
3895     (state.isCalibrated ? ` (${formatNumber(ecdStats.mean * state.unitsPerPixel)} ${state.unit})` : '');
3896   elements.particleStdDev.textContent = formatNumber(ecdStats.std) + ' px';
3897
3898   elements.particleResults.classList.remove('hidden');
3899   drawHistogram(elements.particleHistogram, state.particles.map(p => p.ecdPx), '#f97316');
3900   updateStatsButtonsVisibility();
3901 }
3902
3903 async function analyzeHoles() {
3904   const startTime = Date.now();
3905   showProcessingModal('Analyzing Holes');
3906
3907   await sleep(50); // Let modal render
3908
3909   const imageData = getImageData();
3910
3911   // Determine analysis region
3912   const roi = state.roi || { x: 0, y: 0, width: imageData.width, height: imageData.height };
3913
3914   const totalSteps = 4;
3915   let currentStep = 0;
3916
3917   // Step 1: Convert to grayscale
3918   updateProgress('Converting to grayscale...', (++currentStep / totalSteps) * 100, null);
3919   await sleep(10);
3920   let gray = toGrayscale(imageData);
3921
3922   // Step 2: Background subtraction
3923   if (document.getElementById('holeBackgroundSubtraction').checked) {
3924     const elapsed = (Date.now() - startTime) / 1000;
3925     const estimated = elapsed * (totalSteps / currentStep) - elapsed;
3926     updateProgress('Subtracting background...', (++currentStep / totalSteps) * 100, estimated);
3927     await sleep(10);
3928     const radius = parseInt(document.getElementById('holeRollingBallRadius').value);
3929     gray = backgroundSubtractRollingBall(gray, imageData.width, imageData.height, radius);
3930   } else {
3931     currentStep++;
3932   }
3933
3934   // Step 3: Smoothing
3935   const smoothMethod = document.getElementById('holeSmoothingMethod').value;
3936   if (smoothMethod !== 'none') {
3937     const elapsed = (Date.now() - startTime) / 1000;
3938     const estimated = elapsed * (totalSteps / currentStep) - elapsed;
3939     updateProgress('Applying smoothing filter...', (++currentStep / totalSteps) * 100, estimated);
3940     await sleep(10);
3941
3942     if (smoothMethod === 'gaussian3') {
3943       gray = blur3x3(gray, imageData.width, imageData.height);
3944     } else if (smoothMethod === 'median3') {
3945       gray = medianFilter(gray, imageData.width, imageData.height, 3);
3946     } else if (smoothMethod === 'median5') {
3947       gray = medianFilter(gray, imageData.width, imageData.height, 5);
3948     }
3949   } else {
3950     currentStep++;
3951   }
3952 }

```

```

3953 // Step 4: Thresholding and labeling
3954 const elapsed = (Date.now() - startTime) / 1000;
3955 const estimated = elapsed * (totalSteps / currentStep) - elapsed;
3956 updateProgress('Detecting holes...', (++currentStep / totalSteps) * 100, estimated);
3957 await sleep(10);
3958
3959 const threshold = parseInt(elements.holeThreshold.value);
3960 const minAreaInput = parseFloat(elements.minHoleArea.value) || 0.001;
3961 // If calibrated, convert from real units back to pixels
3962 const minArea = state.isCalibrated ? Math.round(minAreaInput / Math.pow(state.unitsPerPixel, 2)) :
Math.round(minAreaInput);
3963 const darkIsObject = document.querySelector('input[name="holePolarity"]:checked').value === 'dark';
3964 const useAdaptive = document.getElementById('adaptiveThresholdHole').checked;
3965 const blockSize = parseInt(document.getElementById('holeAdaptiveBlockSize').value);
3966 const useMorphOpening = document.getElementById('holeMorphologicalOpening').checked;
3967
3968 // Store original morphological setting and temporarily set it for this analysis
3969 const originalMorphSetting = elements.morphologicalOpening ? elements.morphologicalOpening.checked : false;
3970 if (elements.morphologicalOpening) {
3971     elements.morphologicalOpening.checked = useMorphOpening;
3972 }
3973
3974 state.holes = thresholdAndLabelComponents(
3975     gray,
3976     imageData.width,
3977     imageData.height,
3978     threshold,
3979     !darkIsObject,
3980     minArea,
3981     useAdaptive,
3982     blockSize,
3983     roi
3984 );
3985
3986 // Restore original morphological setting
3987 if (elements.morphologicalOpening) {
3988     elements.morphologicalOpening.checked = originalMorphSetting;
3989 }
3990
3991 // Finalize
3992 updateProgress('Finalizing results...', 100, 0);
3993 await sleep(10);
3994
3995 updateHoleResults();
3996 drawOverlay();
3997 updateGuidance();
3998 updateSaveButtonState();
3999
4000 // Show completion briefly before hiding
4001 await sleep(300);
4002 hideProcessingModal();
4003 }
4004
4005 function updateHoleResults() {
4006     const stats = computeStats(state.holes, 'areaPx');
4007     const ecdStats = computeStats(state.holes, 'ecdPx');
4008
4009     elements.holeCount.textContent = stats.count;
4010     elements.holeMeanArea.textContent = formatNumber(stats.mean) + ' px²' +
4011         (state.isCalibrated ? ` (${formatNumber(stats.mean * Math.pow(state.unitsPerPixel, 2))} ${state.unit}²)` : '');
4012     elements.holeMeanECD.textContent = formatNumber(ecdStats.mean) + ' px' +
4013         (state.isCalibrated ? ` (${formatNumber(ecdStats.mean * state.unitsPerPixel)} ${state.unit})` : '');
4014     elements.holeStdDev.textContent = formatNumber(ecdStats.std) + ' px';
4015
4016     elements.holeResults.classList.remove('hidden');
4017     drawHistogram(elements.holeHistogram, state.holes.map(h => h.ecdPx), '#a855f7');
4018     updateStatsButtonsVisibility();
4019 }
4020
4021 function updateMeasurementDisplays() {
4022     // Recalculate real values with new scale
4023     state.lengths.forEach(len => {
4024         len.realLength = state.isCalibrated ? len.pixelLength * state.unitsPerPixel : null;
4025     });
4026     state.areas.forEach(area => {
4027         area.realArea = state.isCalibrated ? area.pixelArea * Math.pow(state.unitsPerPixel, 2) : null;
4028     });
4029     state.particles.forEach(p => {
4030         p.areaReal = state.isCalibrated ? p.areaPx * Math.pow(state.unitsPerPixel, 2) : null;
4031         p.ecdReal = state.isCalibrated ? p.ecdPx * state.unitsPerPixel : null;
4032     });

```

```

4033     state.holes.forEach(h => {
4034         h.areaReal = state.isCalibrated ? h.areaPx * Math.pow(state.unitsPerPixel, 2) : null;
4035         h.ecdReal = state.isCalibrated ? h.ecdPx * state.unitsPerPixel : null;
4036     });
4037
4038     updateLengthDisplay();
4039     updateAngleDisplay();
4040     updateAreaDisplay();
4041     if (state.particles.length) updateParticleResults();
4042     if (state.holes.length) updateHoleResults();
4043     updateStatsButtonsVisibility();
4044 }
4045
4046 // ===== PER-FEATURE EXPORT =====
4047 function buildFeatureTSV(type) {
4048     let tsv = 'Type\tID\tValue_px\tValue_real\tUnit\n';
4049
4050     if (type === 'length') {
4051         state.lengths.forEach((len, i) => {
4052             tsv += `Length\tL${i + 1}\t${len.pixelLength.toFixed(4)}\t${state.isCalibrated ? len.realLength.toFixed(4) :
4053             ''}\t${state.isCalibrated ? state.unit : 'px'}\n`;
4054         });
4055         return tsv;
4056     }
4057
4058     if (type === 'angle') {
4059         state.angles.forEach((ang, i) => {
4060             const v = Number.isFinite(ang.angleDeg) ? ang.angleDeg : 0;
4061             tsv += `Angle\tAng${i + 1}\t${v.toFixed(4)}\t${v.toFixed(4)}\tdeg\n`;
4062         });
4063         return tsv;
4064     }
4065
4066     if (type === 'area') {
4067         state.areas.forEach((area, i) => {
4068             tsv += `Area\tA${i + 1}\t${area.pixelArea.toFixed(4)}\t${state.isCalibrated ? area.realArea.toFixed(4) :
4069             ''}\t${state.isCalibrated ? state.unit + '^2' : 'px^2'}\n`;
4070         });
4071         return tsv;
4072     }
4073
4074     if (type === 'particles') {
4075         state.particles.forEach((p) => {
4076             tsv += `Particle_Area\tP${p.id}\t${p.areaPx}\t${state.isCalibrated ? p.areaReal.toFixed(4) : ''}\t${state.isCalibra
4077             ? state.unit + '^2' : 'px^2'}\n`;
4078             tsv += `Particle_ECD\tP${p.id}\t${p.ecdPx.toFixed(4)}\t${state.isCalibrated ? p.ecdReal.toFixed(4) :
4079             ''}\t${state.isCalibrated ? state.unit : 'px'}\n`;
4080         });
4081         return tsv;
4082     }
4083
4084     if (type === 'holes') {
4085         state.holes.forEach((h) => {
4086             tsv += `Hole_Area\tH${h.id}\t${h.areaPx}\t${state.isCalibrated ? h.areaReal.toFixed(4) : ''}\t${state.isCalibrated
4087             ? state.unit + '^2' : 'px^2'}\n`;
4088             tsv += `Hole_ECD\tH${h.id}\t${h.ecdPx.toFixed(4)}\t${state.isCalibrated ? h.ecdReal.toFixed(4) :
4089             ''}\t${state.isCalibrated ? state.unit : 'px'}\n`;
4090         });
4091         return tsv;
4092     }
4093
4094     function escapeCsvValue(value) {
4095         const text = String(value ?? '');
4096         if (!/[",\n\r]/.test(text)) return text;
4097         return `"${text.replace(/"/g, '"')}"`;
4098     }
4099
4100     function buildFeatureCSV(type) {
4101         const tsv = buildFeatureTSV(type);
4102         const rows = tsv
4103             .trim()
4104             .split('\n')
4105             .map((line) => line.split('\t').map(escapeCsvValue).join(','));
4106         return rows.length ? `${rows.join('\n')}\n` : '';
4107     }
4108
4109     async function copyFeatureTSV(type) {
4110         const tsv = buildFeatureTSV(type);

```

```

4108     const hasRows = tsv.trim().split('\n').length > 1;
4109     if (!hasRows) return;
4110
4111     try {
4112         if (navigator.clipboard && navigator.clipboard.writeText) {
4113             await navigator.clipboard.writeText(tsv);
4114             return;
4115         }
4116     } catch (e) {
4117         // Fall back below
4118     }
4119
4120     fallbackCopyText(tsv);
4121 }
4122
4123 function downloadFeatureCSV(type) {
4124     const csv = buildFeatureCSV(type);
4125     const hasRows = csv.trim().split('\n').length > 1;
4126     if (!hasRows) return;
4127
4128     const blob = new Blob([csv], { type: 'text/csv;charset=utf-8' });
4129     const url = URL.createObjectURL(blob);
4130     const a = document.createElement('a');
4131     a.href = url;
4132     a.download = `micromasure_${type}.csv`;
4133     a.click();
4134     URL.revokeObjectURL(url);
4135 }
4136
4137 function saveScreenshot() {
4138     if (!state.image) return;
4139
4140     // Create a temporary canvas to combine main image and overlay
4141     const exportCanvas = document.createElement('canvas');
4142     exportCanvas.width = state.image.width;
4143     exportCanvas.height = state.image.height;
4144     const exportCtx = exportCanvas.getContext('2d');
4145
4146     // Draw the main image
4147     exportCtx.drawImage(state.image, 0, 0);
4148
4149     // Draw the overlay on top
4150     exportCtx.drawImage(elements.overlayCanvas, 0, 0);
4151
4152     // Convert to blob and download
4153     exportCanvas.toBlob((blob) => {
4154         const url = URL.createObjectURL(blob);
4155         const a = document.createElement('a');
4156         a.href = url;
4157         a.download = 'micromasure_screenshot.png';
4158         a.click();
4159         URL.revokeObjectURL(url);
4160     }, 'image/png');
4161 }
4162
4163 function clearAllParticles() {
4164     state.particles = [];
4165     if (elements.particleResults) elements.particleResults.classList.add('hidden');
4166     updateStatsButtonsVisibility();
4167     drawOverlay();
4168     updateGuidance();
4169     updateSaveButtonState();
4170 }
4171
4172 function clearAllHoles() {
4173     state.holes = [];
4174     if (elements.holeResults) elements.holeResults.classList.add('hidden');
4175     updateStatsButtonsVisibility();
4176     drawOverlay();
4177     updateGuidance();
4178     updateSaveButtonState();
4179 }
4180
4181 function resetProject() {
4182     state.isCalibrated = false;
4183     state.calibrationLine = null;
4184     state.lengths = [];
4185     state.lengthPoints = [];
4186     state.anglePoints = [];
4187     state.angles = [];
4188     state.editingAngleIndex = null;

```

```

4189     state.draggedAnglePointIndex = null;
4190     state.angleEditBackup = null;
4191     state.areaMode = 'polygon';
4192     state.areas = [];
4193     state.areaPoints = [];
4194     state.editingAreaIndex = null;
4195     state.draggedPointIndex = null;
4196     state.isDrawingCircle = false;
4197     state.circleDraft = null;
4198     state.particles = [];
4199     state.holes = [];
4200     state.roi = null;
4201     state.isDrawingROI = false;
4202     state.roiStart = null;
4203     state.roiMode = null;
4204
4205     elements.scaleInfo.classList.add('hidden');
4206     elements.particleResults.classList.add('hidden');
4207     elements.holeResults.classList.add('hidden');
4208
4209     // Reset ROI UI
4210     elements.particleROIInfo.classList.add('hidden');
4211     elements.selectParticleROI.classList.remove('hidden');
4212     elements.selectParticleROI.textContent = '■ Select ROI (Draw Rectangle)';
4213     elements.selectParticleROI.classList.add('btn-secondary');
4214     elements.selectParticleROI.classList.remove('btn-primary');
4215
4216     elements.holeROIInfo.classList.add('hidden');
4217     elements.selectHoleROI.classList.remove('hidden');
4218     elements.selectHoleROI.textContent = '■ Select ROI (Draw Rectangle)';
4219     elements.selectHoleROI.classList.add('btn-secondary');
4220     elements.selectHoleROI.classList.remove('btn-primary');
4221
4222     if (elements.editAngleModeControls) elements.editAngleModeControls.classList.add('hidden');
4223
4224     // Reset Area mode UI
4225     if (elements.closePolygonBtn) elements.closePolygonBtn.classList.add('hidden');
4226     if (elements.editModeControls) elements.editModeControls.classList.add('hidden');
4227     if (elements.areaInstructions) {
4228         elements.areaInstructions.classList.remove('hidden');
4229         elements.areaInstructions.textContent = 'Polygon: click points to create polygon.';
4230     }
4231     if (elements.areaModePolygonBtn) {
4232         elements.areaModePolygonBtn.classList.add('btn-primary');
4233         elements.areaModePolygonBtn.classList.remove('btn-secondary');
4234     }
4235     if (elements.areaModeCircleBtn) {
4236         elements.areaModeCircleBtn.classList.add('btn-secondary');
4237         elements.areaModeCircleBtn.classList.remove('btn-primary');
4238     }
4239
4240     updateLengthDisplay();
4241     updateAngleDisplay();
4242     updateAreaDisplay();
4243     drawOverlay();
4244     updateGuidance();
4245     updateSaveButtonState();
4246 }
4247
4248 function resetToStart() {
4249     // Clear all measurements/results/state using existing logic
4250     resetProject();
4251
4252     // Return to the initial "no image" state
4253     state.image = null;
4254     state.imageData = null;
4255     state.zoom = 100;
4256     state.panX = 0;
4257     state.panY = 0;
4258     state.isPanning = false;
4259
4260     // Keep onboarding overlay disabled
4261     state.quickStartDismissed = true;
4262
4263     // UI reset
4264     elements.zoomSlider.value = 100;
4265     elements.zoomValue.textContent = '100%';
4266     elements.imageDimensions.classList.add('hidden');
4267     elements.fitToView.classList.add('hidden');
4268     const panControls = document.getElementById('panControls');
4269     if (panControls) panControls.classList.add('hidden');

```



```

4270     elements.noImagePlaceholder.classList.remove('hidden');
4271     elements.mainCanvas.classList.add('hidden');
4272     elements.overlayCanvas.classList.add('hidden');
4273
4274     elements.setScaleBtn.disabled = true;
4275     elements.analyzeParticles.disabled = true;
4276     elements.analyzeHoles.disabled = true;
4277     elements.selectParticleROI.disabled = true;
4278     elements.selectHoleROI.disabled = true;
4279
4280     // Hide calibration input panel if open
4281     elements.calibrationInputs.classList.add('hidden');
4282     state.isSettingScale = false;
4283     state.scalePoints = [];
4284
4285     // Clear canvases
4286     mainCtx.clearRect(0, 0, elements.mainCanvas.width, elements.mainCanvas.height);
4287     overlayCtx.clearRect(0, 0, elements.overlayCanvas.width, elements.overlayCanvas.height);
4288
4289     updateGuidance();
4290     updateSaveButtonState();
4291 }
4292
4293 function openResetConfirm() {
4294     if (!elements.resetConfirmModal) return;
4295     elements.resetConfirmModal.classList.remove('hidden');
4296 }
4297
4298 function closeResetConfirm() {
4299     if (!elements.resetConfirmModal) return;
4300     elements.resetConfirmModal.classList.add('hidden');
4301 }
4302
4303 function openSaveImageConfirm() {
4304     if (!elements.saveImageConfirmModal || elements.saveScreenshot.disabled) return;
4305     elements.saveImageConfirmModal.classList.remove('hidden');
4306 }
4307
4308 function closeSaveImageConfirm() {
4309     if (!elements.saveImageConfirmModal) return;
4310     elements.saveImageConfirmModal.classList.add('hidden');
4311 }
4312
4313 function openHomeConfirm() {
4314     if (!elements.homeConfirmModal) return;
4315     elements.homeConfirmModal.classList.remove('hidden');
4316 }
4317
4318 function closeHomeConfirm() {
4319     if (!elements.homeConfirmModal) return;
4320     elements.homeConfirmModal.classList.add('hidden');
4321 }
4322
4323 function openDemoImageConfirm() {
4324     if (!elements.demoImageConfirmModal) return;
4325     elements.demoImageConfirmModal.classList.remove('hidden');
4326 }
4327
4328 function closeDemoImageConfirm() {
4329     if (!elements.demoImageConfirmModal) return;
4330     elements.demoImageConfirmModal.classList.add('hidden');
4331 }
4332
4333 function goToFacilityHomePage() {
4334     window.location.href = 'https://www.inanolab.com/#extras';
4335 }
4336
4337 // ===== EVENT LISTENERS =====
4338 elements.imageInput.addEventListener('change', (e) => {
4339     if (e.target.files[0]) loadImage(e.target.files[0]);
4340     e.target.value = '';
4341 });
4342
4343 elements.zoomSlider.addEventListener('input', (e) => {
4344     state.zoom = parseInt(e.target.value);
4345     elements.zoomValue.textContent = state.zoom + '%';
4346     if (state.image) updateCanvasTransform();
4347 });
4348
4349 elements.fitToView.addEventListener('click', fitToView);
4350

```

```

4351
4352 // Pan buttons
4353 elements.panLeft.addEventListener('click', () => {
4354     state.panX += 50;
4355     updateCanvasTransform();
4356 });
4357
4358 elements.panRight.addEventListener('click', () => {
4359     state.panX -= 50;
4360     updateCanvasTransform();
4361 });
4362
4363 elements.panUp.addEventListener('click', () => {
4364     state.panY += 50;
4365     updateCanvasTransform();
4366 });
4367
4368 elements.panDown.addEventListener('click', () => {
4369     state.panY -= 50;
4370     updateCanvasTransform();
4371 });
4372
4373 elements.resetPan.addEventListener('click', () => {
4374     state.panX = 0;
4375     state.panY = 0;
4376     updateCanvasTransform();
4377 });
4378
4379 elements.setScaleBtn.addEventListener('click', setScale);
4380 elements.saveScale.addEventListener('click', saveScale);
4381 elements.cancelScale.addEventListener('click', cancelScale);
4382 elements.resetScaleBtn.addEventListener('click', resetScale);
4383 elements.clearLengths.addEventListener('click', clearAllLengths);
4384 if (elements.lengthStatsBtn) elements.lengthStatsBtn.addEventListener('click', () => openStatsModal('length'));
4385 if (elements.lengthHistBtn) elements.lengthHistBtn.addEventListener('click', () => openHistogramModal('length'));
4386 if (elements.clearAngles) elements.clearAngles.addEventListener('click', clearAllAngles);
4387 if (elements.angleStatsBtn) elements.angleStatsBtn.addEventListener('click', () => openStatsModal('angle'));
4388 if (elements.angleHistBtn) elements.angleHistBtn.addEventListener('click', () => openHistogramModal('angle'));
4389 if (elements.saveAngleEdit) elements.saveAngleEdit.addEventListener('click', saveAngleEdit);
4390 if (elements.cancelAngleEdit) elements.cancelAngleEdit.addEventListener('click', cancelAngleEdit);
4391 elements.saveLengthEdit.addEventListener('click', saveLengthEdit);
4392 elements.cancelLengthEdit.addEventListener('click', cancelLengthEdit);
4393 elements.clearAreas.addEventListener('click', clearAllAreas);
4394 if (elements.areaStatsBtn) elements.areaStatsBtn.addEventListener('click', () => openStatsModal('area'));
4395 if (elements.areaHistBtn) elements.areaHistBtn.addEventListener('click', () => openHistogramModal('area'));
4396 if (elements.areaModePolygonBtn) elements.areaModePolygonBtn.addEventListener('click', () => setAreaMode('polygon'));
4397 if (elements.areaModeCircleBtn) elements.areaModeCircleBtn.addEventListener('click', () => setAreaMode('circle'));
4398 elements.closePolygonBtn.addEventListener('click', () => {
4399     if (state.areaMode === 'polygon' && state.areaPoints.length >= 3) {
4400         addAreaMeasurement(state.areaPoints);
4401         state.areaPoints = [];
4402         elements.closePolygonBtn.classList.add('hidden');
4403         drawOverlay();
4404     }
4405 });
4406 elements.saveAreaEdit.addEventListener('click', saveAreaEdit);
4407 elements.cancelAreaEdit.addEventListener('click', cancelAreaEdit);
4408 elements.analyzeParticles.addEventListener('click', analyzeParticles);
4409 elements.analyzeHoles.addEventListener('click', analyzeHoles);
4410 if (elements.particleStatsBtn) elements.particleStatsBtn.addEventListener('click', () => openStatsModal('particles'));
4411 if (elements.holeStatsBtn) elements.holeStatsBtn.addEventListener('click', () => openStatsModal('holes'));
4412 if (elements.particleHistBtn) elements.particleHistBtn.addEventListener('click', () => openHistogramModal('particles'));
4413 if (elements.holeHistBtn) elements.holeHistBtn.addEventListener('click', () => openHistogramModal('holes'));
4414 elements.selectParticleROI.addEventListener('click', () => startROISelection('particles'));
4415 elements.clearParticleROI.addEventListener('click', () => clearROI('particles'));
4416 elements.selectHoleROI.addEventListener('click', () => startROISelection('holes'));
4417 elements.clearHoleROI.addEventListener('click', () => clearROI('holes'));
4418 elements.saveScreenshot.addEventListener('click', openSaveImageConfirm);
4419 if (elements.homeBtn) elements.homeBtn.addEventListener('click', openHomeConfirm);
4420 if (elements.loadDemoBtn) elements.loadDemoBtn.addEventListener('click', openDemoImageConfirm);
4421
4422 // Per-feature export buttons
4423 if (elements.copyLengthTSVBtn) elements.copyLengthTSVBtn.addEventListener('click', () => copyFeatureTSV('length'));
4424 if (elements.downloadLengthTSVBtn) elements.downloadLengthTSVBtn.addEventListener('click', () =>
4425     downloadFeatureCSV('length'));
4426 if (elements.copyAngleTSVBtn) elements.copyAngleTSVBtn.addEventListener('click', () => copyFeatureTSV('angle'));
4427 if (elements.downloadAngleTSVBtn) elements.downloadAngleTSVBtn.addEventListener('click', () => downloadFeatureCSV('angle'));
4428 if (elements.copyAreaTSVBtn) elements.copyAreaTSVBtn.addEventListener('click', () => copyFeatureTSV('area'));
4429 if (elements.downloadAreaTSVBtn) elements.downloadAreaTSVBtn.addEventListener('click', () => downloadFeatureCSV('area'));
4430 if (elements.copyParticlesTSVBtn) elements.copyParticlesTSVBtn.addEventListener('click', () => copyFeatureTSV('particles'));
4431 if (elements.downloadParticlesTSVBtn) elements.downloadParticlesTSVBtn.addEventListener('click', () =>

```

```

        downloadFeatureCSV('particles'));
4431 if (elements.copyHolesTSVBtn) elements.copyHolesTSVBtn.addEventListener('click', () => copyFeatureTSV('holes'));
4432 if (elements.downloadHolesTSVBtn) elements.downloadHolesTSVBtn.addEventListener('click', () => downloadFeatureCSV('holes'));
4433
4434 if (elements.clearParticles) elements.clearParticles.addEventListener('click', clearAllParticles);
4435 if (elements.clearHoles) elements.clearHoles.addEventListener('click', clearAllHoles);
4436
4437 // Header reset with confirmation
4438 if (elements.headerResetBtn) elements.headerResetBtn.addEventListener('click', openResetConfirm);
4439 if (elements.confirmResetNo) elements.confirmResetNo.addEventListener('click', closeResetConfirm);
4440 if (elements.confirmResetYes) elements.confirmResetYes.addEventListener('click', () => {
4441     closeResetConfirm();
4442     resetToStart();
4443 });
4444 if (elements.resetConfirmModal) {
4445     elements.resetConfirmModal.addEventListener('click', (e) => {
4446         if (e.target === elements.resetConfirmModal) closeResetConfirm();
4447     });
4448 }
4449 if (elements.confirmSaveImageNo) elements.confirmSaveImageNo.addEventListener('click', closeSaveImageConfirm);
4450 if (elements.confirmSaveImageYes) elements.confirmSaveImageYes.addEventListener('click', () => {
4451     closeSaveImageConfirm();
4452     saveScreenshot();
4453 });
4454 if (elements.saveImageConfirmModal) {
4455     elements.saveImageConfirmModal.addEventListener('click', (e) => {
4456         if (e.target === elements.saveImageConfirmModal) closeSaveImageConfirm();
4457     });
4458 }
4459 if (elements.confirmHomeNo) elements.confirmHomeNo.addEventListener('click', closeHomeConfirm);
4460 if (elements.confirmHomeYes) elements.confirmHomeYes.addEventListener('click', () => {
4461     closeHomeConfirm();
4462     goToFacilityHomePage();
4463 });
4464 if (elements.homeConfirmModal) {
4465     elements.homeConfirmModal.addEventListener('click', (e) => {
4466         if (e.target === elements.homeConfirmModal) closeHomeConfirm();
4467     });
4468 }
4469 if (elements.confirmDemoImageNo) elements.confirmDemoImageNo.addEventListener('click', closeDemoImageConfirm);
4470 if (elements.confirmDemoImageYes) elements.confirmDemoImageYes.addEventListener('click', () => {
4471     closeDemoImageConfirm();
4472     loadDemoImage();
4473 });
4474 if (elements.demoImageConfirmModal) {
4475     elements.demoImageConfirmModal.addEventListener('click', (e) => {
4476         if (e.target === elements.demoImageConfirmModal) closeDemoImageConfirm();
4477     });
4478 }
4479
4480 // Documentation modal
4481 document.getElementById('helpBtn').addEventListener('click', () => {
4482     document.getElementById('docModal').classList.remove('hidden');
4483     try { document.getElementById('closeDocBtn')?.focus(); } catch {}
4484 });
4485
4486 document.getElementById('closeDocBtn').addEventListener('click', () => {
4487     document.getElementById('docModal').classList.add('hidden');
4488 });
4489
4490 // Close documentation modal when clicking outside
4491 document.getElementById('docModal').addEventListener('click', (e) => {
4492     if (e.target.id === 'docModal') {
4493         document.getElementById('docModal').classList.add('hidden');
4494     }
4495 });
4496
4497 // Close documentation modal with Escape (match Roughness Visualizer)
4498 document.addEventListener('keydown', (e) => {
4499     if (e.key !== 'Escape') return;
4500     const docModal = document.getElementById('docModal');
4501     if (docModal && !docModal.classList.contains('hidden')) docModal.classList.add('hidden');
4502 });
4503
4504 // Quick Start overlay
4505 if (elements.quickStartOk) {
4506     elements.quickStartOk.addEventListener('click', () => {
4507         state.quickStartDismissed = true;
4508         updateGuidance();
4509     });
4510 }

```

```

4511 // Particle preprocessing UI controls
4512 elements.backgroundSubtraction.addEventListener('change', (e) => {
4513     elements.backgroundOptions.classList.toggle('hidden', !e.target.checked);
4514 });
4515
4516 elements.adaptiveThreshold.addEventListener('change', (e) => {
4517     elements.manualThresholdControls.classList.toggle('hidden', e.target.checked);
4518     elements.adaptiveThresholdControls.classList.toggle('hidden', !e.target.checked);
4519 });
4520
4521 // Hole preprocessing UI controls
4522 document.getElementById('holeBackgroundSubtraction').addEventListener('change', (e) => {
4523     document.getElementById('holeBackgroundOptions').classList.toggle('hidden', !e.target.checked);
4524 });
4525
4526 document.getElementById('adaptiveThresholdHole').addEventListener('change', (e) => {
4527     document.getElementById('manualThresholdHoleControls').classList.toggle('hidden', e.target.checked);
4528     document.getElementById('adaptiveThresholdHoleControls').classList.toggle('hidden', !e.target.checked);
4529 });
4530
4531 document.getElementById('autoThresholdHoleBtn').addEventListener('click', () => {
4532     if (!state.image) return;
4533
4534     const imageData = getImageData();
4535     let gray = toGrayscale(imageData);
4536
4537     // Apply same preprocessing as hole analysis
4538     if (document.getElementById('holeBackgroundSubtraction').checked) {
4539         const radius = parseInt(document.getElementById('holeRollingBallRadius').value);
4540         gray = backgroundSubtractRollingBall(gray, imageData.width, imageData.height, radius);
4541     }
4542
4543     const smoothMethod = document.getElementById('holeSmoothingMethod').value;
4544     if (smoothMethod === 'gaussian3') {
4545         gray = blur3x3(gray, imageData.width, imageData.height);
4546     } else if (smoothMethod === 'median3') {
4547         gray = medianFilter(gray, imageData.width, imageData.height, 3);
4548     } else if (smoothMethod === 'median5') {
4549         gray = medianFilter(gray, imageData.width, imageData.height, 5);
4550     }
4551
4552     const threshold = otsuThreshold(gray);
4553     elements.holeThreshold.value = threshold;
4554     elements.holeThresholdValue.textContent = threshold;
4555 });
4556
4557 // Descriptive stats modal controls
4558 if (elements.closeStatsModalBtn) elements.closeStatsModalBtn.addEventListener('click', closeStatsModal);
4559 if (elements.closeStatsModalBtnBottom) elements.closeStatsModalBtnBottom.addEventListener('click', closeStatsModal);
4560 if (elements.copyStatsTSVBtn) elements.copyStatsTSVBtn.addEventListener('click', copyStatsTSV);
4561 if (elements.statsTypeBtns) {
4562     elements.statsTypeBtns.forEach(btn => {
4563         btn.addEventListener('click', () => setStatsType(btn.dataset.statsType));
4564     });
4565 }
4566
4567 if (elements.descriptiveStatsModal) {
4568     elements.descriptiveStatsModal.addEventListener('click', (e) => {
4569         if (e.target === elements.descriptiveStatsModal) closeStatsModal();
4570     });
4571 }
4572
4573 // Histogram modal controls
4574 if (elements.closeHistModalBtn) elements.closeHistModalBtn.addEventListener('click', closeHistogramModal);
4575 if (elements.closeHistModalBtnBottom) elements.closeHistModalBtnBottom.addEventListener('click', closeHistogramModal);
4576 if (elements.downloadHistSvgBtn) elements.downloadHistSvgBtn.addEventListener('click', downloadCurrentHistogramSvg);
4577 if (elements.histBinsNumber) {
4578     elements.histBinsNumber.addEventListener('input', () => {
4579         const v = Math.max(3, Math.min(80, parseInt(elements.histBinsNumber.value || '15', 10)));
4580         elements.histBinsNumber.value = String(v);
4581         renderHistogramModal();
4582     });
4583 }
4584 if (elements.histXTicksNumber) {
4585     elements.histXTicksNumber.addEventListener('input', () => {
4586         const v = Math.max(2, Math.min(12, parseInt(elements.histXTicksNumber.value || '6', 10)));
4587         elements.histXTicksNumber.value = String(v);
4588         renderHistogramModal();
4589     });
4590 }
4591 if (elements.histYTicksNumber) {

```

```

4592     elements.histYTicksNumber.addEventListener('input', () => {
4593         const v = Math.max(2, Math.min(12, parseInt(elements.histYTicksNumber.value || '6', 10)));
4594         elements.histYTicksNumber.value = String(v);
4595         renderHistogramModal();
4596     });
4597 }
4598 if (elements.histMetricSelect) elements.histMetricSelect.addEventListener('change', renderHistogramModal);
4599 if (elements.histTypeBtns) {
4600     elements.histTypeBtns.forEach(btn => {
4601         btn.addEventListener('click', () => setHistType(btn.dataset.histType));
4602     });
4603 }
4604 if (elements.histogramModal) {
4605     elements.histogramModal.addEventListener('click', (e) => {
4606         if (e.target === elements.histogramModal) closeHistogramModal();
4607     });
4608 }
4609
4610 elements.autoThresholdBtn.addEventListener('click', () => {
4611     if (!state.image) return;
4612
4613     const imageData = getImageData();
4614     let gray = toGrayscale(imageData);
4615
4616     // Apply same preprocessing as particle analysis
4617     if (elements.backgroundSubtraction.checked) {
4618         const radius = parseInt(elements.rollingBallRadius.value);
4619         gray = backgroundSubtractRollingBall(gray, imageData.width, imageData.height, radius);
4620     }
4621
4622     const smoothMethod = elements.smoothingMethod.value;
4623     if (smoothMethod === 'gaussian3') {
4624         gray = blur3x3(gray, imageData.width, imageData.height);
4625     } else if (smoothMethod === 'median3') {
4626         gray = medianFilter(gray, imageData.width, imageData.height, 3);
4627     } else if (smoothMethod === 'median5') {
4628         gray = medianFilter(gray, imageData.width, imageData.height, 5);
4629     }
4630
4631     const threshold = otsuThreshold(gray);
4632     elements.particleThreshold.value = threshold;
4633     elements.thresholdValue.textContent = threshold;
4634 });
4635
4636 elements.particleThreshold.addEventListener('input', (e) => {
4637     elements.thresholdValue.textContent = e.target.value;
4638 });
4639
4640 elements.holeThreshold.addEventListener('input', (e) => {
4641     elements.holeThresholdValue.textContent = e.target.value;
4642 });
4643
4644 // Tab switching
4645 elements.tabBtns.forEach(btn => {
4646     btn.addEventListener('click', () => {
4647         elements.tabBtns.forEach(b => b.classList.remove('active'));
4648         btn.classList.add('active');
4649
4650         const tab = btn.dataset.tab;
4651         state.currentTool = tab;
4652
4653         // Clear in-progress area when switching away from area tool
4654         if (tab !== 'area') {
4655             let needsRedraw = false;
4656
4657             if (state.areaPoints.length > 0) {
4658                 state.areaPoints = [];
4659                 elements.closePolygonBtn.classList.add('hidden');
4660                 needsRedraw = true;
4661             }
4662
4663             if (state.isDrawingCircle || state.circleDraft) {
4664                 state.isDrawingCircle = false;
4665                 state.circleDraft = null;
4666                 _circlePointerId = null;
4667                 needsRedraw = true;
4668             }
4669
4670             if (needsRedraw) drawOverlay();
4671         }
4672     });

```

```

4673 // Clear in-progress angle when switching away from angle tool
4674 if (tab !== 'angle' && state.anglePoints.length > 0) {
4675     state.anglePoints = [];
4676     drawOverlay();
4677 }
4678
4679 document.querySelectorAll('.tab-content').forEach(content => content.classList.add('hidden'));
4680 document.getElementById(tab + 'Tab').classList.remove('hidden');
4681 });
4682 });
4683
4684 // Canvas interactions (Pointer Events)
4685 // Unifies mouse + touch + pen, and makes edit-handle dragging work on phones.
4686 const _activePointers = new Map(); // pointerId -> { x, y, startX, startY, startTs, pointerType }
4687 let _panPointerType = null;
4688 let _panPrimaryPointerId = null;
4689 let _roiPointerId = null;
4690 let _dragPointerId = null;
4691 let _circlePointerId = null;
4692
4693 const _lastTap = {
4694     mouse: { ts: 0, x: 0, y: 0 },
4695     touch: { ts: 0, x: 0, y: 0 },
4696     pen: { ts: 0, x: 0, y: 0 },
4697     unknown: { ts: 0, x: 0, y: 0 }
4698 };
4699
4700 function pointerCanvasCoords(e) {
4701     const rect = elements.overlayCanvas.getBoundingClientRect();
4702     const scale = state.zoom / 100;
4703     return {
4704         x: (e.clientX - rect.left) / scale,
4705         y: (e.clientY - rect.top) / scale
4706     };
4707 }
4708
4709 function canvasHitRadius() {
4710     // Aim for ~18 screen px hit radius regardless of zoom.
4711     const scale = state.zoom / 100;
4712     const r = 18 / (scale || 1);
4713     return Math.max(6, Math.min(30, r));
4714 }
4715
4716 function beginEditHandleDragAt(coords) {
4717     const hit = canvasHitRadius();
4718
4719     if (state.editingLengthIndex !== null) {
4720         const len = state.lengths[state.editingLengthIndex];
4721         const dist1 = Math.hypot(coords.x - len.p1.x, coords.y - len.p1.y);
4722         const dist2 = Math.hypot(coords.x - len.p2.x, coords.y - len.p2.y);
4723         if (dist1 <= hit) {
4724             state.draggedLengthPoint = 'p1';
4725             return true;
4726         }
4727         if (dist2 <= hit) {
4728             state.draggedLengthPoint = 'p2';
4729             return true;
4730         }
4731     }
4732
4733     if (state.editingAreaIndex !== null) {
4734         const area = state.areas[state.editingAreaIndex];
4735         for (let i = 0; i < area.points.length; i++) {
4736             const p = area.points[i];
4737             const dist = Math.hypot(coords.x - p.x, coords.y - p.y);
4738             if (dist <= hit) {
4739                 state.draggedPointIndex = i;
4740                 return true;
4741             }
4742         }
4743     }
4744
4745     if (state.editingAngleIndex !== null) {
4746         const ang = state.angles[state.editingAngleIndex];
4747         const pts = [ang.p1, ang.p2, ang.p3];
4748         for (let i = 0; i < pts.length; i++) {
4749             const p = pts[i];
4750             const dist = Math.hypot(coords.x - p.x, coords.y - p.y);
4751             if (dist <= hit) {
4752                 state.draggedAnglePointIndex = i;
4753                 return true;

```

```

4754     }
4755   }
4756 }
4757
4758   return false;
4759 }
4760
4761 function handleCanvasTap(coords) {
4762   if (!state.image) return;
4763
4764   // Don't allow other interactions while drawing ROI
4765   if (state.isDrawingROI) return;
4766
4767   // Setting scale mode (exactly 2 points)
4768   if (state.isSettingScale) {
4769     if (state.scalePoints.length >= 2) return;
4770     state.scalePoints.push(coords);
4771     if (state.scalePoints.length === 2) {
4772       elements.setScaleBtn.textContent = 'Set Scale';
4773     }
4774     drawOverlay();
4775     return;
4776   }
4777
4778   // Length tool
4779   if (state.currentTool === 'length') {
4780     state.lengthPoints.push(coords);
4781     if (state.lengthPoints.length === 2) {
4782       const len = measureLength(state.lengthPoints[0], state.lengthPoints[1]);
4783       addLengthMeasurement(len);
4784       state.lengthPoints = [];
4785     }
4786     drawOverlay();
4787     return;
4788   }
4789
4790   // Angle tool (3 clicks; 2nd click is vertex)
4791   if (state.currentTool === 'angle') {
4792     // In edit mode, don't add new points with regular click
4793     if (state.editingAngleIndex !== null) return;
4794
4795     state.anglePoints.push(coords);
4796     if (state.anglePoints.length === 3) {
4797       addAngleMeasurement(state.anglePoints[0], state.anglePoints[1], state.anglePoints[2]);
4798       state.anglePoints = [];
4799     }
4800     drawOverlay();
4801     return;
4802   }
4803
4804   // Area tool
4805   if (state.currentTool === 'area') {
4806     // In edit mode, don't add new points with regular click
4807     if (state.editingAreaIndex !== null) return;
4808
4809     // Circle mode uses drag-to-draw; ignore tap-to-add behavior.
4810     if (state.areaMode !== 'polygon') return;
4811
4812     state.areaPoints.push(coords);
4813
4814     // Show close button when 3+ points
4815     if (state.areaPoints.length >= 3) {
4816       elements.closePolygonBtn.classList.remove('hidden');
4817       elements.closePolygonBtn.textContent = `Close Polygon (${state.areaPoints.length} points)`;
4818     }
4819
4820     drawOverlay();
4821     return;
4822   }
4823 }
4824
4825 function isTap(pointer) {
4826   const dt = Date.now() - pointer.startTs;
4827   if (dt > 300) return false;
4828   const dx = pointer.x - pointer.startX;
4829   const dy = pointer.y - pointer.startY;
4830   const dist = Math.hypot(dx, dy);
4831   // Use a screen-space threshold (~8px)
4832   return dist <= 8;
4833 }
4834

```

```

4835 function isDoubleTapForType(pointerType, x, y) {
4836     const t = _lastTap[pointerType] || _lastTap.unknown;
4837     const now = Date.now();
4838     const dt = now - t.ts;
4839     const dist = Math.hypot(x - t.x, y - t.y);
4840     return dt > 0 && dt < 300 && dist < 20;
4841 }
4842
4843 function recordTap(pointerType, x, y) {
4844     const slot = _lastTap[pointerType] || _lastTap.unknown;
4845     slot.ts = Date.now();
4846     slot.x = x;
4847     slot.y = y;
4848 }
4849
4850 function clearTap(pointerType) {
4851     const slot = _lastTap[pointerType] || _lastTap.unknown;
4852     slot.ts = 0;
4853     slot.x = 0;
4854     slot.y = 0;
4855 }
4856
4857 function getPanMidpoint() {
4858     let sumX = 0;
4859     let sumY = 0;
4860     let n = 0;
4861     for (const p of _activePointers.values()) {
4862         if (_panPointerType && p.pointerType !== _panPointerType) continue;
4863         sumX += p.x;
4864         sumY += p.y;
4865         n++;
4866     }
4867     if (!n) return null;
4868     return { x: sumX / n, y: sumY / n };
4869 }
4870
4871 function endPointerInteraction(pointerId) {
4872     if (_roiPointerId === pointerId) {
4873         _roiPointerId = null;
4874         // Finish ROI drawing
4875         if (state.isDrawingROI && state.roiStart) {
4876             finishROISelection();
4877         }
4878     }
4879
4880     if (_circlePointerId === pointerId) {
4881         _circlePointerId = null;
4882         state.isDrawingCircle = false;
4883         state.circleDraft = null;
4884     }
4885
4886     if (_dragPointerId === pointerId) {
4887         _dragPointerId = null;
4888         state.draggedPointIndex = null;
4889         state.draggedLengthPoint = null;
4890         state.draggedAnglePointIndex = null;
4891     }
4892
4893     if (_panPrimaryPointerId === pointerId) {
4894         _panPrimaryPointerId = null;
4895     }
4896
4897     // If we were touch/pen panning and fewer than 2 pointers remain, stop panning.
4898     if (state.isPanning && (_panPointerType === 'touch' || _panPointerType === 'pen')) {
4899         let remaining = 0;
4900         for (const p of _activePointers.values()) {
4901             if (p.pointerType === _panPointerType) remaining++;
4902         }
4903         if (remaining < 2) {
4904             state.isPanning = false;
4905             _panPointerType = null;
4906         }
4907     }
4908
4909     // If mouse panning and primary pointer released, stop.
4910     if (state.isPanning && _panPointerType === 'mouse' && !_panPrimaryPointerId) {
4911         state.isPanning = false;
4912         _panPointerType = null;
4913     }
4914
4915     const cursor = (state.editingAreaIndex !== null || state.editingLengthIndex !== null || state.editingAngleIndex !== null) ? 'crosshair' : 'default';

```



```

4916         ? 'pointer'
4917         : 'crosshair';
4918     elements.overlayCanvas.style.cursor = cursor;
4919 }
4920
4921 elements.overlayCanvas.addEventListener('pointerdown', (e) => {
4922     if (!state.image) return;
4923
4924     // Track pointers for multi-touch panning.
4925     _activePointers.set(e.pointerId, {
4926         x: e.clientX,
4927         y: e.clientY,
4928         startX: e.clientX,
4929         startY: e.clientY,
4930         startTs: Date.now(),
4931         pointerType: e.pointerType || 'unknown'
4932     });
4933
4934     // Capture so we keep receiving move/up even if finger leaves canvas.
4935     if (elements.overlayCanvas.setPointerCapture) {
4936         elements.overlayCanvas.setPointerCapture(e.pointerId);
4937     }
4938
4939     // Mouse pan: middle button OR Alt+Left
4940     if ((e.pointerType === 'mouse') && (e.button === 1 || (e.button === 0 && e.altKey))) {
4941         state.isPanning = true;
4942         _panPointerType = 'mouse';
4943         _panPrimaryPointerId = e.pointerId;
4944         state.lastPanX = e.clientX;
4945         state.lastPanY = e.clientY;
4946         elements.overlayCanvas.style.cursor = 'grabbing';
4947         e.preventDefault();
4948         return;
4949     }
4950
4951     // Touch/pen pan begins when 2 pointers are active.
4952     if ((e.pointerType === 'touch' || e.pointerType === 'pen')) {
4953         let count = 0;
4954         for (const p of _activePointers.values()) {
4955             if (p.pointerType === e.pointerType) count++;
4956         }
4957         if (count >= 2) {
4958             if (state.isDrawingCircle || state.circleDraft) {
4959                 state.isDrawingCircle = false;
4960                 state.circleDraft = null;
4961                 _circlePointerId = null;
4962                 drawOverlay();
4963             }
4964             state.isPanning = true;
4965             _panPointerType = e.pointerType;
4966             const mid = getPanMidpoint();
4967             if (mid) {
4968                 state.lastPanX = mid.x;
4969                 state.lastPanY = mid.y;
4970             }
4971             elements.overlayCanvas.style.cursor = 'grabbing';
4972             e.preventDefault();
4973             return;
4974         }
4975     }
4976
4977     // ROI drawing mode (single pointer)
4978     if (state.isDrawingROI && (e.button === 0 || e.pointerType !== 'mouse')) {
4979         const coords = pointerCanvasCoords(e);
4980         state.roiStart = coords;
4981         _roiPointerId = e.pointerId;
4982         e.preventDefault();
4983         return;
4984     }
4985
4986     // Edit handle dragging (single pointer)
4987     if ((e.button === 0 || e.pointerType !== 'mouse') && (state.editingLengthIndex !== null || state.editingAreaIndex !== null)) {
4988         const coords = pointerCanvasCoords(e);
4989         if (beginEditHandleDragAt(coords)) {
4990             _dragPointerId = e.pointerId;
4991             elements.overlayCanvas.style.cursor = 'move';
4992             e.preventDefault();
4993             return;
4994         }
4995     }

```

```

4996
4997 // Area: Circle drawing mode (single pointer drag)
4998 if ((e.button === 0 || e.pointerType !== 'mouse') && state.currentTool === 'area' && state.areaMode === 'circle' &&
state.editingAreaIndex === null && _circlePointerId === null) {
4999     const coords = pointerCanvasCoords(e);
5000     state.isDrawingCircle = true;
5001     state.circleDraft = { center: coords, radiusPx: 0 };
5002     _circlePointerId = e.pointerId;
5003     drawOverlay();
5004     e.preventDefault();
5005     return;
5006 }
5007 }, { passive: false });
5008
5009 elements.overlayCanvas.addEventListener('pointermove', (e) => {
5010     const p = _activePointers.get(e.pointerId);
5011     if (p) {
5012         p.x = e.clientX;
5013         p.y = e.clientY;
5014     }
5015
5016     // Panning
5017     if (state.isPanning) {
5018         if (_panPointerType === 'mouse') {
5019             if (e.pointerId !== _panPrimaryPointerId) return;
5020             state.panX += e.clientX - state.lastPanX;
5021             state.panY += e.clientY - state.lastPanY;
5022             state.lastPanX = e.clientX;
5023             state.lastPanY = e.clientY;
5024             updateCanvasTransform();
5025             e.preventDefault();
5026             return;
5027         }
5028
5029         // Touch/pen panning uses midpoint
5030         const mid = getPanMidpoint();
5031         if (mid) {
5032             state.panX += mid.x - state.lastPanX;
5033             state.panY += mid.y - state.lastPanY;
5034             state.lastPanX = mid.x;
5035             state.lastPanY = mid.y;
5036             updateCanvasTransform();
5037             e.preventDefault();
5038             return;
5039         }
5040     }
5041
5042     // ROI drawing
5043     if (state.isDrawingROI && state.roiStart && _roiPointerId === e.pointerId) {
5044         const coords = pointerCanvasCoords(e);
5045         const x = Math.min(state.roiStart.x, coords.x);
5046         const y = Math.min(state.roiStart.y, coords.y);
5047         const width = Math.abs(coords.x - state.roiStart.x);
5048         const height = Math.abs(coords.y - state.roiStart.y);
5049         state.roi = { x, y, width, height };
5050         drawOverlay();
5051         e.preventDefault();
5052         return;
5053     }
5054
5055     // Area: circle draft update
5056     if (state.isDrawingCircle && _circlePointerId === e.pointerId && state.circleDraft) {
5057         const coords = pointerCanvasCoords(e);
5058         const c = state.circleDraft.center;
5059         state.circleDraft.radiusPx = Math.hypot(coords.x - c.x, coords.y - c.y);
5060         drawOverlay();
5061         e.preventDefault();
5062         return;
5063     }
5064
5065     // Drag length endpoint
5066     if (_dragPointerId === e.pointerId && state.draggedLengthPoint !== null && state.editingLengthIndex !== null) {
5067         const coords = pointerCanvasCoords(e);
5068         const len = state.lengths[state.editingLengthIndex];
5069         len[state.draggedLengthPoint] = coords;
5070         drawOverlay();
5071         e.preventDefault();
5072         return;
5073     }
5074
5075     // Drag area vertex

```

```

5076     if (_dragPointerId === e.pointerId && state.draggedPointIndex !== null && state.editingAreaIndex !== null) {
5077         const coords = pointerCanvasCoords(e);
5078         const area = state.areas[state.editingAreaIndex];
5079         area.points[state.draggedPointIndex] = coords;
5080         drawOverlay();
5081         e.preventDefault();
5082         return;
5083     }
5084
5085     // Drag angle handle
5086     if (_dragPointerId === e.pointerId && state.draggedAnglePointIndex !== null && state.editingAngleIndex !== null) {
5087         const coords = pointerCanvasCoords(e);
5088         const ang = state.angles[state.editingAngleIndex];
5089         if (state.draggedAnglePointIndex === 0) ang.p1 = coords;
5090         if (state.draggedAnglePointIndex === 1) ang.p2 = coords;
5091         if (state.draggedAnglePointIndex === 2) ang.p3 = coords;
5092         ang.angleDeg = computeSelectedAngleDeg(ang.p1, ang.p2, ang.p3, !!ang.reflex);
5093         drawOverlay();
5094         e.preventDefault();
5095         return;
5096     }
5097     }, { passive: false });
5098
5099     function handlePointerUpOrCancel(e) {
5100         const p = _activePointers.get(e.pointerId);
5101         if (!p) {
5102             endPointerInteraction(e.pointerId);
5103             return;
5104         }
5105
5106         const pointerType = p.pointerType || 'unknown';
5107         const wasDragging = (_dragPointerId === e.pointerId);
5108         const wasROI = (_roiPointerId === e.pointerId);
5109         const wasCircle = (_circlePointerId === e.pointerId);
5110
5111         // Update pointer end position
5112         p.x = e.clientX;
5113         p.y = e.clientY;
5114
5115         // Remove pointer now; this affects multi-touch panning end logic.
5116         _activePointers.delete(e.pointerId);
5117
5118         // Circle mode: finalize or cancel circle drawing on pointer up/cancel.
5119         if (wasCircle && state.circleDraft) {
5120             const endCoords = pointerCanvasCoords(e);
5121             const center = state.circleDraft.center;
5122             const radiusPx = Math.hypot(endCoords.x - center.x, endCoords.y - center.y);
5123
5124             state.isDrawingCircle = false;
5125             state.circleDraft = null;
5126             _circlePointerId = null;
5127
5128             // Only commit if we are still in circle mode and this is a normal pointer-up.
5129             if (e.type === 'pointerup' && state.currentTool === 'area' && state.areaMode === 'circle' && radiusPx >= 2) {
5130                 addCircleAreaMeasurement(center, radiusPx);
5131             } else {
5132                 drawOverlay();
5133             }
5134
5135             clearTap(pointerType);
5136             endPointerInteraction(e.pointerId);
5137             e.preventDefault();
5138             return;
5139         }
5140
5141         // Consider tap ONLY if it wasn't a drag/ROI/pan
5142         const tappable = !state.isPanning && !wasDragging && !wasROI;
5143         if (tappable && isTap(p)) {
5144             const nowX = e.clientX;
5145             const nowY = e.clientY;
5146             const doubleTap = isDoubleTapForType(pointerType, nowX, nowY);
5147
5148             if (pointerType === 'touch' || pointerType === 'pen') {
5149                 // Touch/pen: on double-tap, close polygon without adding another point.
5150                 if (doubleTap) {
5151                     if (state.currentTool === 'area' && state.areaMode === 'polygon' && state.areaPoints.length >= 3) {
5152                         addAreaMeasurement(state.areaPoints);
5153                         state.areaPoints = [];
5154                     }
5155                     clearTap(pointerType);
5156                 } else {

```

```

5157         recordTap(pointerType, nowX, nowY);
5158         handleCanvasTap(pointerCanvasCoords(e));
5159     }
5160 } else {
5161     // Mouse: preserve old behavior (second click adds point; then we close if it was a double-click).
5162     const coords = pointerCanvasCoords(e);
5163     handleCanvasTap(coords);
5164     if (doubleTap) {
5165         if (state.currentTool === 'area' && state.areaMode === 'polygon' && state.areaPoints.length >= 3) {
5166             addAreaMeasurement(state.areaPoints);
5167             state.areaPoints = [];
5168         }
5169         clearTap(pointerType);
5170     } else {
5171         recordTap(pointerType, nowX, nowY);
5172     }
5173 }
5174 }
5175
5176 endPointerInteraction(e.pointerId);
5177 }
5178
5179 elements.overlayCanvas.addEventListener('pointerup', handlePointerUpOrCancel, { passive: false });
5180 elements.overlayCanvas.addEventListener('pointercancel', handlePointerUpOrCancel, { passive: false });
5181
5182 // Set crosshair cursor when image loaded
5183 elements.overlayCanvas.style.cursor = 'crosshair';
5184 // Prevent browser scrolling/zoom gestures from stealing pointer events on mobile.
5185 elements.overlayCanvas.style.touchAction = 'none';
5186 // Hint browser that these elements will transform frequently
5187 elements.mainCanvas.style.willChange = 'transform';
5188 elements.overlayCanvas.style.willChange = 'transform';
5189
5190 // Mouse wheel zoom
5191 elements.canvasWrapper.addEventListener('wheel', (e) => {
5192     if (!state.image) return;
5193
5194     e.preventDefault();
5195
5196     const zoomStep = 10;
5197     const delta = e.deltaY < 0 ? zoomStep : -zoomStep;
5198     const newZoom = Math.max(50, Math.min(500, state.zoom + delta));
5199
5200     if (newZoom !== state.zoom) {
5201         state.zoom = newZoom;
5202         elements.zoomSlider.value = state.zoom;
5203         elements.zoomValue.textContent = state.zoom + '%';
5204         updateCanvasTransform();
5205     }
5206 }, { passive: false });
5207
5208 // Keyboard controls for panning
5209 document.addEventListener('keydown', (e) => {
5210     if (!state.image) return;
5211
5212     const panStep = 50;
5213     let handled = false;
5214
5215     switch(e.key) {
5216         case 'ArrowLeft':
5217             state.panX += panStep;
5218             handled = true;
5219             break;
5220         case 'ArrowRight':
5221             state.panX -= panStep;
5222             handled = true;
5223             break;
5224         case 'ArrowUp':
5225             state.panY += panStep;
5226             handled = true;
5227             break;
5228         case 'ArrowDown':
5229             state.panY -= panStep;
5230             handled = true;
5231             break;
5232     }
5233
5234     if (handled) {
5235         e.preventDefault();
5236         updateCanvasTransform();
5237     }

```

```

5238 });
5239
5240 // Recenter on viewport changes (mobile orientation/resize)
5241 let _resizeTimer = null;
5242 function _scheduleTransformUpdate() {
5243   if (!state.image) return;
5244   if (_resizeTimer) clearTimeout(_resizeTimer);
5245   _resizeTimer = setTimeout(() => {
5246     // Recompute fit on viewport changes for mobile usability
5247     fitToView();
5248   }, 150);
5249 }
5250 window.addEventListener('resize', _scheduleTransformUpdate);
5251 window.addEventListener('orientationchange', _scheduleTransformUpdate);
5252 if (window.visualViewport) {
5253   window.visualViewport.addEventListener('resize', _scheduleTransformUpdate);
5254 }
5255
5256 // Drag and Drop support
5257 elements.canvasWrapper.addEventListener('dragover', (e) => {
5258   e.preventDefault();
5259   e.stopPropagation();
5260   elements.canvasWrapper.classList.add('drag-over');
5261 });
5262
5263 elements.canvasWrapper.addEventListener('dragleave', (e) => {
5264   e.preventDefault();
5265   e.stopPropagation();
5266   // Only remove class if leaving the wrapper itself, not child elements
5267   if (e.target === elements.canvasWrapper) {
5268     elements.canvasWrapper.classList.remove('drag-over');
5269   }
5270 });
5271
5272 elements.canvasWrapper.addEventListener('drop', (e) => {
5273   e.preventDefault();
5274   e.stopPropagation();
5275   elements.canvasWrapper.classList.remove('drag-over');
5276
5277   const files = e.dataTransfer.files;
5278   if (files.length > 0) {
5279     loadImage(files[0]);
5280   }
5281 });
5282
5283 // Mobile-friendly tap to load (fallback when drag-drop isn't available)
5284 elements.canvasWrapper.addEventListener('click', () => {
5285   if (!state.image && elements.imageInput) {
5286     elements.imageInput.click();
5287   }
5288 });
5289 elements.canvasWrapper.addEventListener('touchend', () => {
5290   if (!state.image && elements.imageInput) {
5291     elements.imageInput.click();
5292   }
5293 });
5294
5295 // Initial button state
5296 updateStatsButtonsVisibility();
5297 updateGuidance();
5298 </script>
5299 </body>
5300 </html>

```